

Índice

1. CONOCIMIENTOS BÁSICOS DE MICROCONTROLADORES.....	7
1.1 DEFINICIÓN DE UN MICROCONTROLADOR.	8
1.2 APLICACIONES.....	8
1.3 MICROCONTROLADORES Y MICROPROCESADORES.....	10
1.3.1 Microprocesador.....	10
Arquitectura.....	10
Arquitectura Harvard y von Neumman.....	11
1.3.2 Microcontroladores.....	13
Partes del Microcontrolador.....	13
1. Memoria del programa.....	13
2. Memoria de datos.....	14
3. Líneas de E/S para los controladores de periféricos.....	15
4. Recursos Auxiliares.....	15
1.3.3 ¿Cuándo usar un microprocesador y un microcontrolador?.....	16
2. EL PIC16F84A.....	17
2.1 DEFINICIÓN.....	18
2.2 TERMINALES DEL PIC16F84A.....	19
2.3 FRECUENCIA DE FUNCIONAMIENTO (CICLOS DE RELOJ).	20
2.4 RESET.....	21
3. PRIMER CONTACTO CON EL PIC16F84A.....	23
3.1 MARCO TEÓRICO.....	24
3.1.1 Lenguaje de programación.....	24
3.1.2 Instalación del MicroCode Studio.....	24
3.1.3 Instalación del Programador PIC-600.....	25
3.2 COMANDOS SUGERIDOS.....	28
3.3 PRÁCTICA DE LABORATORIO NO. 1. - PARPADEO DE UN LED.	30
3.4 PROBLEMAS RETO.	39
1. Semáforo.....	39
2. Secuencia libre:.....	40
4. COMANDOS BÁSICOS DE PICBASIC.....	41
4.1 MARCO TEÓRICO.....	42
4.1.1 Comando básicos.....	42
4.2 PRÁCTICAS DE LABORATORIO NO.2.- LEER EL ESTADO DE UN PUERTO.....	53
5. COMUNICACIÓN SERIAL ENTRE EL PIC Y LA PC.....	57
5.1 MARCO TEÓRICO.....	58
5.1.1 Comunicación Serial.....	58
5.1.2 Comandos sugeridos.....	61
5.2 PRÁCTICA DE LABORATORIO NO.3.-COMUNICACIÓN SERIAL.	65
5.3 PROBLEMAS RETO.	73
1. Comunicación de la PC al PIC.....	73
6. PROGRAMACIÓN APLICADA USANDO DISPLAYS DE 7 SEGMENTOS.....	76
6.1 MARCO TEÓRICO.....	77
6.1.1 El display de 7 segmentos.....	77
6.1.2 Multiplexación con Displays.....	78

6.1.3 Comandos sugeridos.....	81
6.2 PRÁCTICAS DE LABORATORIO No.4.- CONVERTIDOR BCD A 7 SEGMENTOS.	84
6.3 PROBLEMAS RETO.	89
1. Anuncio estático.....	89
2. Anuncio en Movimiento.....	90
3. Contador digital de 0 a 99.	90
4. Contador ascendente y descendente de 0-99.	91
5. Contador de 0 a 9999.	91
7. PROGRAMACIÓN APLICADA USANDO LA PANTALLA LCD.	92
7.1 MARCO TEÓRICO	93
7.1.1 La pantalla LCD 16X2	93
7.1.2 Terminales de la pantalla LCD de 16x2.....	94
7.1.3 Comandos sugeridos.....	94
Locaciones de Memoria en la pantalla LCD de 16x2	94
7.2 PRÁCTICAS DE LABORATORIO No.5.- LA PANTALLA LCD.	97
7.3 PROBLEMA RETO.	103
1. Desplazamiento de una palabra en la pantalla LCD.....	103
2. Contador de personas.....	103
8. PROYECTOS FINALES.....	106
8.1 PROYECTOS FINAL NO. 1.- MEDICIÓN DE DISTANCIA CON SENSOR ULTRASÓNICO.	107
8.1.1 Marco teórico	107
Definición.....	107
EI SRF04	108
Características técnicas.....	109
8.1.2 Comandos sugeridos.....	110
8.2 PROYECTOS FINAL NO. 2.- CISTERNA-TINACO	115
8.2.1 Marco teórico	115
Comandos sugeridos.....	115
8.3 PROYECTOS FINAL NO.3.- CERRADURA ELECTRÓNICA	123
8.3.1 Marco teórico	123
Comandos sugeridos.....	123

Índice por prácticas, retos y proyectos

Práctica No.1.- Parpadeo de un LED.	30
Problemas reto	
1. Semáforo	39
2. Secuencia libre.	40
Práctica No.2.- Leer el estado de un puerto.	53
Práctica No.3.- Comunicación serial.	65
Problemas reto	
1. Comunicación de la PC al PIC.	73
Práctica No.4.- Convertidor BCD a 7 segmentos.	84
Problemas reto	
1. Anuncio estático.	89
2. Anuncio en movimiento.	90
3. Contador digital de 0 a 99	90
4. Contador ascendente y descendente de 0 a 99.	91
5. Contador de 0 a 9999.	91
Práctica No. 5.- La pantalla LCD.	97
1. Desplazamiento de una palabra en la pantalla LCD.	103
2. Contador de personas.	103
<u>Proyectos finales.</u>	
1. Medición de distancia con un sensor ultrasónico.	107
2. Cisterna-Tinaco.	115
3. Cerradura electrónica.	123

Índice de figuras.

FIGURA. 1. APLICACIONES DE LOS MICROCONTROLADORES.	8
FIGURA. 2. ARQUITECTURA DE UN MICROPROCESADOR.	11
FIGURA 1. ARQUITECTURA VAN NEUMANN.	11
FIGURA 2. ARQUITECTURA HARVARD.	12
FIGURA 3. ESTRUCTURA DE UN MICROCONTROLADOR.....	13
FIGURA 4. TERMINALES DEL MICROCONTROLADOR.....	19
FIGURA 5. OSCILADOR CON CRISTAL XT.....	20
FIGURA 6. CONEXIÓN DE UN CRISTAL OSCILADOR XT EN LA PRÁCTICA.	21
FIGURA 7. OSCILADOR RC.....	21
FIGURA 8. CIRCUITO CON PUSH-BUTTON PARA LA RENICIALIZACIÓN DEL PIC.....	22
FIGURA 9. CIRCUITO PARA PROVOCAR LA RENICIALIZACIÓN "POWER ON RESET"	22
FIGURA 10. ASISTENTE DE INSTALACIÓN DEL MICROCODE STUDIO.....	24
FIGURA 11. INSTALACIÓN DE LAS LIBRERIAS DE MICROCODE STUDIO.....	25
FIGURA 12. ASISTENTE DE INSTALACIÓN DEL PIC-600.	26
FIGURA 13. INSTALACIÓN DEL DRIVER PARA EL MODULO PROGRAMADOR PIC-600.....	26
FIGURA 14. SOFTWARE PIC-600.....	27
FIGURA 15. REFERENCIA AL PUERTO DEL PIC EN LA PROGRAMACIÓN.....	29
FIGURA 16. CIRCUITO DE LA PRÁCTICA 1.....	31
FIGURA 17. MICROCODE STUDIO.	31
FIGURA 18. CONFIGURACIÓN DEL MICROCODE STUDIO.....	32
FIGURA 19. ARCHIVOS ".BAS".	33
FIGURA 20. COMPILACIÓN DE UN PROGRAMA.	33
FIGURA 21. PIC-600	34
FIGURA 22. CONFIGURACIÓN DEL PIC-600.	34
FIGURA 23. MODULO PROGRAMADOR DE STEREN PIC-600.	35
FIGURA 24. CONEXIÓN DEL PIC-600 A LA PC.....	35
FIGURA 25. SOFTWARE PIC-600 LISTO PARA INICIAR LA PROGRAMACIÓN DEL PIC.	36
FIGURA 26. EL PIC-600 IDENTIFICA EL PIC16F84A.	36
FIGURA 27. SELECCIÓN DEL ARCHIVO ".HEX".	37
FIGURA 28. CARGAR EL PROGRAMA ".HEX" AL PIC16F84A.....	37
FIGURA 29. CIRCUITO PARA EL SEMÁFORO.	40
FIGURA 30. COMENTARIOS EN LA PROGRAMACIÓN PICBASIC.	42
FIGURA 31. CIRCUITO DE CONTROL DE LA PRÁCTICA 2.....	54
FIGURA 32. CONECTOR DB9	59

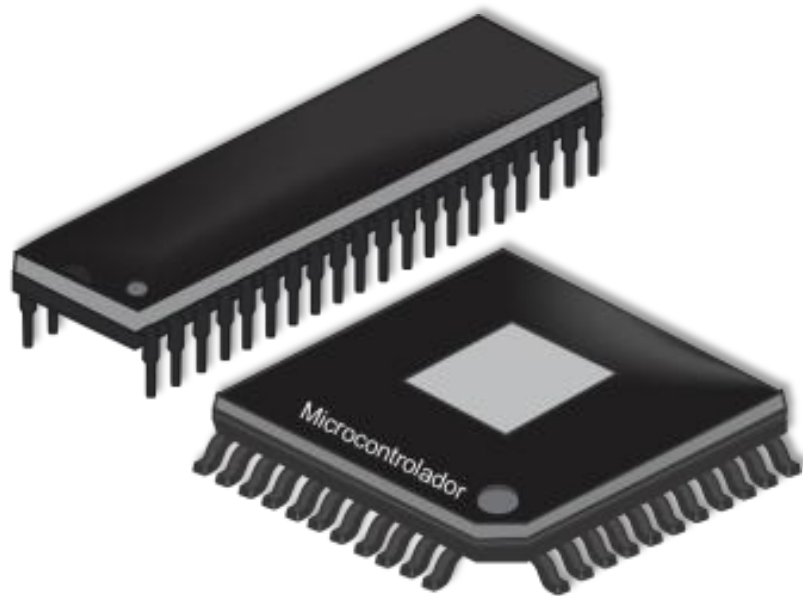
FIGURA 33. ENVÍO DE DATOS DEL PIC A LA PC POR CONEXIÓN SERIAL.....	60
FIGURA 34. RECEPCIÓN DE DATOS DE LA PC AL PIC POR CONEXIÓN SERIAL.	60
FIGURA 35. INCLUIR LIBRERÍAS EN LA PROGRAMACIÓN.....	61
FIGURA 36. COMUNICACIÓN PIC-PC.....	66
FIGURA 37. CONEXIÓN DEL CABLE SERIAL A LA PC.	66
FIGURA 38. PUERTOS COM INSTALADOS EN LA PC.....	67
FIGURA 39. CONFIGURACIÓN DEL PUERTO COM.....	68
FIGURA 40. BOTÓN PARA ACTIVAR LA COMUNICACIÓN SERIAL.	69
FIGURA 41. SERIAL COMMUNICATION WINDOW.	69
FIGURA 42. CONFIGURACIÓN DE LA HERRAMIENTA “SERIAL COMMUNICATION WINDOW”	70
FIGURA 43. CONEXIÓN ENTRE EL PIC Y LA PC.	70
FIGURA 44. CONEXIÓN ESTABLECIDA ENTRE EL PIC Y LA PC.	71
FIGURA 45. DIAGRAMA DE CONEXIÓN PARA EL ENVÍO DE DATOS DE LA PC AL PIC.....	73
FIGURA 46. ENVÍO DE UNA VARIABLE POR COMUNICACIÓN SERIAL.	74
FIGURA 47. ENVÍO DE DATOS DE LA PC AL PIC.....	75
FIGURA 48. ESTRUCTURA INTERNA DE UN DISPLAY DE 7 SEGMENTOS.....	77
FIGURA 51. . ESTRUCTURA DE UN DISPLAY DE CÁTODO COMÚN.	77
FIGURA 52. ESTRUCTURA INTERNA DE UN DISPLAY DE ÁNODO COMÚN.	77
FIGURA 49. LOS 7 SEGMENTOS DEL DISPLAY.....	78
FIGURA 50. MULTIPLEXACIÓN CON DISPLAYS DE 7 SEGMENTOS.....	79
FIGURA 51. DIAGRAMAS DE CONEXIÓN PARA UN DISPLAY DE ÁNODO Y CÁTODO COMÚN.....	80
FIGURA 52. MULTIPLEXEO CON 4 DISPLAYS.	82
FIGURA 53. DIAGRAMA DEL CIRCUITO PARA UN CONVERTIDOS BCD A 7 SEGMENTOS.	85
FIGURA 54. PANTALLA LCD DE 16X2.....	93
FIGURA 5955. CONFIGURACIÓN DE TERMINALES DE LA PANTALLA LCD DE 16X2.....	94
FIGURA 60. LOCALIDADES DE MEMORIA DE UNA PANTALLA LCD DE 16X2.....	94
FIGURA 61. CONEXIÓN DE 4 BITS DE UNA PANTALLA LCD DE 16X2.	98
FIGURA 62. PRIMER PROGRAMA EN LA PANTALLA LCD.....	99
FIGURA 63.DIAGRAMA DE UN DETECTOR DE PERSONAS.	105
FIGURA 64.FUNCIONAMIENTO DE UN SENSOR ULTRASÓNICO.....	107
FIGURA 65. SENSOR ULTRASÓNICO SRF04.	108
FIGURA 66. DIAGRAMA DE TIEMPOS DEL SENSOR SRF04.....	109
FIGURA 67. TERMINALES DEL SENSOR SRF04.	110
FIGURA 68. DIAGRAMA DE CONEXIÓN DEL SENSOR SRF04.....	113
FIGURA 69. ESQUEMA DEL SISTEMA CISTERNA-TINACO.....	118

FIGURA 70. DIAGRAMA DEL SISTEMA CISTERNA-TINACO.	121
FIGURA 71.ESQUEMA DE LA CERRADURA ELECTRÓNICA. . .	124

Índice de tablas

TABLA 1. CARACTERÍSTICAS DEL PIC16F84A	18
TABLA 2. SECUENCIA DEL SEMÁFORO.	39
TABLA 3. TIPOS DE VARIABLES EN PBP.	43
TABLA 4. LÍMITES PARA TIPOS DE ARREGLOS EN PBP.....	44
TABLA 5. CONFIGURACIÓN DE PUERTOS COMO SALIDAS Y ENTRADAS.	44
TABLA 6. COMANDO DEFINE.	50
TABLA 7. OPERADORES MATEMÁTICOS.	51
TABLA 8. OPERADORES LÓGICOS.....	51
TABLA 9. COMANDOS BÁSICOS EN PBP.....	52
TABLA 10. TERMINALES DEL CONECTOR DB9.....	58
TABLA 11. COMANDO INCLUDE.....	62
TABLA 12. SECUENCIA DE VALORES PARA FORMAR NÚMEROS EN EL DISPLAY.	78
TABLA 13. DECODIFICACIÓN DEL COMANDO DCD.....	83
TABLA 14. TABLA DE VALORES PARA GENERAR EL CÓDIGO DE BCD A 7 SEGMENTOS.	86
TABLA 15. VALORES DECIMALES, BINARIOS Y HEXADECIMALES.	86
TABLA 15. LOCALIDADES DE MEMORIA DE UNA PANTALLA LCD DE 16X2.....	95
TABLA 16. COMANDOS BÁSICOS PARA LA PANTALLA LCD.....	96
TABLA 17. CARACTERÍSTICAS TÉCNICAS DEL SENSOR SRF04.....	109
TABLA 18. CARACTERÍSTICAS TÉCNICAS DEL SENSOR SRF04.....	118
TABLA 19. CONDICIONES QUE DEBE CUMPLIR EL SISTEMA CISTERNA-TINACO.....	120

1. Conocimientos básicos de Microcontroladores



1.1 Definición de un Microcontrolador.

Es un circuito integrado programable que contiene todos los componentes de un computador. Se emplea para controlar el funcionamiento de una tarea determinada, y debido a su reducido tamaño, suele ir incorporado en el propio dispositivo al que gobierna. Esta última característica es la que le confiere la denominación de “controlador incrustado” (*embedded controller*).

1.2 Aplicaciones.

Siendo dispositivos dedicados los microcontroladores tienen un gran campo de aplicación en diferentes ramas de la industria, (*figura 1*). Se mencionan algunas de sus aplicaciones:

- Dispositivos de medición: Es muy común encontrar microcontroladores en dispositivos de medición digitales como: multímetros, luxómetros, termómetros, termohigrómetros, etc., instrumentos de medición dedicados simplemente a medir la variable específica.

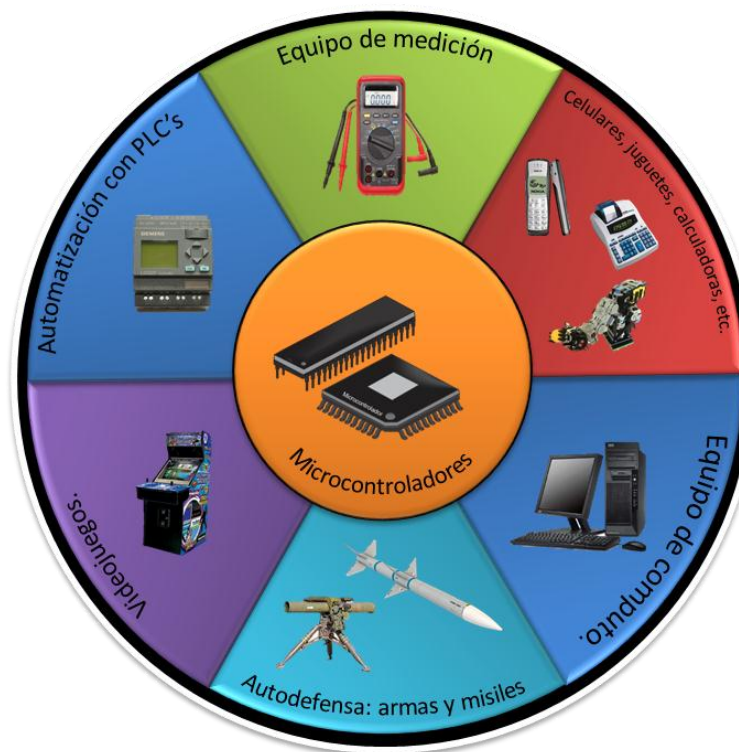


Figura. 1. Aplicaciones de los Microcontroladores.

- Sistemas de automatización industriales: Los Controladores Lógicos Programables (PLC's) por sus siglas en inglés, son usados para automatizar procesos industriales e incorporan un microprocesador en su arquitectura.
- Videojuegos: En videoconsolas como SNK Neo-Geo y SEGA Mega Drive se pueden encontrar microcontroladores especializados para el sonido. Estas máquinas de arcade usan nuevas versiones de uno de los primeros microcontroladores que salieron al mercado: el "ZILOG Z80".
- Computadoras: Algunos dispositivos de los equipos de cómputo incorporan microcontrolares, programados desde su fabricación para operaciones concretas como: la unidad de CD-ROM, el disco duro, tarjeta de sonido, etc. De igual manera también se encuentran en dispositivos de salida como impresoras y lectores de código de barras
- Sector de defensa: De igual manera se encuentran en los circuitos electrónicos de misiles y armas antitanques. Sirven para la toma de decisiones como detonarse en un momento determinado, no activarse sin un código, seguir objetivos, permiten ser operados a distancia, etc.
- Otras aplicaciones: Calculadoras digitales, sistemas de alarmas, relojes digitales, juguetes, teléfonos celulares, etc.

1.3 Microcontroladores y Microprocesadores.

Es muy común en algunas personas confundir los conceptos de microcontrolador y microprocesador, inclusive usar ambas palabras para referirse a lo mismo. Sin embargo cada uno de ellos tienen características diferentes como: su arquitectura, sus aplicaciones hasta el precio en el mercado.

1.3.1 Microprocesador.

Un microprocesador o procesador, es un circuito integrado que contiene la Unidad Central de Proceso (UCP o CPU por sus siglas en ingles) y es el encargado de procesar las operaciones aritmético-lógicas necesarias para el funcionamiento de una computadora, por eso es considerado como el cerebro de esta.

Arquitectura

En el microprocesador se encuentran los buses de dirección, datos y control, los cuales son accesibles por medio de las terminales que asoman al exterior del mismo, permitiendo la conexión de la Memoria RAM, la ROM, la memoria FLASH, así como los diferentes módulos que permiten acoplar dispositivos de E/S, (figura 2). Su estructura al fin forma una computadora totalmente funcional que puede ser adaptable a cualquier tipo de aplicación dependiendo de las necesidades del fabricante o del usuario. En otras palabras la arquitectura de un microprocesador fue desarrollada para permitir el acoplamiento de varios circuitos integrados dependiendo de la aplicación a la que será destinada, es por eso que es considerado un *sistema abierto*.

Definiciones:

Bus: Medio de transmisión digital (conjunto de cables o pistas en un circuito impreso) que intercomunica al procesador con los diferentes periféricos, permitiendo la transmisión de los datos (conjuntos de bits) necesarios para realizar un proceso.

Bus de datos: Bus bidireccional responsable de enviar los datos o instrucciones hacia los demás periféricos para realizar un proceso. La velocidad de la transmisión de datos depende de la capacidad del procesador, puede ser 8, 16, 32 o 64 bits.

Bus de control: Bus bidireccional responsable de Interpretar las instrucciones enviadas por el bus de datos para posteriormente sincronizar o coordinar las actividades de cada uno de los periféricos haciendo uso de las IRQ (Petición de Interrupción) y las DMA (Acceso Directo a Memoria).

Bus de direcciones: Bus unidireccional que envía información sobre las direcciones o localidades de memoria que indican donde se leerán o escribirán las instrucciones que viajan a través del bus de datos.

El bus de datos envía las instrucciones para realizar los procesos, el de control coordina estos procesos y el de dirección indica las localidades de memoria donde se irán almacenando las instrucciones necesarias para realizarlos.

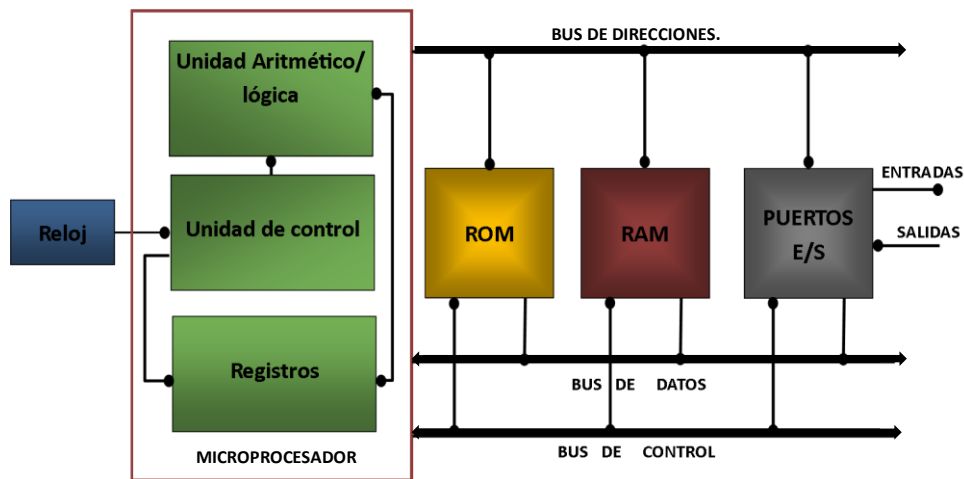


Figura. 2. Arquitectura de un Microprocesador.

Arquitectura Harvard y von Neumann

La necesidad de conseguir elevados rendimientos en el procesamiento de las instrucciones ha desembocado en el empleo generalizado de procesadores de arquitectura *Harvard* frente a los tradicionales que seguían la arquitectura de *von Neumann*. Esta última se caracteriza porque la UCP (Unidad Central de Proceso) se conectaba con una memoria única, donde coexistían datos e instrucciones, a través de un sistema de instrucciones, (figura 3).

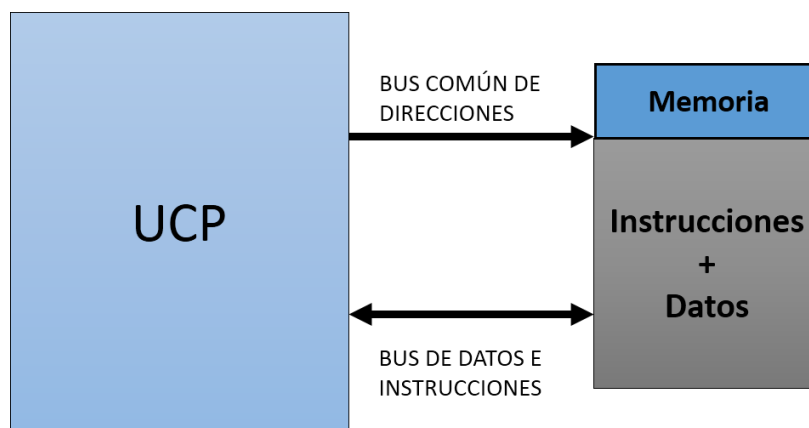


Figura 1. Arquitectura von Neumann.

En la arquitectura de “von Neumann” la UCP se comunicaba a través de un sistema de buses con la Memoria, donde se guardan las instrucciones y los datos.

En la arquitectura *Harvard* son independientes la memoria de instrucciones y la memoria de datos y cada una dispone de su propio sistema de buses para el acceso. Esta dualidad, además de propiciar el paralelismo, permite la adecuación del tamaño de las palabras y los buses a los requerimientos específicos de las instrucciones y los datos. También la capacidad de cada memoria es diferente, (figura 4).

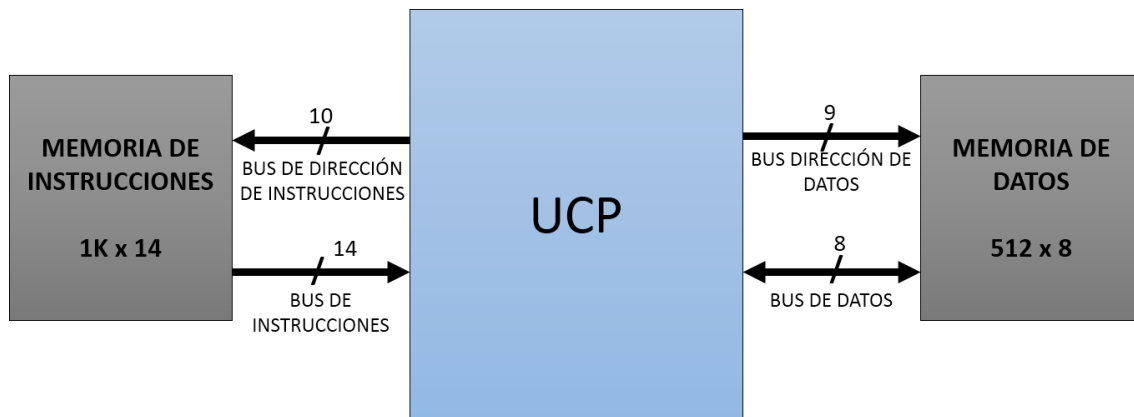


Figura 2. Arquitectura Harvard.

En la arquitectura “*Harvard*” la memoria de instrucciones y la de datos son independientes, lo que permite optimizar sus características y propiciar el paralelismo. En la figura 4, la memoria de instrucciones tiene 1 K posiciones de 14 bits cada una, mientras que la de datos sólo dispone de 512 posiciones de un byte.

El procesador de los modernos microcontroladores responde a la arquitectura *RISC* (*Computadores de Juego de Instrucciones Reducido*), que se identifica por poseer un repertorio de instrucciones máquina pequeño y simple, de forma que la mayor parte de las instrucciones se ejecuta en un ciclo de instrucción.

Otra aportación frecuente que aumenta el rendimiento del computador es el fomento del paralelismo implícito, que consiste en la segmentación del procesador (pipe-line) descomponiéndolo en etapas para poder procesar una instrucción diferente en cada una de ellas y trabajar con varias a la vez.

El alto rendimiento y elevada velocidad que alcanzan los modernos procesadores, como el que poseen los microcontroladores PIC, se debe a la conjunción de tres técnicas:

- *Arquitectura Harvard*
- *Computador tipo RISC.*
- *Segmentación.*

1.3.2 Microcontroladores

Los microcontroladores por otro lado desde su fabricación son diseñados para contener internamente todos sus componentes, encapsulando en forma de circuito integrado: la unidad Central de Proceso, la ROM, la memoria Flash, los buses de direcciones, datos y control, etc., asomando únicamente las terminales de E/S. Es como tener una computadora completa pero en menor escala y sin tanta capacidad de procesamiento como los microprocesadores, (*figura 5*). Además su diseño como tal provoca un inconveniente al momento de requerir más recursos, puesto que el usuario está sujeto a las condiciones de fábrica: no hay posibilidad de modificación o escalabilidad. Debido a las características mencionadas es que el microcontrolador es denominado un *sistema cerrado*.

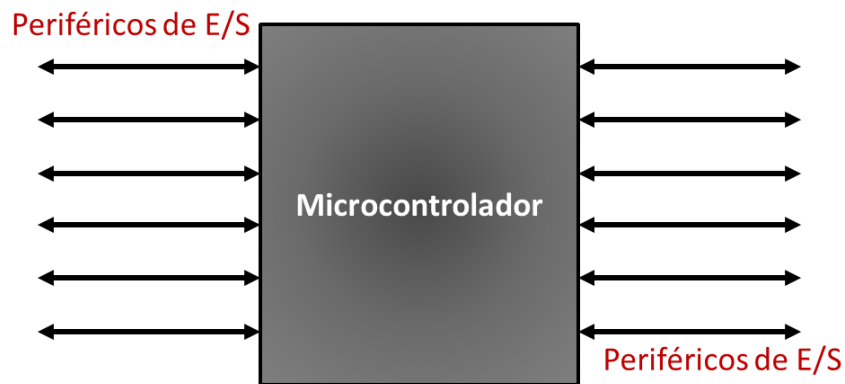


Figura 3. Estructura de un microcontrolador.

Partes del Microcontrolador

1. Memoria del programa

El microcontrolador está diseñado para que en su memoria de programa se almacenen todas las instrucciones del programa de control. No hay posibilidad de utilizar memorias externas de ampliación.

Como el programa a ejecutar siempre es el mismo, debe de estar grabado en forma permanente. Los tipos de memoria adecuados para soportar esta función admiten cinco versiones diferentes:

- **ROM con máscara:** En este tipo de memoria el programa se graba en el chip durante el proceso de su fabricación mediante el uso de “máscaras”. Los altos costes de diseño e instrumental sólo aconsejan usar este tipo de memoria cuando se precisan series muy grandes.
- **EPROM:** La grabación de esta memoria se realiza mediante un dispositivo físico gobernado desde un computador personal, que recibe el nombre de grabador. En la superficie de la cápsula del microcontrolador existe una ventana de cristal por la que se puede someter al chip de la memoria a rayos ultravioletas para producir su borrado y emplearla nuevamente.
- **OTP (Programable una vez):** Este modelo de memoria sólo se puede grabar una vez por parte del usuario, utilizando el mismo procedimiento que con la memoria EPROM. Posteriormente no se puede borrar. Su bajo precio y sencillez de la grabación aconsejan este tipo de memoria para prototipos finales y series de producción cortas.
- **EEPROM:** La grabación es similar a las memorias OTP y EPROM, pero el borrado es mucho más sencillo al poderse efectuar de la misma forma que el grabado, es decir, eléctricamente. Sobre el mismo zócalo del grabador puede ser programada y borrada tantas veces como se quiera, lo cual la hace ideal en la enseñanza y en la creación de nuevos proyectos.
- **FLASH:** Se trata de una memoria no volátil, de bajo consumo, que puede escribir y borrar en circuito al igual que las EEPROM, pero suelen disponer de mayor capacidad que estas últimas. El borrado sólo es posible con bloques completos y no se puede realizar sobre posiciones concretas.

Son muy recomendables en aplicaciones en las que sea necesario modificar el programa a lo largo de la vida del producto, como consecuencia del desgaste o cambios de piezas, como sucede con los vehículos.

2. Memoria de datos

Los datos que manejan los programas varían continuamente, y esto exige que la memoria que les contiene debe de ser de lectura y escritura, por lo que la memoria RAM estática (SRAM) es la más adecuada, aunque sea volátil.

Hay microcontroladores que también disponen como memoria de datos una de lectura y escritura no volátil, del tipo EEPROM. De esta forma, un corte en el suministro de la alimentación no ocasiona la pérdida de la información, que esta disponible al reiniciarse el programa.

La memoria tipo EEPROM y la tipo FLASH pueden escribirse y borrarse eléctricamente. Sin necesidad de sacar el circuito integrado del zócalo del grabador pueden ser escritas y borradas numerosas veces.

3. Líneas de E/S para los controladores de periféricos.

A excepción de dos patitas destinadas a recibir la alimentación, otras dos para el cristal de cuarzo, que regula la frecuencia de trabajo, y una más para provocar el Reset, Las restantes patitas del microcontrolador sirven para soportar su comunicación con los periféricos externos que controla.

Las líneas de E/S que se adaptan con los periféricos manejan información en paralelo y se agrupan en conjuntos de ocho, que reciben el nombre de *Puertas*. Hay modelos con líneas que soportan la comunicación en serie, otros disponen de conjuntos de líneas que implementan puertas de comunicación para diversos protocolos, como el I²C, el USB, etc.

4. Recursos Auxiliares

Según las aplicaciones a las que orienta el fabricante cada modelo de microcontrolador, incorpora una diversidad de complementos que refuerzan la potencia y flexibilidad del dispositivo. Entre los recursos más comunes se citan los siguientes:

- a) *Circuito reloj*, encargado de generar los impulsos que sincronizan el funcionamiento de todo el sistema.
- b) *Temporizadores*, orientados a controlar tiempos.
- c) *Perro guardián (Watchdog)*, destinado a provocar una reinicialización cuando el programa queda bloqueado.
- d) *Conversores AD y DA*, para poder recibir y enviar señales analógicas.
- e) *Comparadores analógicos*, para verificar el valor de una señal analógica.
- f) *Sistema de protección ante fallos de alimentación*.

g) *Estado de Reposo*, en el que el sistema queda “congelado” y el consumo de energía se reduce al mínimo. Información obtenida de (José M. Angulo Usategui, 2003).

1.3.3 ¿Cuándo usar un microprocesador y un microcontrolador?

Los microcontroladores debido a sus capacidades son empleados para desarrollar tareas específicas, por tal motivo son considerados como *dispositivos dedicados* a diferencia del microprocesador que es adaptable a múltiples aplicaciones. Con lo comentado hasta ahora se pensaría que el microcontrolador se ve superado en todo por el procesador, y puede poner en duda el uso del primero. Sin embargo cada uno juega un papel importante en el uso de las tecnologías, por ejemplo, supóngase la elaboración de una calculadora básica, un dispositivo que lo único que requiere es realizar operaciones aritméticas y mostrar los dígitos en una pantalla LCD. Un microcontrolador sería suficiente para realizar esta tarea, sin embargo un procesador también puede ser una opción. Ahora imagínese usar todo un sistema de cómputo basado en un procesador para fabricar un dispositivo que su única tarea sea la de resolver operaciones aritméticas, sería un despilfarro de recursos e incluso el precio de la calculadora fabricada sería mucho mayor, esto se debe a que los microprocesadores son mucho más costosos que los microcontroladores.

Ahora suponga una PC, un equipo que el usuario usa para ver videos, escuchar música, escribir documentos, navegar en internet, etc., un microcontrolador no tendría la capacidad de realizar todas estas tareas, por lo que se tendría que fabricar un microcontrolador demasiado potente que encapsule dentro de él lo necesario para cumplir las exigencias de las diferentes aplicaciones. Esto se traduciría en un microcontrolador de muy elevado precio y aun así podría si quiera superar las características de los procesadores actuales. Por lo tanto, la solución lógica sería usar un procesador debido a sus características de adaptabilidad a cualquier aplicación.

2. El PIC16F84A



2.1 Definición

El " PIC 16F84 " es un microcontrolador del tipo EEPROM (Electrically-Erasable Programmable Read-Only Memory) Rom Programable y Borrable Eléctricamente de 1 KBytes, con memoria de programa tipo FLASH, lo que representa gran facilidad en el desarrollo de prototipos y en su aprendizaje ya que no se requiere de borrado con luz ultravioleta como las versiones EPROM (Erasable Programmable Read-Only Memory), en su lugar permite reprogramarlo nuevamente sin ser borrado con anterioridad. Por esta razón, lo usaremos en la mayoría de aplicaciones que se desarrollan a lo largo del estudio.

Algunas de sus características principales se muestran en la *tabla 1*:

Tabla 1. Características del PIC16F84A

PIC16F84A	
Alimentación	- De 4 a 5,5 v en configuración de oscilador XT, RC y LP - De 4,5 a 5.5 v en configuración de oscilador HS.
Consumo típico	- De 1,8 a 4.5 mA en configuración de oscilador RC y XT (FOSC=4 MHz, VDD=5,5V). - de 3 a 10 mA en configuración de oscilador RC y XT durante la programación de la FLASH (FOSC=4MHz, VDD=5,5V).
Memoria Flash	1024 palabra de 14 bits.
Memoria RAM	Dividida en 2 áreas: 22 registros de propósito específico (SFR) y 68 de propósito general (GPR) como memoria de datos.
Bus de datos	8 bits.
Ciclos de escritura/borrado	1000 ciclos
Tiempo de almacenamiento de la memoria EEPROM	40 años de retención de la memoria EEPROM.
No. de pines	18 pines. 13 pines de E/S con control individual de dirección. - PORTA de 5 bits (RA0-RA4) - PORTB de 8 bits (RB0-RB7)
Otras características	- Contador/Temporizador TMR0 de 8 bits con divisor programable. - Power-on Reset (POR). - Power-up Timer (PWRT). - Oscillator Start-up Timer (OST). - Watchdog Timer (WDT). - Protección de código. - Modo de bajo consumo SLEEP. - Puede operar bajo 4 modos diferentes de oscilador.

- Programación en serie a través de dos pins.
- Tecnología de baja potencia y alta velocidad CMOS Flash/EEPROM.

2.2 Terminales del PIC16F84A

El PIC16F84A cuenta con 18 pines, (*figura 6*) los cuales se describen a continuación:

- **V_{DD}**: Pin por donde se aplica la tensión positiva de la alimentación.
- **V_{SS}**: Pin conectado a tierra o al negativo de la alimentación.
- **OSC1/CLKIN**: Pin por donde se aplica la entrada del circuito oscilador externo que proporciona la frecuencia de trabajo del microcontrolador.
- **OSC2/CLKOUT**: Pin auxiliar del circuito oscilador
- **MCLR#**: Su activación origina la reiniciación o Reset del PIC.
- **RA0 - RA4**: Son las 5 líneas de E/S digitales correspondientes al puerto A.
 - La línea RA4 multiplexa otra función expresada por TOCKI.
- **RB0 – RB7**: Estos 8 pines corresponden a las 8 líneas de E/S digitales del Puerto B.
 - La línea RB0 multiplexa otra función, que es la de servir como entrada a una petición externa de una interrupción, por eso se la denomina RB0/INT.

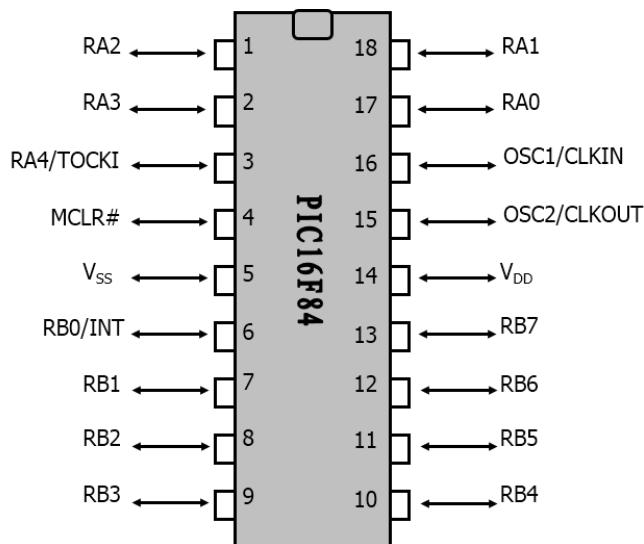


Figura 4. Terminales del microcontrolador.

2.3 Frecuencia de funcionamiento (Ciclos de Reloj).

Todo microcontrolador requiere un circuito externo que le indique la velocidad a la que debe trabajar. Este circuito, que se conoce como oscilador o reloj, es muy simple pero de vital importancia para el buen funcionamiento del sistema. El PIC16F84A puede utilizar cuatro tipos de oscilador diferentes. Estos tipos son:

- RC. Oscilador con resistencia y condensador.
- XT. Cristal (por ejemplo de 1 a 4 MHz).
- HS. Cristal de alta frecuencia (por ejemplo 10 a 20 MHz).
- LP. Cristal para baja frecuencia y bajo consumo de potencia.

Las terminales donde se conecta el circuito temporizador son los pines 15 y 16 (**OSC1/CLKIN y OSC2/CLKOUT**).

En el momento de programar o “quemar” el microcontrolador se debe especificar que tipo de oscilador se usa. Esto se hace a través de unos fusibles llamados “fusibles de configuración” llamados FOSC1 y FOSC0.

El tipo de oscilador que se sugiere para las prácticas es el XT con un cristal de 4 MHz (*figura 7*), porque garantiza precisión y es muy comercial. Internamente esta frecuencia es dividida por cuatro, lo que hace que la frecuencia efectiva de trabajo sea de 1 MHz en este caso, por lo que cada instrucción se ejecuta en un microsegundo. El cristal debe ir acompañado de dos condensadores y se conecta como se muestra en la *figura 3.4*.

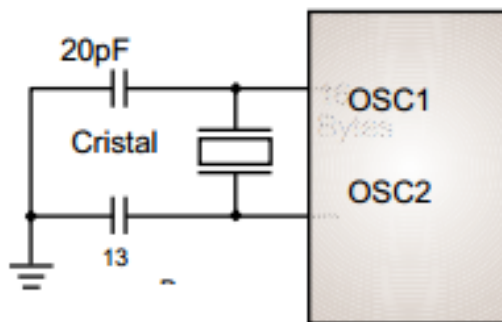


Figura 5. Oscilador con Cristal XT.

En la práctica el cristal oscilador XT se puede conectar como se observa en la *figura 8*.

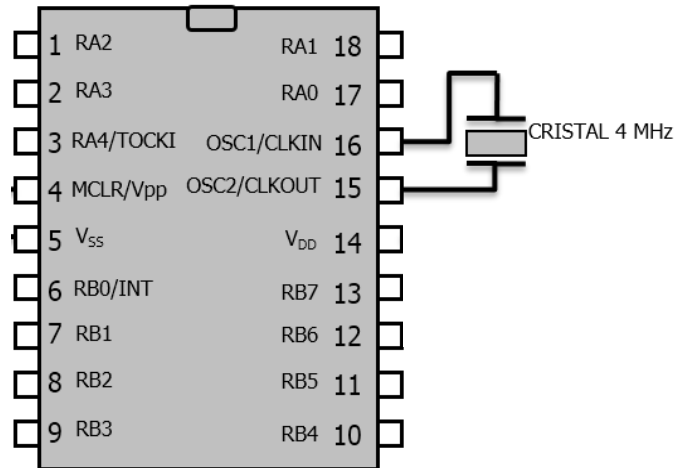


Figura 6. Conexión de un Cristal oscilador XT en la práctica.

Si no se requiere mucha precisión en el oscilador y se quiere economizar dinero, se puede utilizar una resistencia y un condensador al que se le conoce como oscilador RC, como se muestra en la *figura 9*.

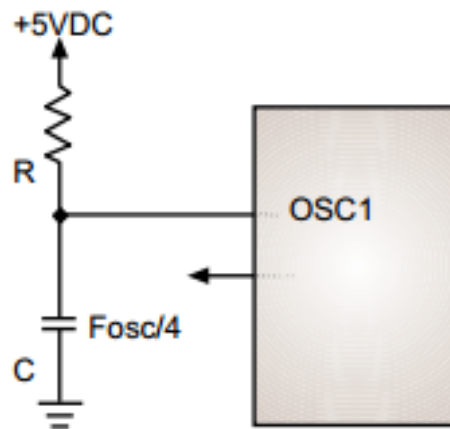


Figura 7. Oscilador RC.

2.4 Reset

En los microcontroladores se requiere un pin de reset para reiniciar el funcionamiento del sistema cuando sea necesario, ya sea por una falla que se presente o porque así se haya diseñado el sistema. El pin de reset en los PIC es llamado *MCLR* (*master clear*). Existen varias formas de resetear o reiniciar el sistema, el circuito más simple para

provocar un Reset manualmente es al presionar un pulsador como se muestra en la *figura 10*.

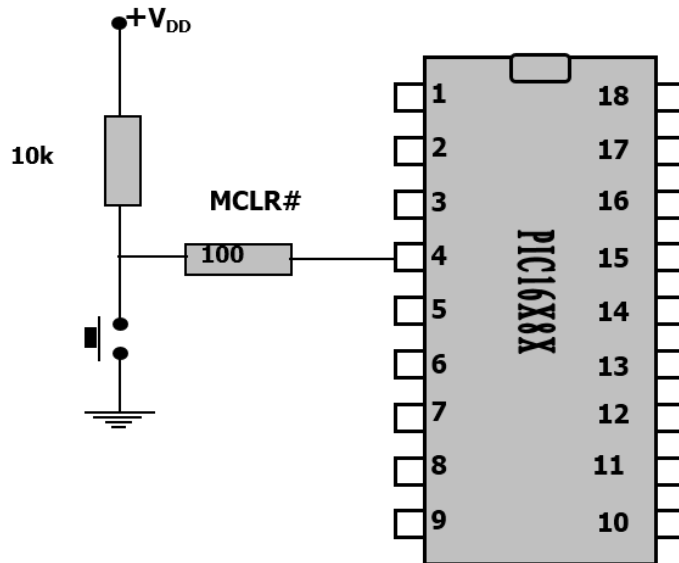


Figura 8. Circuito con push-button para la reniciación del PIC.

También es muy habitual utilizar la configuración de la *figura 11*. Este se reiniciará cada que se apague y se encienda la alimentación del PIC, es conocida como *Power On Reset (Restablecer en el apagado)*.

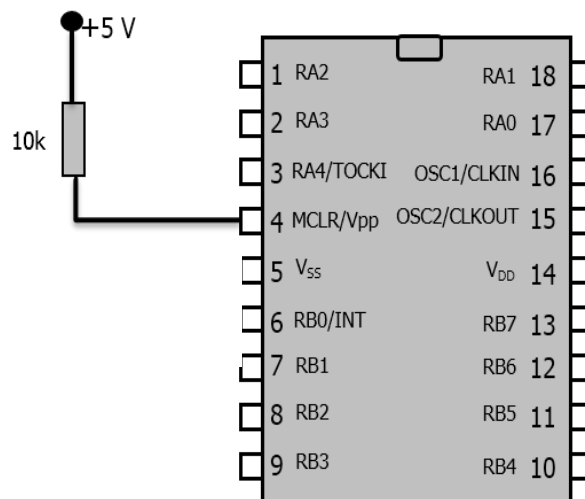
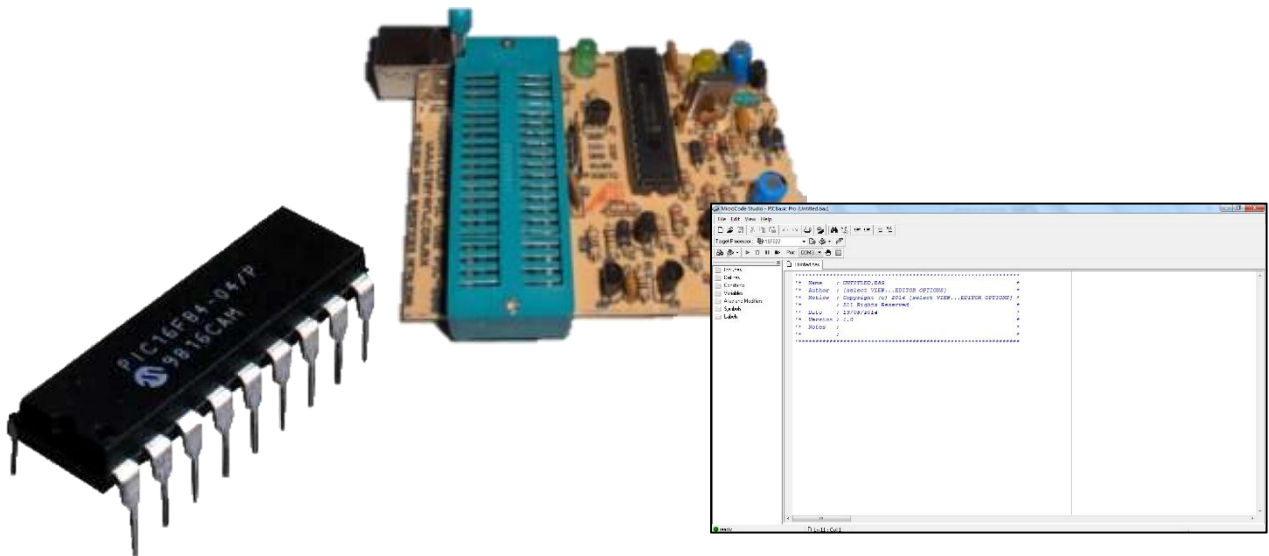


Figura 9. Circuito para provocar la reniciación "Power On Reset"



3.1 Marco teórico

3.1.1 Lenguaje de programación.

El compilador PicBasic Pro será el lenguaje de programación a usar a lo largo de este manual, este es un lenguaje de alto nivel que hace más rápido y fácil programar microcontroladores PIC Micro de Microship Technology. El lenguaje Basic es mucho más fácil de leer y escribir que el lenguaje ensamblador Microchip.

Este lenguaje hace uso de comandos con los que posiblemente el lector ya este muy familiarizado. Por ejemplo: IF..THEN..ELSE..ENDIF, AND, OR, GOTO, WHILE..WEND, FOR..NEXT, etc. Por lo tanto los comandos usados no serán desconocidos para el usuario. A lo largo del manual se irán explicando algunos de los comandos más usados.

3.1.2 Instalación del MicroCode Studio.

El software a usar será el MicroCode Studio 2.1 el cuál permite usar el lenguaje PicBasic. Sin embargo si se cuenta con otro software u otra versión más reciente de igual manera se puede usar. El procedimiento de instalación es muy sencillo, solamente basta con seguir los pasos del asistente de instalación (*figura 12*), por lo que no se explicará en este manual.

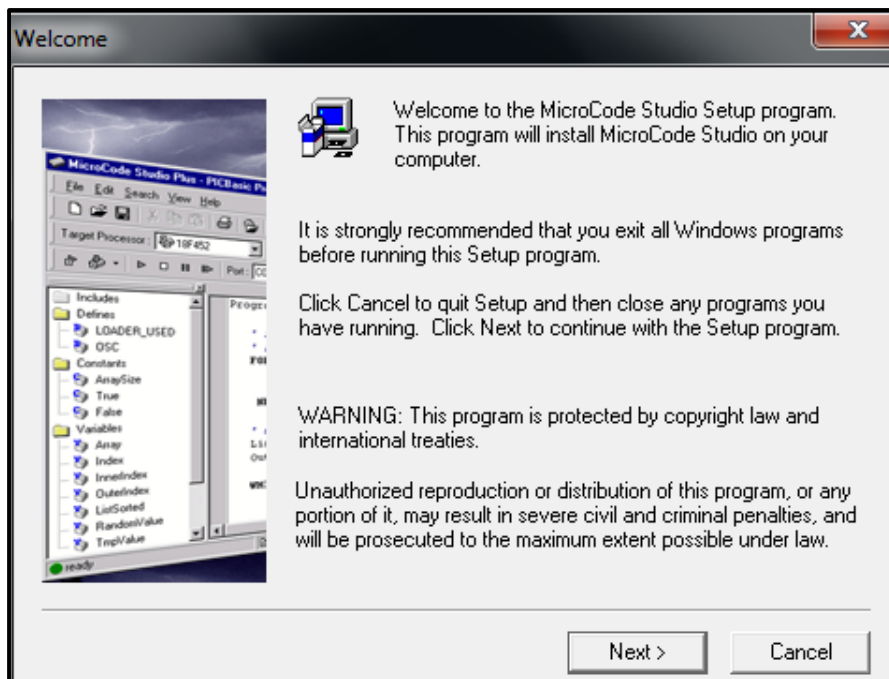
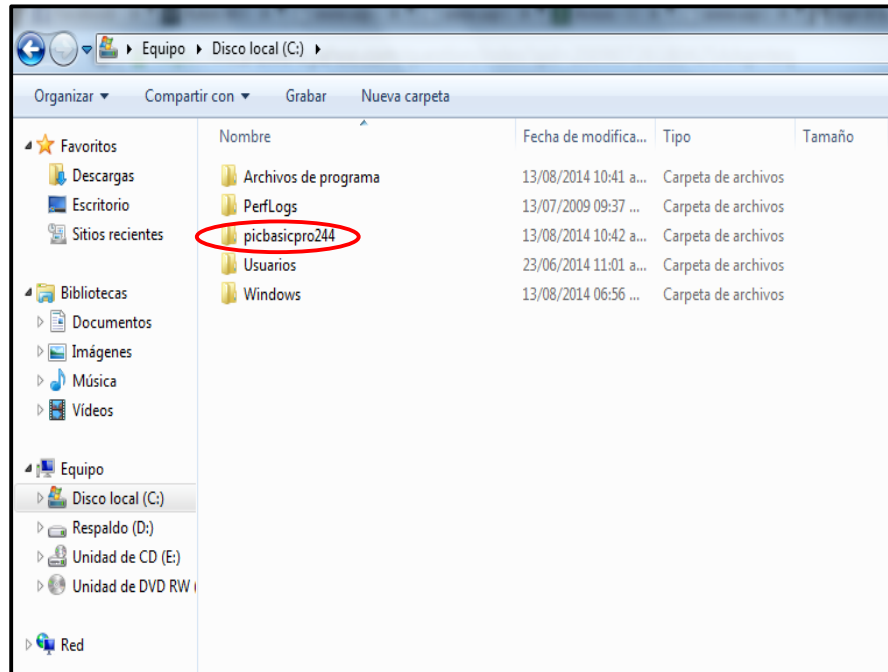


Figura 10. Asistente de instalación del MicroCode Studio

Al finalizar la instalación del microCode Studio se necesita de un conjunto de librerías que contienen los códigos y datos necesarios para que el lenguaje de programación funcione correctamente. Estas librerías deben de copiarse en el directorio raíz (C:\), observe el circulo de color rojo en la *figura 13*. Si esto no se realiza, al momento de compilar un programa puede causar errores. Las librerías y el software se te deben de entregar junto con este manual, en caso contrario pídele a tu profesor que te las facilite.



Organizar Compartir con Grabar Nueva carpeta

Favoritos: Descargas, Escritorio, Sitios recientes

Bibliotecas: Documentos, Imágenes, Música, Videos

Equipo: Disco local (C:), Respaldo (D:), Unidad de CD (E:), Unidad de DVD RW, Red

Figura 11. Instalación de las librerías de MicroCode Studio.

3.1.3 Instalación del Programador PIC-600

El PIC-600 es un módulo programador de microcontroladores PICs de la marca STEREN. La comunicación con la computadora es a través de una interfaz USB y permite programar una amplia variedad de microcontroladores PICs. Es un módulo muy práctico, sencillo de usar y de bajo costo. Los controladores para la instalación de este dispositivo se pueden encontrar en la página del fabricante en el siguiente enlace: <http://www.steren.com.mx/files/product.asp?d=3329&dt=1&p=2644>. Los drivers son compatibles con Windows XP, Vista, seven y 8.

La instalación de los controladores es muy sencilla solamente siga los pasos del asistente de instalación hasta finalizar (*figura 14*).



Figura 12. Asistente de instalación del PIC-600.

Al finalizar el asistente diríjase al administrador de dispositivos y verifique que el dispositivo se haya instalado con éxito, como se muestra en la *figura 15* en el círculo de color rojo, si aparece un signo de admiración de color amarillo donde dice PIC-600, vuelva a instalar el driver, posiblemente hubo un error durante la instalación. Asegúrese de instalar el driver adecuado para el sistema operativo que este usando.

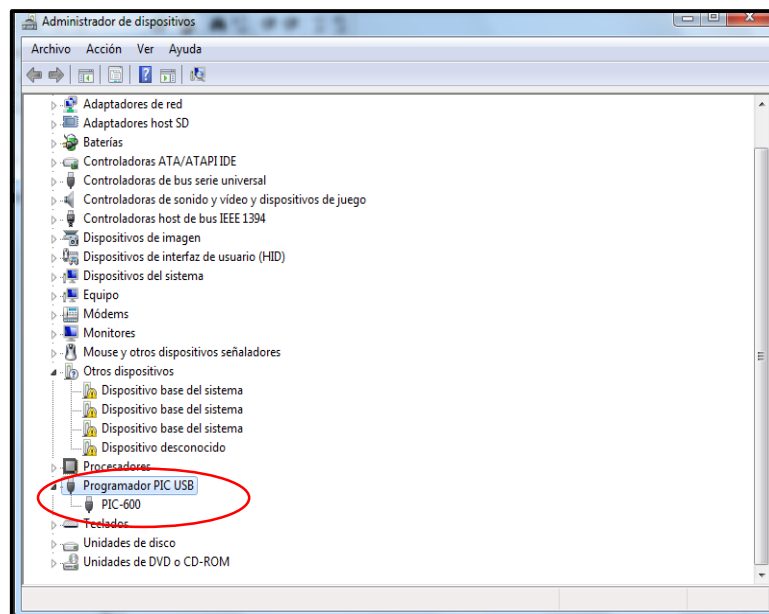


Figura 13. Instalación del driver para el modulo programador PIC-600.

Si el controlador se instaló correctamente, en el escritorio se debe de identificar un icono con la leyenda “PIC-600”. Ejecútelo y observará una ventana igual a la de la *figura 16*. Este será el software necesario para poder cargar los programas que creemos al microcontrolador.

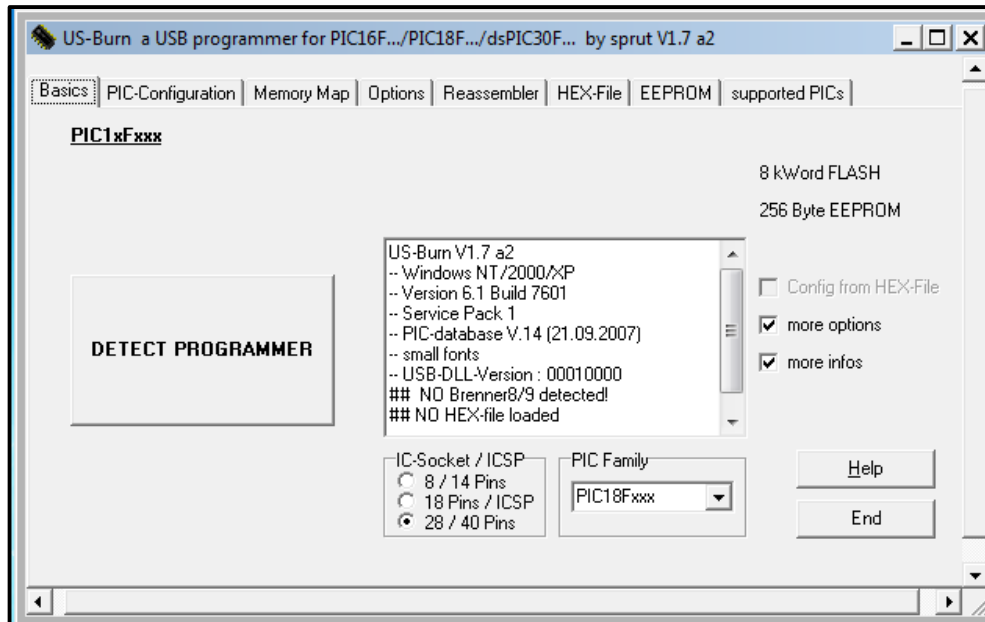


Figura 14. Software PIC-600.

3.2 Comandos sugeridos

Para los ejercicios de esta sección se recomienda aplicar los siguientes comandos:

GOTO

Una herramienta importante en la programación son los ciclos infinitos, con MicroCode Studio para definir un ciclo siga la siguiente estructura:

Estructura: *Nombre del ciclo:*

GOTO *nombre del ciclo*

Ejemplo:

proceso: *'Nombre del ciclo (llamado proceso) + dos puntos.*

Instrucciones dentro del ciclo.

GOTO proceso *'Se cierra el ciclo con la palabra GOTO + el nombre del ciclo*

HIGH, LOW y PAUSE

- Para activar un puerto se puede usar el comando **HIGH**.

Estructura: *HIGH número de puerto*

- Para desactivar un puerto se puede usar el comando **LOW**.

Estructura: *LOW número de puerto*

- Para pausar alguna instrucción específica se puede usar el comando **PAUSE**, el cuál debe de ubicarse inmediatamente después de la instrucción que se desee demorar. El tiempo se expresa en milisegundos.

Estructura: *Instrucción anterior*

PAUSE *tiempo de demora expresada en milisegundos.*

Ejemplo:

HIGH PORTA.0 *'Activa el puerto A.0 del microcontrolador.*

PAUSE 1000 *'Mantiene el puerto A.0 activo durante 1000 ms, es decir un segundo.*

LOW PORTA.0 *'Desactiva el puerto A.0*

NOTA: Para referirse a un puerto físico del microcontrolador al momento de la programación, simplemente hay que escribir el número correspondiente de ese puerto.

En el ejemplo mostrado "HIGH PORTA.0", se activa el puerto RA0, como se muestra en la figura 17, corresponde al pin 17 del microcontrolador.

Si por ejemplo se quisiera activar el puerto número 12 que corresponde al puerto RB6, se escribirá de la misma manera "HIGH PORTB.6".

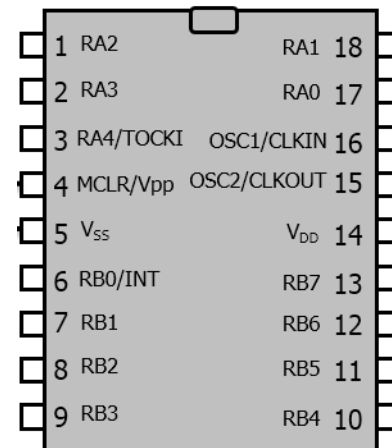


Figura 15. Referencia al puerto del PIC en la programación.

3.3 Práctica de Laboratorio No. 1. - Parpadeo de un LED.

Integrantes:

1. _____
2. _____
3. _____
4. _____

Objetivo general:

Adquirir experiencia en la programación de microcontroladores, elaborando un sencillo programa el cual tendrá la función de hacer parpadear un LED.

Objetivos particulares:

- Aprender a usar el MicroCode Studio.
- Aprender a usar el PIC-600.
- Crear un primer programa en lenguaje PICBASIC.
- Conocer la forma de conexión del PIC16F84A.

Equipo:

- PC de escritorio o laptop.
- Programador PIC-600.
- Cable USB.
- Protoboard.

Material a utilizar:

- PIC16F84A
- 8 resistencias de 330 Ω .
- 1 Resistencia de 100 Ω .
- 1 Resistencia de 10 K Ω .
- 8 LEDs del color deseado.
- 1 Cristal oscilador de 4 MHz
- Cables para conexión.

Desarrollo de la práctica.

1. Arme en el protoboard el circuito que se muestra en la *figura 18*.

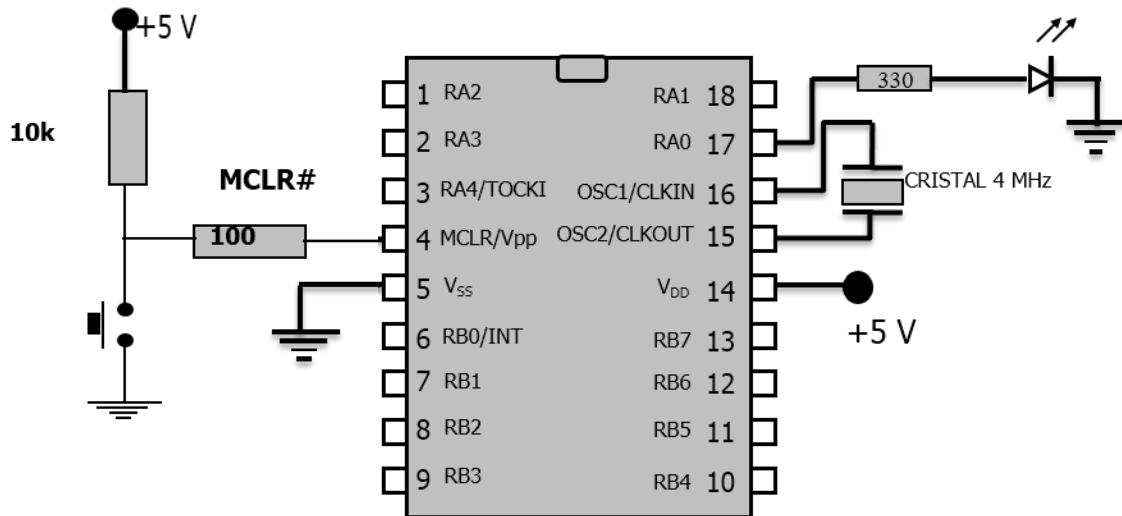


Figura 16. Circuito de la práctica 1.

2. Ejecute el MicroCode Studio, debe de poder observar una ventana igual a la mostrada en la *figura 19*. Para este paso ya debe de tener instalado el MicroCode Studio en su PC junto con las librerías necesarias para su correcto funcionamiento. Consulte los pasos mostrados en el Marco Teórico de esta sección.

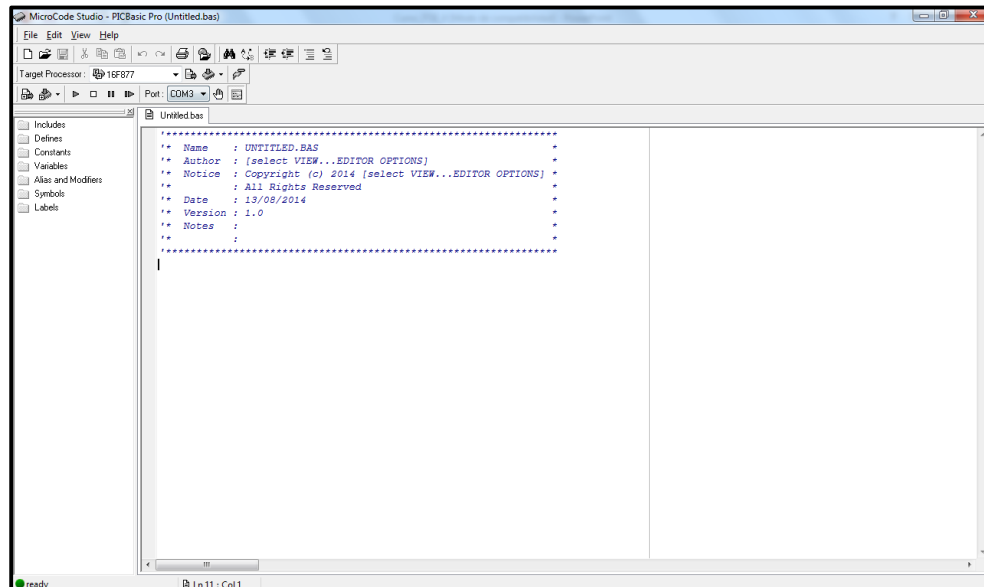


Figura 17. MicroCode Studio.

3. Configure el software de acuerdo al Microcontrolador que desea programar. Para este manual se estará trabajando con el PIC16F84A. Diríjase la opción con la leyenda

“Target processor”, despliegue el menú y seleccione la opción “16F84A”, como se muestra en la figura.

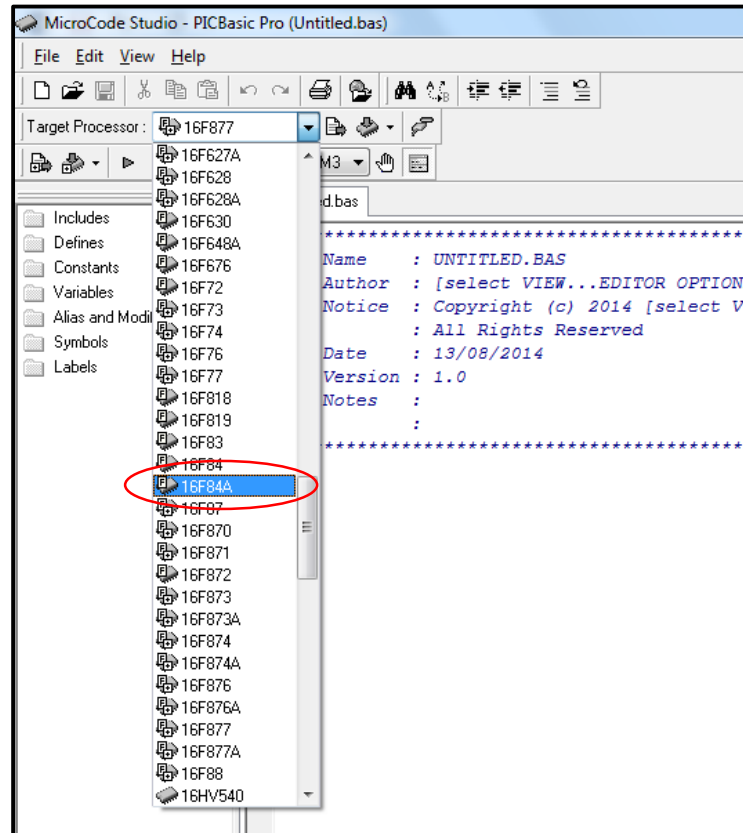


Figura 18. Configuración del MicroCode Studio.

4. Lo siguiente es escribir el código, haga clic en el área de trabajo, una vez que aparezca el cursor, proceda a escribir el siguiente código.

```

loop:
    high PORTA.0      ' enciende el LED
    pause 500         ' demora de .5 segundos
    low PORTA.0       ' apaga el LED
    pause 500         ' demora de .5 segundos
goto loop            ' vuelve a loop y hace parpadear el
end                  ' LED para siempre
  
```


5. Una vez escrito el código, guarde el archivo con el nombre que guste. MicroCode Studio lo guardará con la extensión “**.bas**”. En este caso lo llamaremos **BLINK.bas**, (figura 21).

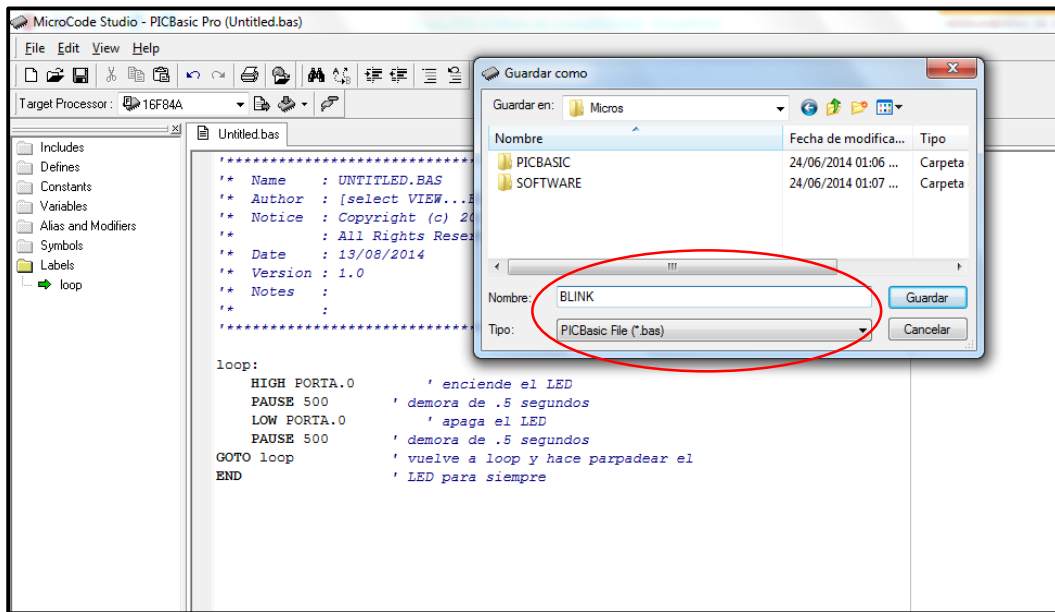


Figura 19. Archivos ".bas".

6. Una vez guardado el archivo elija la opción “**Compile Only**” en la barra de herramientas o presione el botón **F9**, (figura 22), esto es para que el MicroCode Studio convierta las instrucciones del código escrito de un lenguaje de alto nivel como lo es el PicBasic a un lenguaje máquina, el cuál puede entender el microcontrolador.

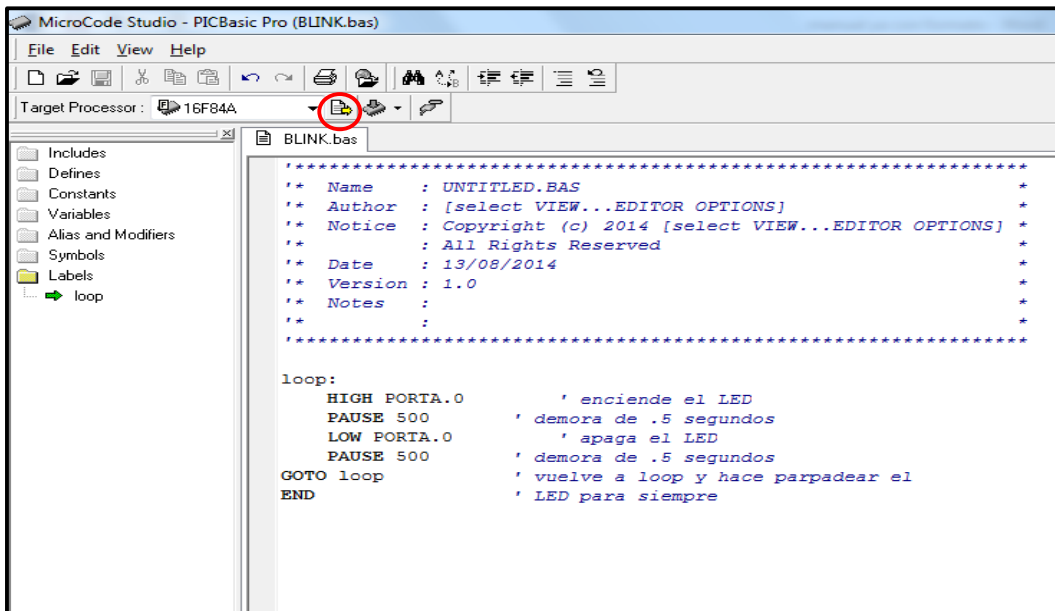


Figura 20. Compilación de un programa.

Ahora ya tenemos nuestro programa listo, solamente falta cargarlo al PIC16F84A.

7. Ejecute el programa PIC-600, el acceso directo debe de instalarse por defecto en el escritorio. Al abrir el programa debes de observar una ventana igual a la mostrada en la *figura 23*. Para este paso ya debiste haber instalado los driver del programador de STEREN. Si tienes duda consulta el marco teórico de esta sección.

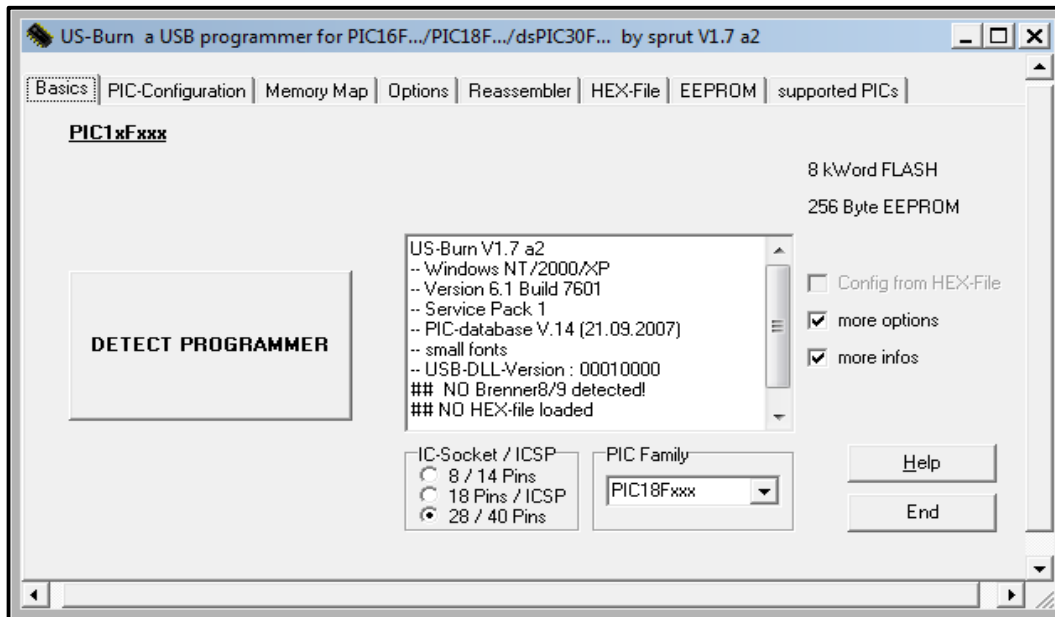


Figura 21. PIC-600

8. Configure el software PIC-600 como se muestra en la *figura 24*.

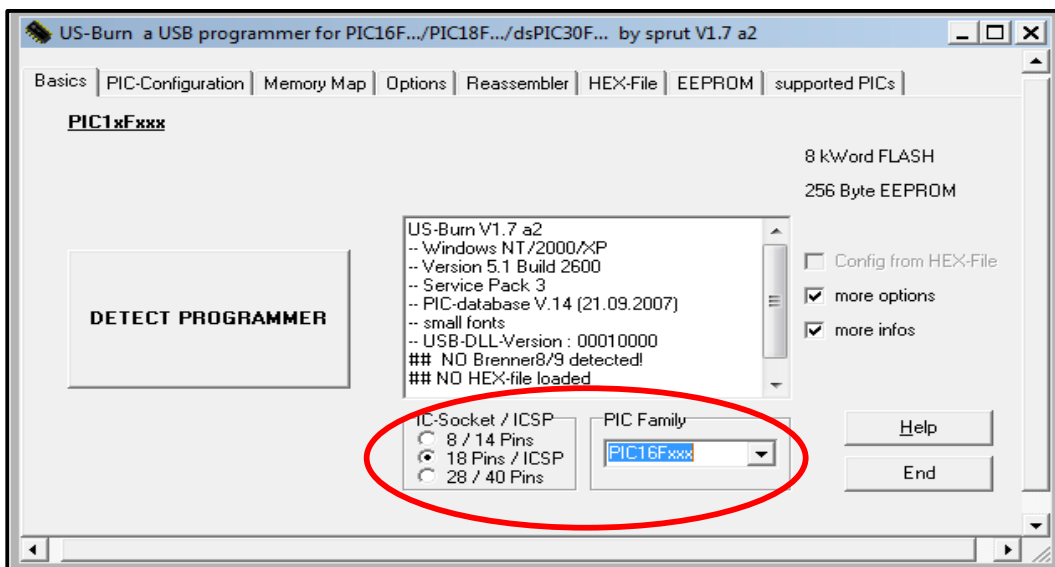


Figura 22. Configuración del PIC-600.

9. Retire el PIC16F84A del circuito que ya debes de tener armado en el protoboard y colócalo en el programador con la muesca hacia arriba como se muestra en la *figura 25*. Luego baje la palanca para asegurar el Microcontrolador.



Figura 23. Módulo programador de STEREN PIC-600.

10. Procede a conectar el programador PIC-600 a uno de los puertos USB de la PC, *figura 26*.

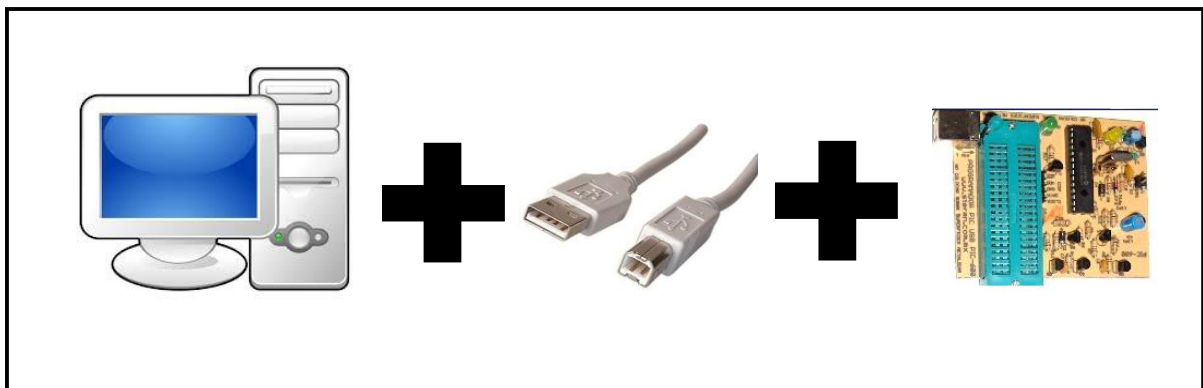


Figura 24. Conexión del PIC-600 a la PC.

11. Haga clic en el botón **DETECT PROGRAMMER** ubicado en la parte izquierda de la pantalla. Al realizar ese paso debe de mostrarse una ventana igual a la de la *figura 27*.

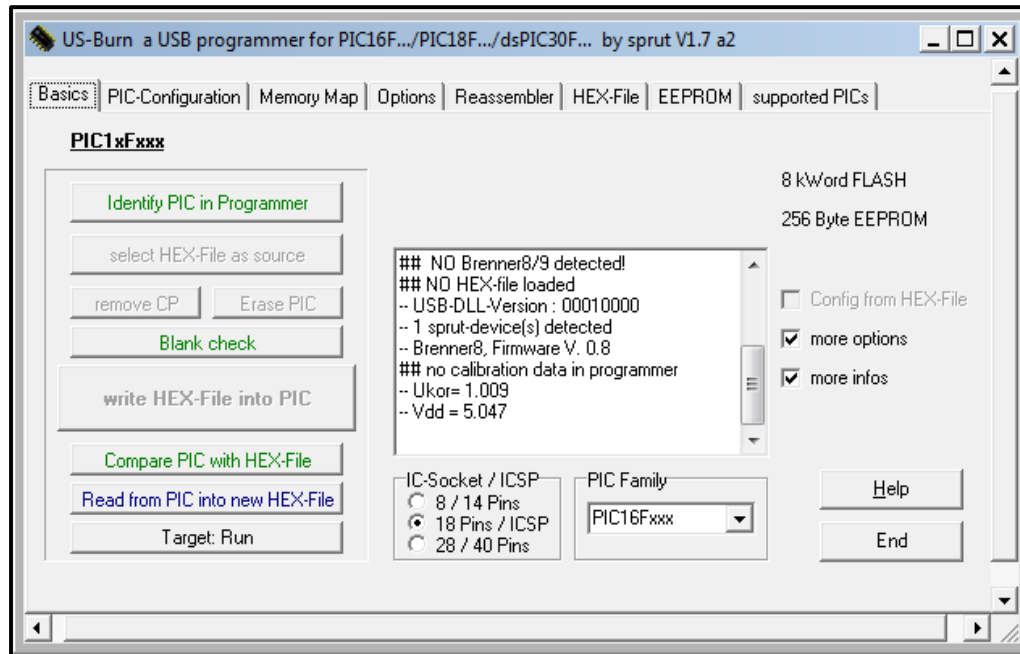


Figura 25. Software PIC-600 listo para iniciar la programación del PIC.

12. Haga clic en el primer botón “**Identify PIC in Programmer**” para que el software identifique el tipo de PIC conectado al Programador. En el cuadro blanco de la ventana se debe mostrar que se trata de un PIC16F84A, (*Figura 28*).

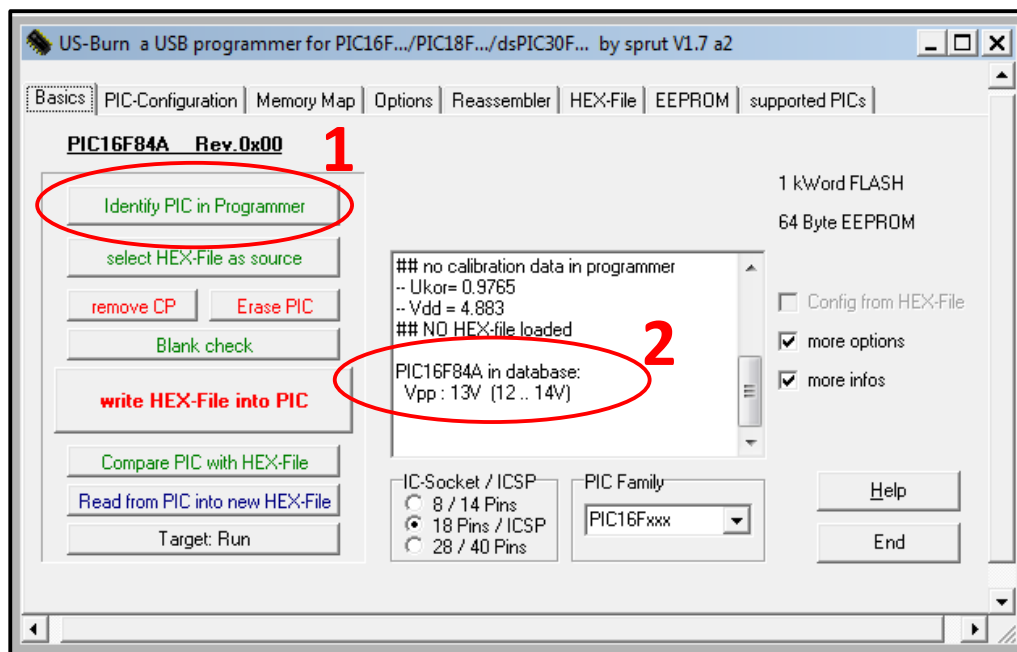


Figura 26. El PIC-600 identifica el PIC16F84A.

13. El siguiente paso es elegir el archivo que creó el MicroCode Studio al momento de compilar el código. Elija la opción **“Select HEX-File as source”** para elegir el archivo con extensión **“.HEX”**, este es un archivo estándar de Intel que puede escribirse dentro del microcontrolador, (figura 29). Este archivo está ubicado en el directorio donde guardó su archivo con extensión **“.bas”** y debe de tener el mismo nombre, en este caso debe de llamarse **BLINK.HEX**. Proceda a abrir el archivo.

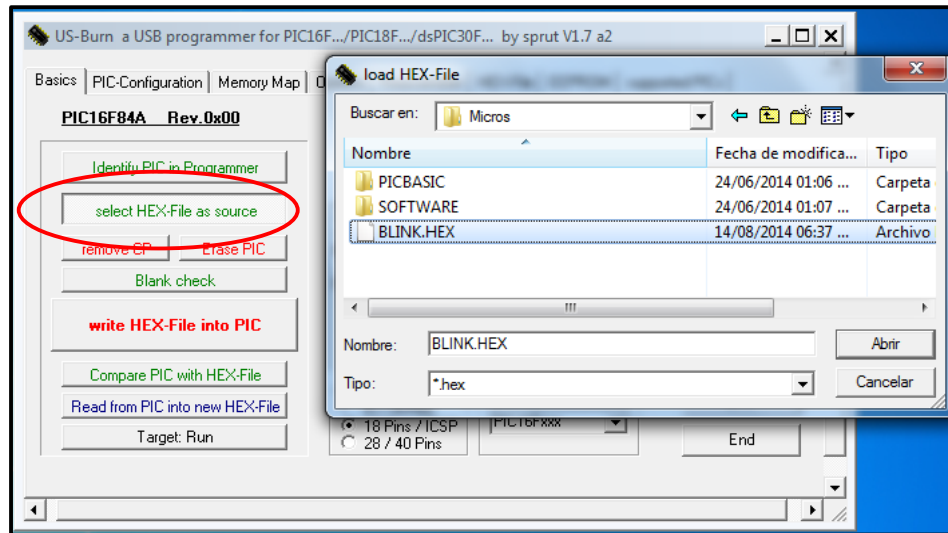


Figura 27. Selección del archivo ".HEX".

14. Una vez hecho lo anterior elija la opción **“Write HEX-File into PIC”** para cargar el programa al PIC16F84A. Al realizar este paso debe de observar una barra de proceso de color verde que debe completar el 100% y al finalizar debe mostrar la leyenda **“OK”** indicando que el programa se cargó correctamente dentro del microcontrolador, (figura 30).

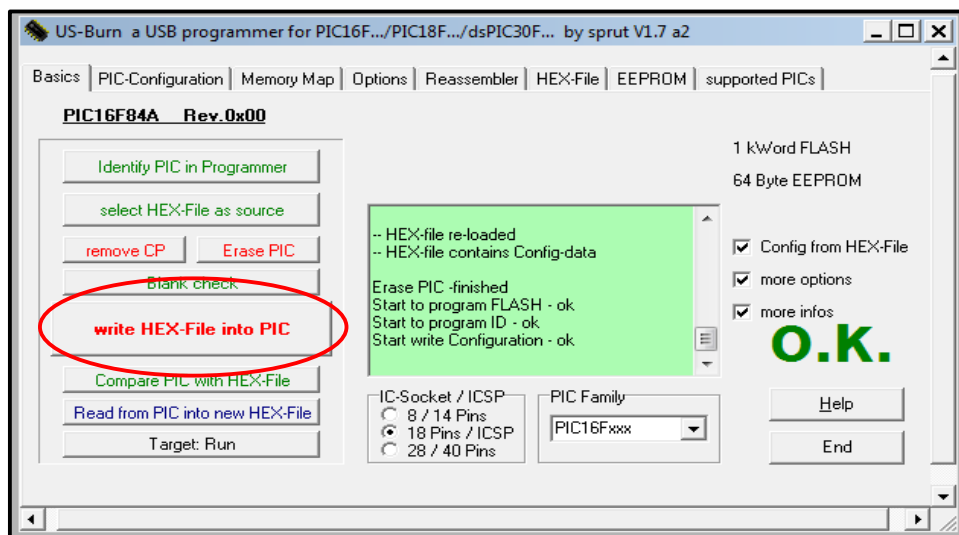


Figura 28. Cargar el programa ".HEX" al PIC16F84A.

15. Una vez que el programa es cargado al PIC retírelo del programador y colóquelo de nuevo en su protoboard, como se mostró en el circuito propuesto en el paso 1. Aliméntelo con una fuente de 5V de CD y observe como el LED parpadea cada .5 segundos.

NOTA: Ahora ya sabes cómo realizar un programa y cómo cargarlo al microcontrolador. Desde ahora realizarás el mismo procedimiento con cada una de las prácticas y problemas retos propuestos en este manual.

Anote sus conclusiones individualmente:

1. _____

2. _____

3. _____

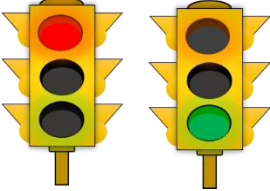
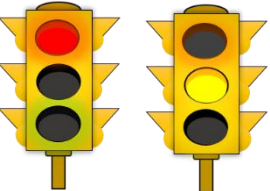
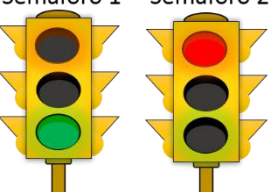
4. _____

3.4 Problemas reto.

1. Semáforo

Realiza un programa que simule un semáforo. Solamente puedes usar los comandos **HIGH**, **LOW**, **PAUSE** y **GOTO**. Su estructura la puedes consultar en la sección de comandos sugeridos. El programa debe de cumplir con la secuencia que se muestra en la siguiente *tabla 2*.

Tabla 2. Secuencia del semáforo.

Semáforo 1 Semáforo 2 	Primer paso: • El LED rojo del semáforo 1 debe mantenerse encendido en este paso. • El LED verde del semáforo 2 debe de mantenerse encendido durante 5 segundos, luego parpadeará 5 veces con intermitencias de 500 ms para después apagarse.
Semáforo 1 Semáforo 2 	Segundo paso: • El LED rojo del semáforo 1 debe mantenerse encendido en este paso. • El LED amarillo del semáforo 2 debe mantenerse encendido durante 3 segundos y luego apagarse.
Semáforo 1 Semáforo 2 	Tercer paso: • Se enciende el LED rojo del semáforo 2 y se mantiene encendido. • Se apaga el LED rojo del semáforo 1 y se enciende el LED verde durante 5 segundos, luego parpadeará 5 veces con intermitencias de 500 ms para después apagarse, y así hasta repetir la secuencia que realizó el semáforo 2. • Repita el ciclo infinidad de veces o un mínimo de 10 transiciones.

Para este problema se propone el circuito de la *figura 31*.

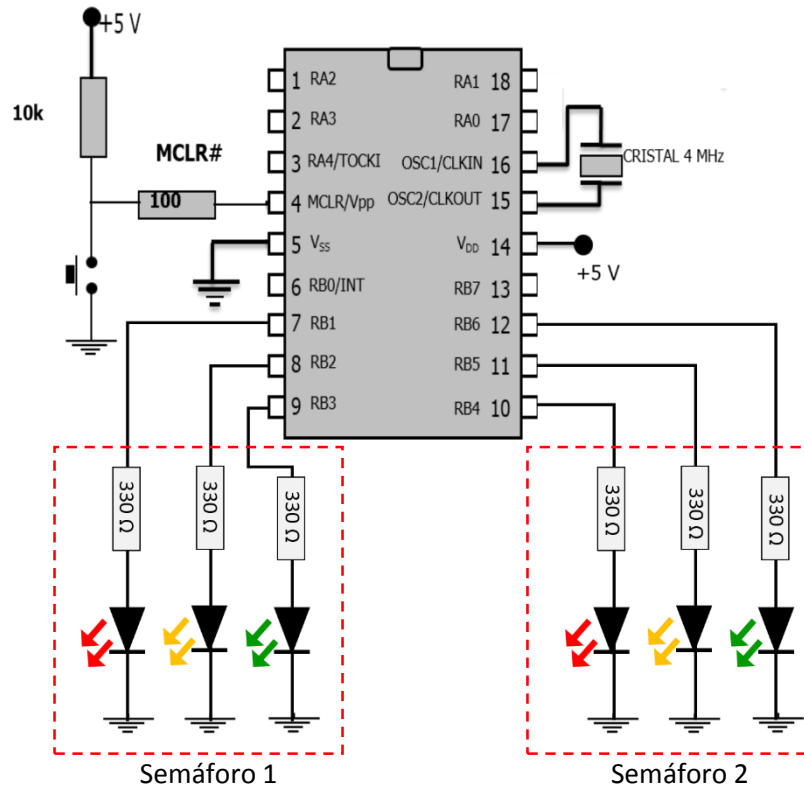
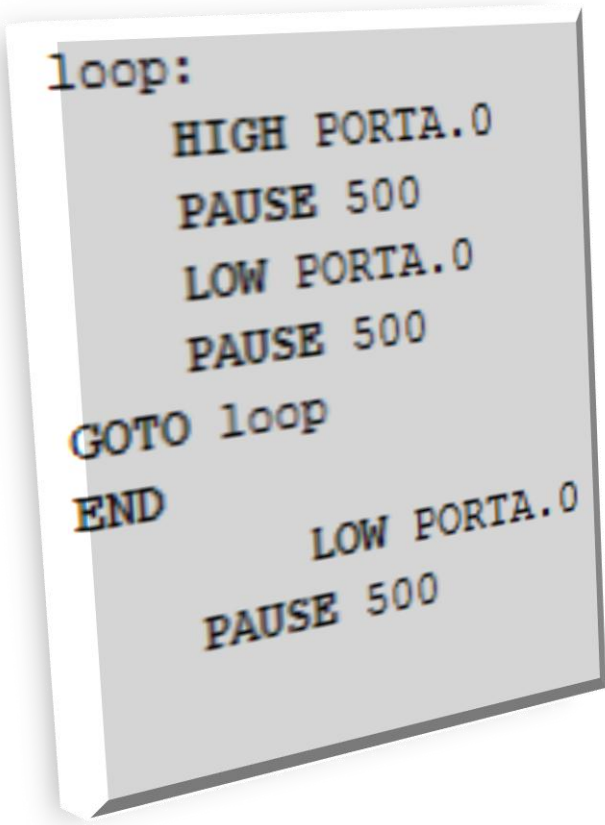


Figura 29. Circuito para el semáforo.

2. Secuencia libre:

Usando los puertos **B del 0-7 y los puertos A del 0-3** del PIC16F84A agregue más LED's; tendrá un total de 12. Conecte los nuevos LED's como se mostró en el ejemplo anterior; puedes variar los colores si lo desea. Después realiza un programa que haga una secuencia, como las de los foquitos de navidad por ejemplo. Recuerda solamente puedes usar los comandos mencionados en el reto 1. Una vez que finalices explícale a tu profesor como funciona tu programa.

4. Comandos básicos de PICBASIC



```
loop:  
  HIGH PORTA.0  
  PAUSE 500  
  LOW PORTA.0  
  PAUSE 500  
GOTO loop  
END  
  LOW PORTA.0  
  PAUSE 500
```

4.1 Marco teórico

4.1.1 Comando básicos

Ahora ya has tenido tu primera experiencia programando microcontroladores PIC usando los comandos: **HIGH**, **LOW**, **PAUSE** y **ciclos con GOTO**, sin embargo, antes de continuar con los temas siguientes tendrás que adquirir más conocimientos sobre los comandos básicos de PBP (PICBASIC PRO).

Comentarios.

Es muy común que al momento de escribir nuestros programas necesitemos agregar comentarios para recordar las tareas que realiza cada código del algoritmo. También son muy útiles cuando otras personas requieren analizar nuestro código, los comentarios ayudan a entender más fácil nuestros programas.

Para escribir comentarios en PBP se usa el apostrofe el cuál esta ubicado normalmente en la tecla donde se encuentra el signo de interrogación (¿) y la diagonal (\) en un teclado de idioma español. Los comentarios normalmente cambian a un color más vivo cuando los escribes, en la *figura 32* se pueden observar de color azul.

```
loop:
    HIGH PORTA.0      ' enciende el LED
    PAUSE 500         ' demora de .5 segundos
    LOW PORTA.0       ' apaga el LED
    PAUSE 500         ' demora de .5 segundos
    GOTO loop         ' vuelve a loop y hace parpadear el
END                  ' LED para siempre
                    ' apaga el LED
    LOW PORTA.0
    PAUSE 500
```

Figura 30. Comentarios en la programación PICBASIC.

Variables

Las Variables son donde se guardan los datos en forma temporal en un programa PBP. Son creadas usando la palabra clave **VAR**. Pueden ser **bits**, **bytes** ó **word**. El espacio para cada variable es automáticamente destinado en la memoria del micro controlador por PicBasic Pro. La estructura para crear una variable es el siguiente:

Estructura: Etiqueta + **VAR** + tamaño

Ejemplo:

Dog **VAR** byte *'Variable del tipo byte.'*

Cat **VAR** bit *'Variable del tipo bit.'*

W0 **VAR** Word *'Variable del tipo Word'*

Al momento de declarar variable trate de que estas sean acorde a la tarea que realizan y no escribir nombres que no recuerdes a la hora de programar. También tienes que tener claro la cantidad de datos que quieras almacenar. En la *tabla 3* se puede ver el tamaño de cada variable.

Tabla 3. Tipos de variables en PBP.

Tamaño	Número máximo de elementos
Bit	Un bit de longitud almacena solo los valores 0 y 1 únicamente.
Byte	Un byte de longitud almacena números enteros entre 0 y 255.
Word	Almacena números entre 0 y 65,535.

Para recordar fácilmente para qué puede ser usado un pin, puedes asignarle un nombre usando el comando **VAR**. De esta manera, el nombre puede ser utilizado luego en cualquier operación:

Estructura: nombre + **VAR** + nombre del puerto o de los puertos.

Ejemplo:

Led **VAR** PORTA.0 *'Asigna el nombre Led al PORTA.O.'*

W1 **VAR** PORTB *'Asigna el nombre W1 a todos los puertos B del PIC.'*

Arreglos (Arrays)

Los arreglos de variables pueden ser creados en una manera similar a las variables.

Estructura: Etiqueta + **VAR** + tamaño + [Número de elementos]

Ejemplo:

Sharks **VAR** byte [10] *'Almacena 10 elementos del tipo byte.'*

Fish **VAR** bit[8] *'Almacena 10 elementos del tipo bit.'*

Dada la forma en que los arreglos están localizados en memoria hay límites de tamaño para cada tipo, observe la *tabla 4*.

Tabla 4. Límites para tipos de arreglos en PBP.

Tamaño	Número máximo de elementos
Bit	128
Byte	64
Word	32

Configurar pin como entrada o salida.

Cuando arranca el PIC, todos los pins son colocados en entrada. Para usar un pin como salida, se debe usar un comando que automáticamente coloque al pin como salida. Esto es muy importante porque al momento de realizar un programa, si no declaras desde el principio que puertos recibirán datos y cuales enviarán, aún el código sea correcto no tendrás los resultados esperados.

Para colocar un pin o port como salida (ó entrada) debe dar valores al registro **TRIS**. Colocando el bit de TRIS como 0, hace su pin una salida, y colocándolo en 1 lo hace una entrada, observe la *tabla 5*.

Tabla 5. Configuración de puertos como salidas y entradas. .

Configuración de un Puerto: salida o entrada	
0	Salida
1	Entrada

Por ejemplo el PIC16F84A cuenta con 8 Puertos **B (RB0-RB7)**. Si deseo declarar todos como entradas se escribe al inicio del programa el código de ejemplo. Observe que son 8 números 1 debido a que son 8 puertos del tipo B:

Ejemplo:

TrisB = %11111111 *‘Establece todos los puertos B como entradas.*

Manera simplificada:

Ejemplo:

TrisB = 1 *‘Establece todos los puertos B como entradas.*

De manera similar si requerimos declarar todos los puertos A como salidas. El PIC16F84A cuenta con 5 puertos A (RA0-RA4), entonces escribimos el siguiente código:

Ejemplo:

```
TrisA = %000000    'Establece todos los puertos B como salidas.'
```

Manera simplificada:

Ejemplo:

```
TrisA = 0          'Establece todos los puertos B como salidas.'
```

Ahora suponga que desea declarar los puertos: **PORTB.0, PORTB.2 y PORTB.7** como entradas y todos los demás como salidas.

Ejemplo:

```
TrisB = %10000101    'Establece solamente los puertos RB.0, RB.2 y RB.7 como  
entradas.'
```

En este último caso, observe que el bit menos significativo (el ubicado más a la derecha) corresponde al PORTB.0 y el más significativo (el ubicado más a la izquierda) hace referencia al PORTB.7 del PIC.

Declaraciones múltiples

Para permitir programas más compactos y agrupamientos lógicos de comandos relacionados, PBP soporta el uso de (:) para separar comandos ubicados en la misma línea. Los siguientes dos ejemplos son equivalentes.

Ejemplo1:

```
W2=W0  
W0=W1    'Declaraciones de igualdades'  
W1=W2
```

Ejemplo2:

```
W2 = w0 : W0 = W1 : W1 = W2    'Declaraciones usando ":"'
```

En los dos casos anteriores el tamaño del código generado es el mismo.

Declaraciones de constantes

Las llamadas constantes pueden ser creadas de manera similar a las variables. Puede ser más conveniente usar un nombre de constante en lugar de un número constante. Si el número necesita ser cambiado, únicamente puede ser cambiando en un lugar del programa donde se define la constante. No pueden guardarse datos variables dentro de una constante.

Ejemplo:

Mice **CON** 3 *'Asigna a la variable "Mice" el valor 3.*
Traps **CON** mice *1000 *'La variable "Traps" toma el valor de multiplica "mice" opor 1000*
'

Condicional WHILE...WEND

Similar al comando GOTO que crea un ciclo infinito, pero con la diferencia que el condicional **WHILE** solamente se ejecutará mientras la condición establecida sea verdadera. El condicional se cierra con la palabra **WEND**.

Ejemplo:

WHILE porta.0 = 1 *' condicional WHILE*
 HIGH portb.4
 PAUSE 1000
 LOW portb.4
WEND *' Se Cierra el condicional con la palabra WEND*

Condicional IF...THEN...ENDIF

En el siguiente ejemplo se observa la estructura del IF..THEN.

Ejemplo:

```
IF porta.0 = 1 THEN           'condicional IF'  
    high portb.4  
    pause 1000  
    low portb.4  
ENDIF                        'Se Cierra el condicional con la palabra ENDIF'
```

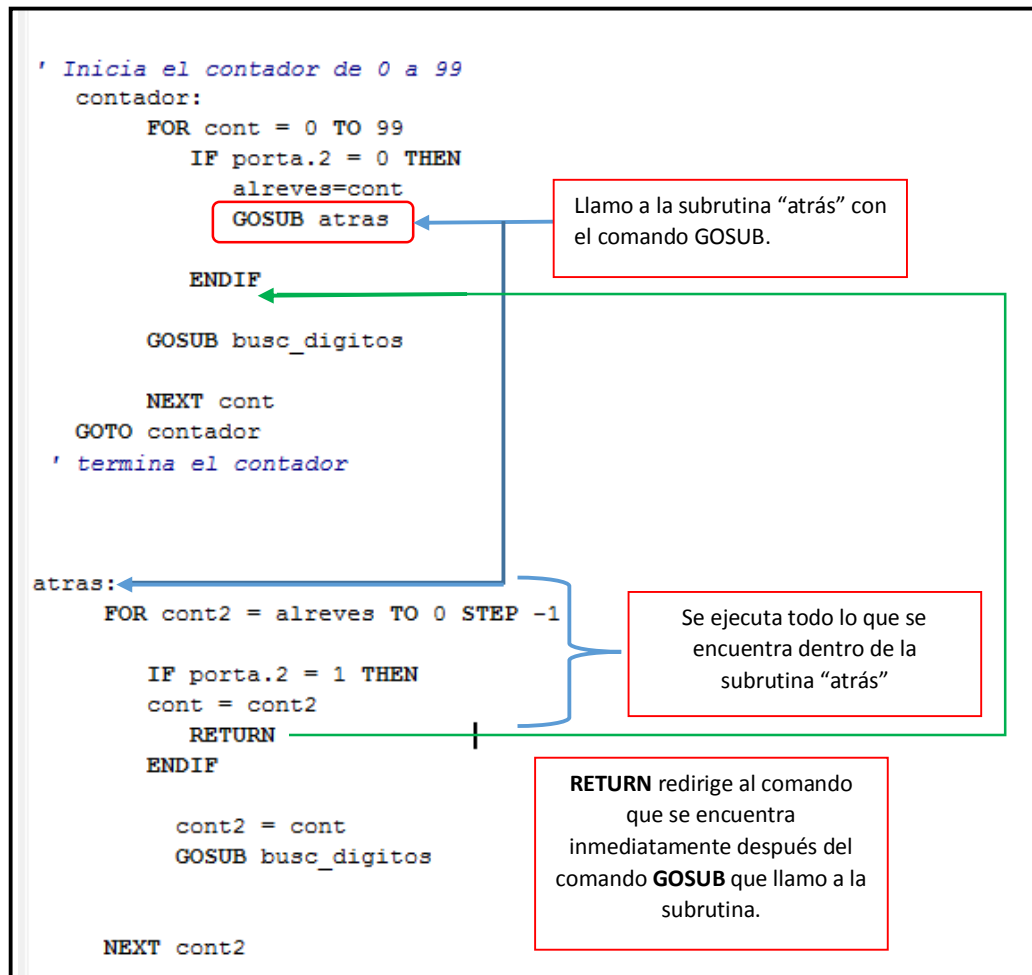
Condicional IF...THEN...ELSE...ENDIF

Ejemplo:

```
IF porta.0 = 1 THEN           'condicional IF'  
    HIGH portb.4  
    PAUSE 1000  
    LOW portb.4  
ELSE                        'Sentencia que se ocupa si la primera es faltas.'  
    HIGH portb.5  
    PAUSE 1000  
    LOW portb.5  
ENDIF                        'Se Cierra el condicional con la palabra ENDIF'
```

Llamar subrutinas “GOSUB”

Salta a la subrutina indicada en la etiqueta, guardando su dirección de regreso en la pila (stack). A diferencia del GOTO, cuando se llega a un **RETURN**, la ejecución sigue con la declaración siguiente al último **GOSUB** ejecutado. En el código mostrado se observa un ejemplo:



Loop FOR...NEXT

El loop **FOR .. NEXT** permite a los programas ejecutar un instrucción un cierto número de veces, usando una variable como contador. Observe el ejemplo:

Ejemplo:

```
FOR decen = 0 to 9      'Inicio un contador de 0 a 9 con la variable "decen".  
  LOW PORTA.0  
  PAUSE 500  
  HIGH PORTA.1  
  PAUSE 50  
NEXT decen              'Continuo con el siguiente contador.
```

'Las instrucciones se ejecutarán 10 veces (0-9)

El programador decide desde donde empieza el conteo. En este caso el conteo inicio desde cero, pero tú puedes modificar el valor de inicio. Si deseas que el conteo sea de 2 en 2, por ejemplo, usa el comando **STEP**.

Ejemplo:

```
FOR cont2 = n to 0 step 2  'El contador iniciará con el valor de la variable "n" y  
                           'realizará el conteo de dos en dos de forma ascendente.  
                           'Instrucciones dentro de contador.  
NEXT cont2                 'Continuo con el siguiente contador.
```

Ahora si deseas que el contador cuente en descenso se coloca la palabra **STEP** seguida del signo negativo "-" y el valor numérico.

Ejemplo:

```
FOR cont2 = n to 0 step -1 'El contador iniciará con el valor de la variable "n" y  
                           'realizará el conteo por unidad de forma descendente.  
                           'Instrucciones dentro de contador descendente. .  
NEXT cont2                 'Continuo con el siguiente contador.
```

El valor de -1 indica que el conteo se hará en descenso pero restando de a una unidad.

DEFINE

Algunos elementos, como el oscilador y las ubicaciones de los pin LCD, están predefinidas en PBP. El comando **DEFINE** le permite a un programa PBP cambiar estas definiciones si así lo desea. Define puede ser usado para cambiar el valor predefinido del oscilador, los pins de DEBUG y el baud rate y las ubicaciones de los pin LCD además de otras cosas. Estas definiciones deben estar en mayúsculas, observe la *tabla 6*.

Tabla 6. Comando DEFINE.

COMANDO DEFINE	
DEFINE BUTTON_PAUSE 50	´ demora en el anti-rebote del botón en ms
DEFINE CHAR_PACING 1000	´ paso de la salida serial en us
DEFINE DEBUG_REG_PORTL	´ depuracion del pin port
DEFINE DEBUG_BIT 0	´ depuracion del pin bit
DEFINE DEBUG_BAUD 2400	´ depuracion del baud rate
DEFINE DEBUG_MODE 1	´ modo depuracion: 0=CIERTO,1=INVERTIDO
DEFINE DEBUG_PACING 1000	´ paso de depuracion en us
DEFINE HSER_RCSTA 90 h	´ setear registro receive
DEFINE HSER_BAUD 2400	´ setear baud rate
DEFINE HSER_EVEN 1	´ usar solo si se desea paridad par
DEFINE HSER_ODD 1	´ usar solo si se desea paridad impar
DEFINE I2C_INTERNAL 1	´ usar para EEPROM interno en 16CEXX y 12CEXX
DEFINE I2C_SLOW 1	´ usar para OSC > 8Mhzcon dispositivos de velocidad standard
DEFINE LCD_DREG PORTB	´ port de data LCD
DEFINE LCD_DBIT 0	´ datos LCD comenzando en bit 0 o 4
DEFINE LCD_RSREG PORTB	´ port de selección de registro LCD
DEFINE LCD_RSBIT 4	´ bit de selección de registro LCD
DEFINE LCD_EREG PORTB	´ port de habilitacion LCD
DEFINE LCD_EBIT 5	´ bit de habilitacion LCD
DEFINE LCD_BITS 4	´ tamaño 4 u 8 de bus de LCD
DEFINE LCD_LINES 2	´ numero de lineas en LCD
DEFINE OSC 4	´ 3 4 8 10 12 16 20
DEFINE OSCCAL_1K 1	´ setea OSCCAL para PIC12C671
DEFINE OSCCAL_2K 1	´ setea OSCCAL para PIC12C672

Operadores matemáticos.

PBP efectúa todas las operaciones matemáticas en orden jerárquico. Esto significa que existe precedencia para los operadores Multiplicación y división, estos son efectuados antes que la suma y resta, por ejemplo: para asegurarse que las operaciones son efectuadas en el orden que se desea, use paréntesis para agrupar las operaciones.

$$A = (B + C) * (D - E)$$

Todas las operaciones matemáticas se realizan sin signo y con una precisión de 16 bit. Los operadores soportados son las mostradas en la *tabla 7*:

Tabla 7. Operadores matemáticos.

Operadores matemáticos	
+	Suma
-	Resta
*	Multiplicación
**	16 bits superiores de la multiplicación
*/	16 bits medios de la multiplicación
/	División
//	Resto (módulo)
<<	Desplazamiento izquierdo
>>	Desplazamiento derecho
ABS	Valor absoluto
COS	Coseno
DCD	2m decodificador
DIG	Digito
MAX	Máximo *
MIN	Mínimo *
NCD	Codificar
REv	Invertir bits
SIN	Seno
SQR	Raiz cuadrada
&	Bit inteligente AND
÷	Bit inteligente OR
^	Bit inteligente EXCLUSIVE OR
~	Bit inteligente NOT
& /	Bit inteligente NAND
÷ /	Bit inteligente NOR

Operadores lógicos

Los operadores lógicos entregan un resultado CIERTO /FALSO de su operación. Valores 0 son tratados como falso. Cualquier otro valor es cierto. Se usan junto a operadores de comparación en una declaración **IF .. THEN**. Los operadores soportados son mostrados en la *tabla 8*:

Tabla 8. Operadores lógicos.

Operador	Descripción
AND o &&	AND lógico
OR o I I	OR lógico
XOR o ^ ^	OR exclusivo lógico
NOT AND	NAND logico
NOT OR	NOR logico
NOT XOR	NXOR logico

Otros comandos

En la *tabla 9* se muestran algunos de los comandos más comunes en PBP.

Tabla 9. Comandos básicos en PBP.

Comandos básicos en PICBASIC	
LOW	Hace bajo la salida de un pin
NAP	Apaga el procesador por un corto periodo de tiempo
ON INTERRUPT	Ejecuta una subrutina BASIC en un interrupt
OUTPUT	Convierte un pin en salida
PAUSE	Demora (resolución 1mseg.)
PAUSEUS	Demora (resolución 1 μseg.)
PEEK	Lee un byte del registro
POKE	Graba un byte en el registro
POT	Lee el potenciómetro en el pin especificado
PULSIN	Mide el ancho de pulso en un pin
PULSOUT	Genera pulso hacia un pin
PWM	Salida modulada en ancho de pulso a un pin
RANDOM	Genera número pseudo-aleatorio
RCTIME	Mide el ancho de pulso en un pin
READ	Lee byte de un chip EEPROM
RESUME	Continúa la ejecución después de una interrupción
REVERSE	Convierte un pin de salida en entrada o uno de entrada en salida
SERIN	Entrada serial asincronica (tipo BS!)
SERIN2	Entrada serial asincronica (tipo BS2)
SEROUT	Salida serial asincronica (tipo BS1)
SEROUT2	Salida serial asincronica (tipo BS2)
SHIFTIN	Entrada serial sincronica
SHIFOUT	Salida serial sincronica
SLEEP	Apaga el procesador por un periodo de tiempo
SOUND	Genera un tono o ruido blanco en un pin
STOP	Detiene la ejecucion del programa
SWAP	Intercambia los valores de dos variables
TOGGLE	Hace salida a un pin y cambia su estado
WRITE	Graba bytes a un chip EEPROM
XIN	Entrada X - 10
XOUT	Salida X - 10

A lo largo del documento se irán presentando más comandos. De igual manera puedes consultar la ayuda del PBP en la opción **“HELP TOPICS”** para poder estudiar más comandos o también puedes consultar el manual **“Compilador PicBasic Pro”** que te puede otorgar tu profesor.

4.2 Prácticas de Laboratorio No.2.- Leer el estado de un puerto.

Integrantes:

1. _____
2. _____
3. _____
4. _____

Objetivo general:

- Crear un programa que lea el estado de un puerto del PIC que servirá como señal de mando para controlar una acción.

Objetivos particulares:

- Estudiar el funcionamiento del comando IF...ENDIF.
- Conectar un push-button como dispositivo de control y lograr que el PIC interprete la señal de mando.

Equipo:

- PC de escritorio o laptop.
- Programador PIC-600.
- Cable USB.
- Protoboard.

Material a utilizar:

- PIC16F84A
- 1 Resistencia de 330 Ω .
- 2 Resistencia de 100 Ω .
- 2 Resistencia de 10 K Ω .
- 1 Push-button.

- 1 LED del color deseado.
- 1 Cristal oscilador de 4 MHz
- Cables para conexión.

Desarrollo de la práctica.

1. Arme el circuito que se muestra en la *figura 33* en su protoboard.

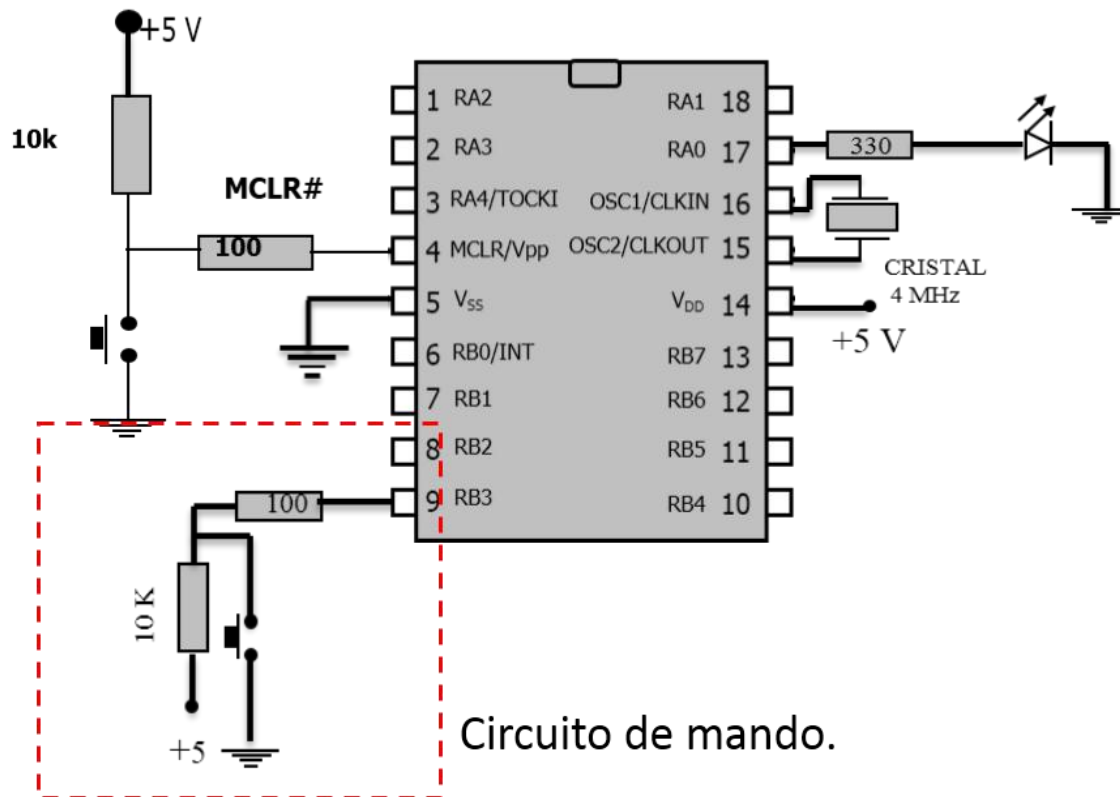


Figura 31. Circuito de control de la práctica 2.

2. Proceda a escribir el siguiente código.

```
i var byte
loop:
  IF PORTB.3=0 THEN
    FOR i=1 TO 5
      High PORTA.0      ' Encender el PORTA.0
      Pause 250         ' Pausar 250 ms.
      Low PORTA.0       ' Apaga el PORTB.0
      Pause 250         ' Pausar 250 ms
    NEXT i
  ENDIF
  Goto loop
End
```

3. Realiza el proceso anterior, configure El MicroCode Studio Pro, compile el código, abra el PIC-600 y configúralo, coloque el PIC en el módulo, por ultimo carga el programa **“.HEX”** dentro del PIC. Observa como trabaja el circuito con el código ya cargado.

Anote sus conclusiones individualmente:

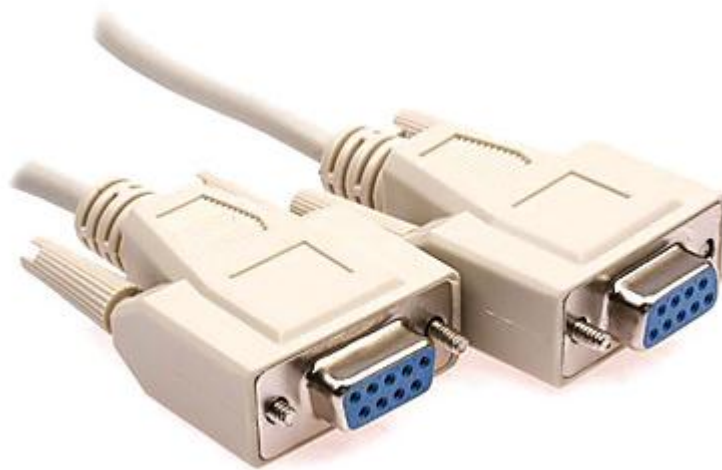
1. _____

2. _____

3. _____

4. _____

5. Comunicación Serial entre el PIC y la PC



5.1 Marco teórico

5.1.1 Comunicación Serial

Los microcontroladores PIC tienen muchas aplicaciones, entre las que se encuentran la comunicación con otros dispositivos, como el caso de una PC. La programación basada en lenguaje PicBasic cuenta con comandos que permiten este tipo de comunicación, los cuales se describirán más adelante.

La comunicación puede ser por puerto serial o incluso por USB, sin embargo para fines de este manual solamente se analizará la comunicación serial por medio del conector **DB9**. El conector DB9 cuenta con nueve pines y cada uno tiene su función, las cuales se describen en la tabla. Existen dos tipos de conector DB9: el conector MACHO y el conector HEMBRA que se muestran en la *figura 34*.

Tabla 10. Terminales del conector DB9.

#	Pin	E/S	Función
1			Tierra de chasis
2	RXD	E	Recibir datos
3	TXD	S	Transmitir datos
4	DTR	S	Terminal de datos lista.
5	SG		Tierra de señal.
6	DSR	E	Equipo de datos listo.
7	RTS	S	Solicitud de envío.
8	CTS	E	Libre para envío.
9	RI	S	Timbre telefónico.

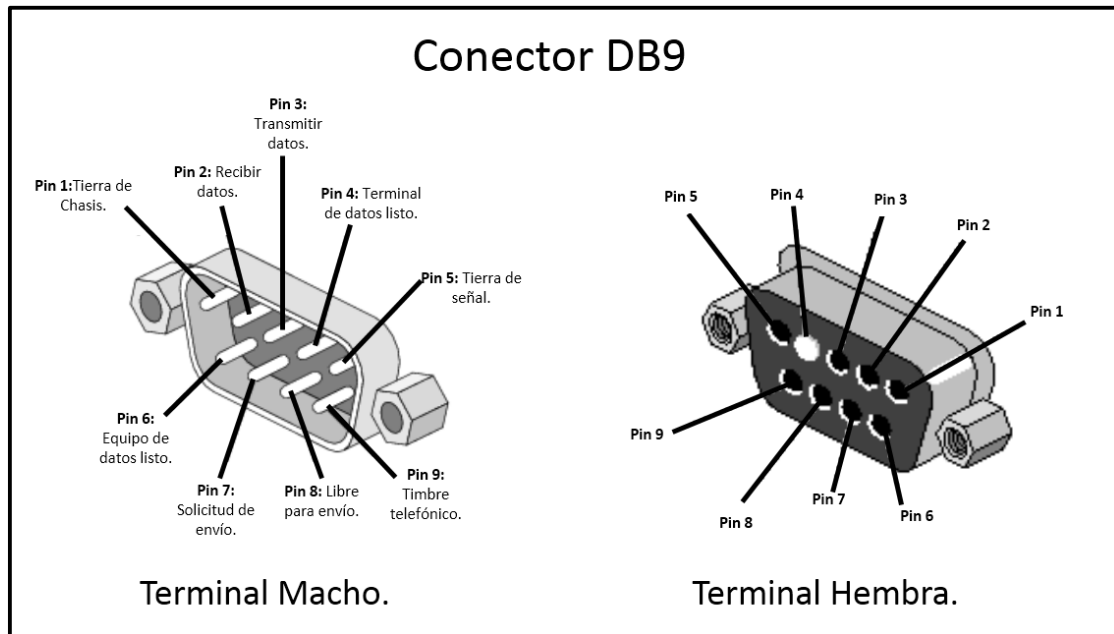


Figura 32. Conector DB9

Para que la PC reciba datos del microcontrolador, se selecciona un puerto del PIC disponible, el cuál enviará los datos, estos deben de transmitirse por medio del pin número **2** del conector DB9 hacia la PC, además entre ellos se conectará una resistencia en serie de $1K\Omega$ para regular la corriente y evitar dañar la terminal elegida para la transmisión. Por último se conecta el pin número **5** del conector DB9 que será la referencia a tierra de la señal enviada, *figura 35*.

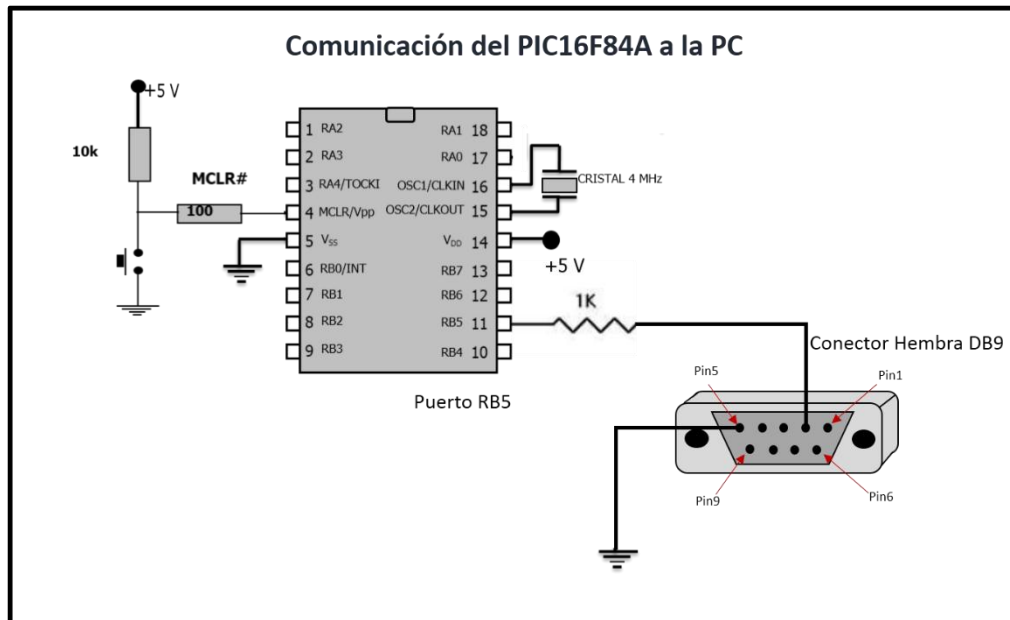


Figura 33. Envío de datos del PIC a la PC por conexión serial.

En caso contrario, si lo que se desea es enviar datos de la PC a al microcontrolador PIC se usará el pin número **3** del DB9 que tiene la función de transmitir datos. También se conectará una resistencia de $1k\Omega$ en serie y el pin número **5** a tierra. Elija un puerto disponible, en la *figura* se puede observar el puerto RB4 para la recepción de datos de la PC.

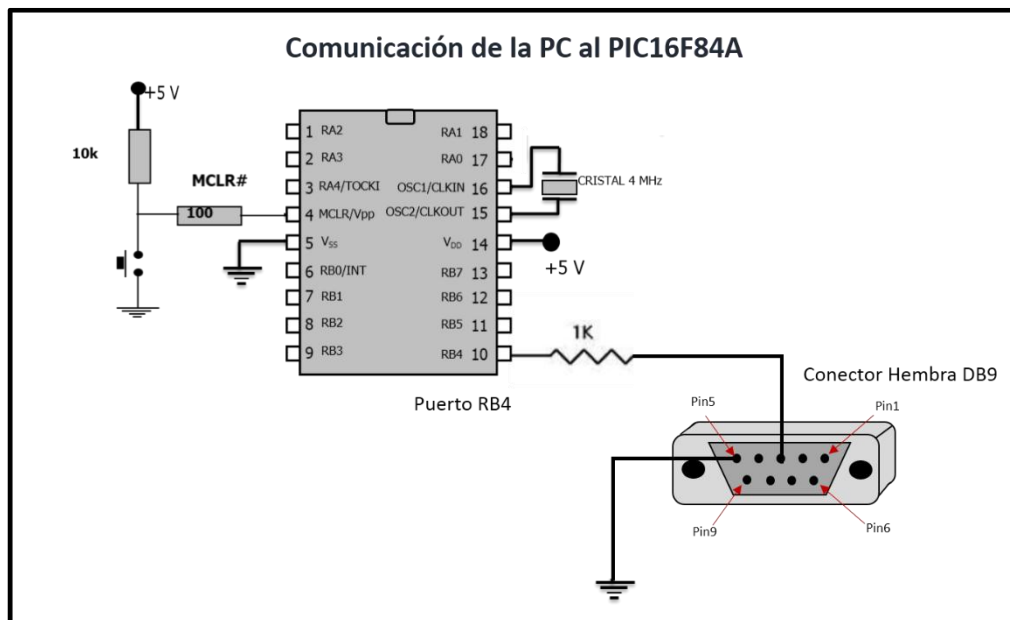


Figura 34. Recepción de datos de la PC al PIC por conexión serial.

5.1.2 Comandos sugeridos

Para los ejercicios de esta sección se recomienda aplicar los siguientes comandos:

Instalación de librerías.

Para realizar la comunicación serial es necesario incluir en nuestro programa los comandos que configurarán al puerto COM de nuestra PC, configuraciones como: la velocidad de transferencia de datos *"baud rate"*, los bits de datos, los bits de parada, etc. Sin embargo dentro de las librerías que se instalaron junto con el MicroCode Studio se encuentran subprogramas que contienen ya los códigos con estos datos. Estas líneas de código fuente se encuentran ubicadas en el subprograma ***"modedefs.bas"*** y son insertadas dentro de nuestro programa exactamente donde se coloca el comando ***"INCLUDE"***.

Ejemplo:

```
INCLUDE "modedefs.bas" 'Llama las instrucciones almacenadas dentro del subprograma  
'modedefs.bas que contiene los datos de configuración de la  
'comunicación serial.
```

El comando **INCLUDE "modedefs.bas"** lo escribirás justo al inicio de tu código, antes de cualquier otro comando, como se muestra en la *figura 37*.

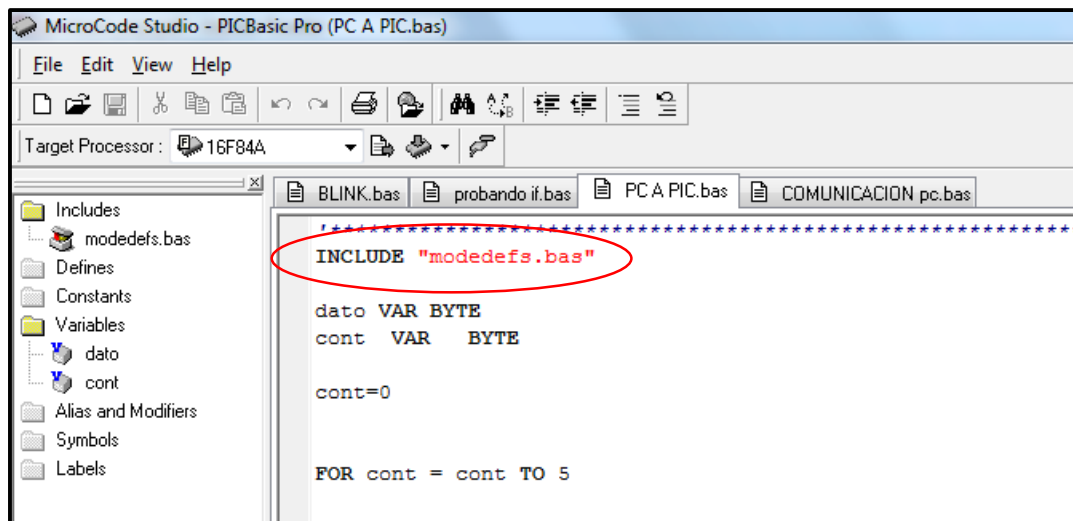


Figura 35. Incluir librerías en la programación.

El comando **INCLUDE** contiene algunos datos como los mostrados en la tabla.

Tabla 11. Comando INCLUDE.

Mode	Mode N°	Baud rate	Estado
T2400	0	2400	LLEVADO A CIERTO
T1200	1	1200	
T9600	2	9600	
T300	3	300	LLEVADO A INVERTIDO
N2400	4	2400	
N1200	5	1200	
N9600	6	9600	
N300	7	300	ABIERTO CIERTO
OT2400	8	2400	
OT1200	9	1200	
OT9600	10	9600	ABIERTO INVERTIDO
OT300	11	300	
ON2400	12	2400	
ON1200	13	1200	
ON9600	14	9600	
ON300	15	300	

NOTA: No olvides escribir el comando `INCLUDE "modedefs.bas"` al inicio de tu programa, si ignoras este pequeño código no funcionará la comunicación serial, aún si usas los comandos `SERIN` y `SEROUT`.

Enviar datos del PIC a la PC por medio del cable serial.

La instrucción para el envío de datos del PIC a la PC es muy sencilla y basta con escribir el siguiente comando:

Estructura: `SEROUT Pin,Modo,["cadena",13,10]`

Ejemplo:

`Serout 4,N2400,["Hello, World!",13,10]` ‘ El PIC manda a la PC la palabra **HELLO**
WORLD por comunicación serial.

- **SEROUT** es el comando para enviar datos del PIC a la PC.

- El número **4** representa el pin por medio del cuál se mandarán los datos a la PC, en este caso será por "PORTB.4". De igual manera es válido escribir SEROUT PORTB.4[".....",13,10], cualquiera es válida.
- **N2400** es tiempo usado para transmitir cada dato y es conocido como "*baudrate*", ambos dispositivos el PIC y la PC (transmisor y receptor) deben de tener el mismo "*baudrate*" para poder sincronizarse y comunicarse para evitar que los datos lleguen ilegibles.
- La palabra que se encierre entre comillas, en el ejemplo "**Hello World**", será la cadena de caracteres que se tendrá que transmitir por comunicación serial.
- El número **13** es conocido como "*retorno de carro*", debido que antiguamente en las máquinas de escribir el "carro" o el rodillo en donde se colocaba la hoja se iba desplazando con cada letra que se escribía, al llegar al tope de la hoja se tenía que recorrer de nuevo el "carro" al principio, para continuar con otra línea y empezar desde la parte izquierda de la hoja.
- El número **10** es el salto de línea, que funcionaría como el ENTER del teclado de tu PC.

Recuerda que para enviar datos del PIC a la PC necesitaras el pin número 2 del puerto DB9.

Envío de variables a la PC con SEROUT

Para enviar el valor de una variable a la PC por comunicación serial se ocupa el símbolo # precedido del nombre de la variable, observe el ejemplo.

Ejemplo:

```
Serout 4,N2400,["Contador = ",#cont,13,10] ‘El PIC manda a la PC el valor de la variable  
‘cont
```

Enviar datos de la PC al PIC por medio del cable Serial.

Cuando se quiere realizar lo contrario, mandar información de la PC al PIC se usa el comando **SERIN**, este comando nos permite mandar un solo carácter por comunicación serial y poder guardarlo en una variable.

Esctructura: SERIN Pin,Modo, ["Carácter"],variable

Ejemplo:

SERIN PORTB.4,N2400,["C"],dato *'La PC manda al PIC el carácter "C" y lo guarda en la variable "dato"*

- La palabra **dato** hace referencia a la variable donde se almacenará el carácter (en este caso la letra "C").

NOTA: Con el comando SERIN solamente podrás enviar un carácter, si requieres mandar una palabra tendrás que utilizar otros comandos los cuales no se usarán en este manual. Si lo deseas, puedes consultar más comandos en la Ayuda del MicroCode Studio en la opción de "Help Topics", o en el manual llamado "Compilador PicBasic Pro" el cuál te podrá facilitar tu profesor que imparte la asignatura de "Arquitectura de Sistemas Digitales".

Recuerda para enviar datos de la PC al PIC necesitaras el pin número **3** del puerto DB9.

5.2 Práctica de Laboratorio No.3.-Comunicación Serial.

Integrantes:

1. _____
2. _____
3. _____
4. _____

Objetivo general:

Establecer comunicación entre el PIC y la PC por medio de comunicación serial.

Objetivos particulares:

- Estudiar el funcionamiento del comando SERIN.
- Estudiar la configuración de las terminales del conector DB9.

Equipo:

- PC de escritorio o laptop.
- Programador PIC-600.
- Cable USB.
- Cable Serial del tipo DB9.
- Protoboard.

Material a utilizar:

- PIC16F84A
- 1 Resistencia de 330 Ω .
- 2 Resistencia de 1 K Ω .
- 1 Resistencia de 100 Ω .
- 1 Resistencia de 10 K Ω .
- 2 LED del color deseado.
- 1 Cristal oscilador de 4 MHz
- Cables para conexión.

Desarrollo de la práctica.

1. Arme el circuito que se muestra en la *figura 38* en su protoboard.

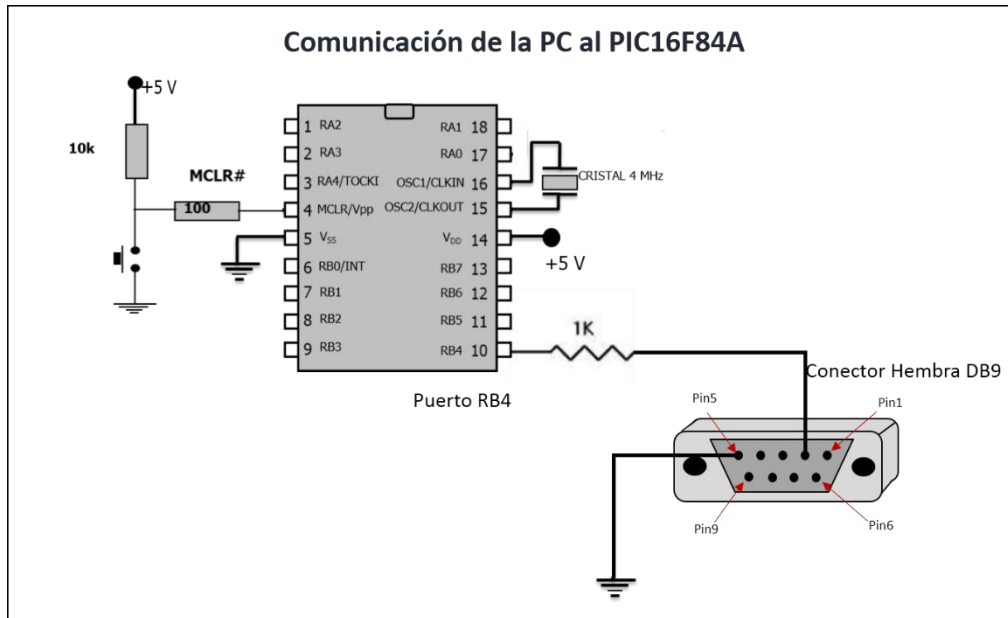


Figura 36. Comunicación PIC-PC.

2. Como se observa en el circuito armado, un extremo del cable serial irá conectado al PIC, el otro extremo del cable serial a la PC. Si tu PC cuenta con dos o más puertos seriales, observe en cual de ellos realizará la conexión, normalmente están señalados por letras (**A, B, etc.**). Si su PC no cuenta con una entrada del tipo DB9 puede usar un adaptador que convierta el conector DB9 a USB. El driver para este dispositivo se puede encontrar muy fácilmente en Internet, *figura 39*.

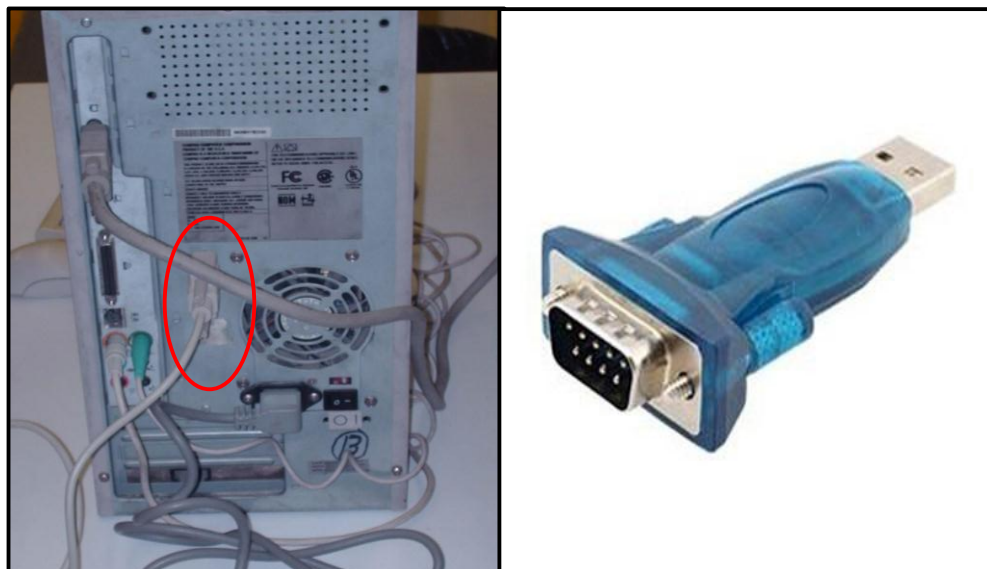


Figura 37. Conexión del cable serial a la PC.

3. Una vez que conecte el cable serial a la PC proceda a ejecutar el administrador de dispositivos de Windows. Diríjase a la opción **“Puertos (COM & LPT)”**, despliegue el menú y podrá observar los puertos de comunicaciones con los que cuenta su PC. En la *figura 40* se muestra que el Sistema Operativo sólo tiene identificado un puerto, que denomina **“Puerto de comunicaciones COM1”**. Recuerde si su PC físicamente tiene más de un puerto serial del tipo DB9, tiene que considerar en que puerto realizó la conexión. Se mencionó en el paso anterior que la PC normalmente trae nombrados sus puertos con letras **(A,B,C,etc)**. Por ejemplo si usted lo conecto en el puerto **B** el sistema seguramente le asignara el nombre **“Puerto de Comunicaciones COM2”**

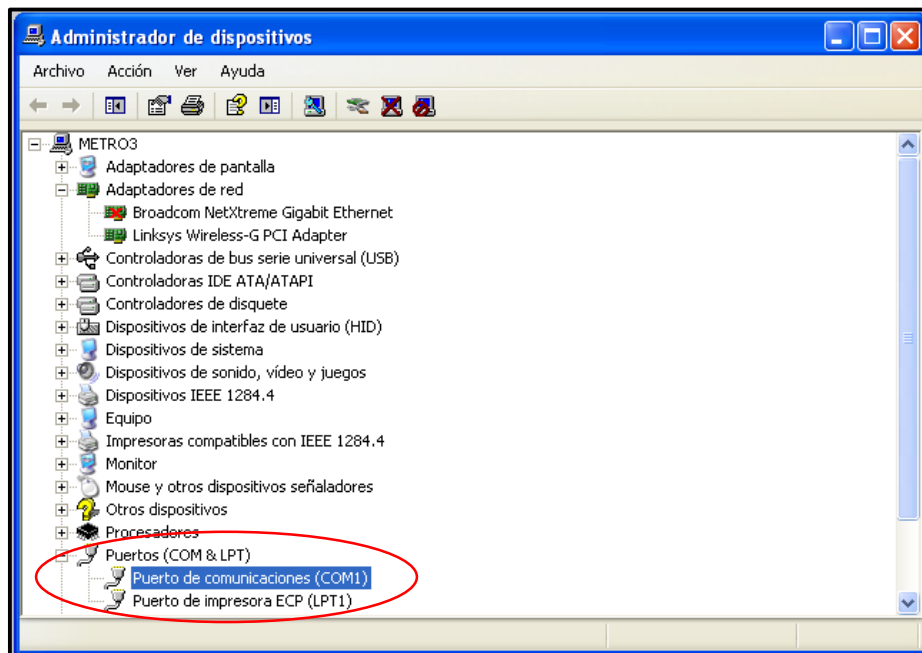


Figura 38. Puertos COM instalados en la PC.

4. Una vez que este seguro del puerto donde conecto el cable serial, haga clic derecho sobre la opción **“Puerto de Comunicaciones COM1”** para desplegar el menú y luego haga clic en **propiedades**. En caso de no realizar la conexión en el COM1, haga el mismo proceso pero en el número de COM que usted eligió.

Una vez hecho este paso, se abrirá una ventana donde tendrá que hacer clic en la pestaña **“Configuración de puerto”**. Proceda a configurar los datos del puerto como se muestra en la *figura 41*. Esta parte es importante para la correcta comunicación entre el PIC y la PC, por lo tanto no debe de saltarse este paso. Al finalizar haga clic en **Aceptar**.

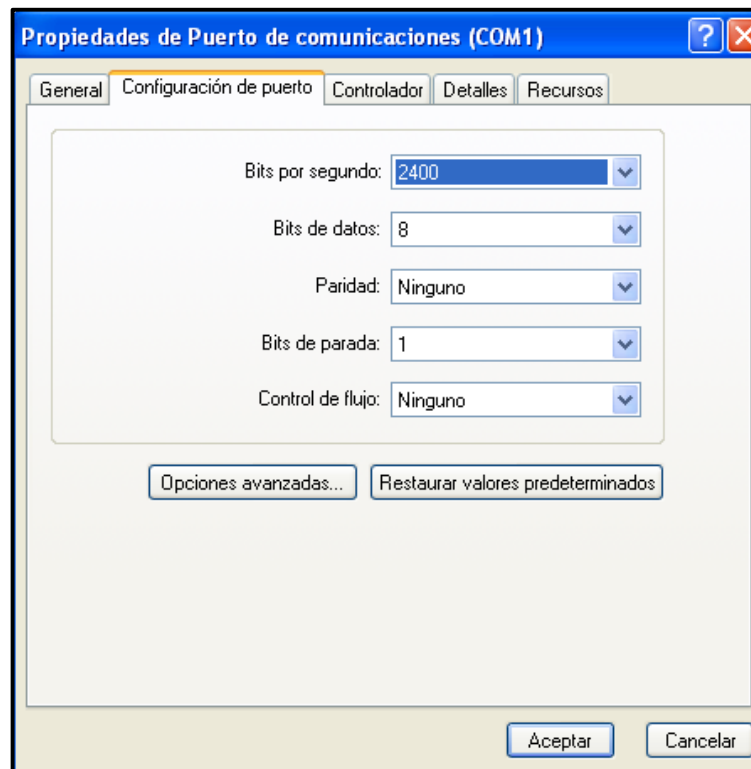


Figura 39. Configuración del puerto COM.

5. Ejecute el MicroCode Studio y proceda a capturar el siguiente código. Recuerde que el modelo del PIC que se usará será el **16F84A**, no olvide configurar el MCS como se mostró en las primeras prácticas.

```

Include "modedefs.bas"      ' Include serial modes

loop:

    Pause 1000                ' Pausar 1000 ms
    High PORTA.0
    Serout 4,N2400,["Arquitectura",13,10]    ' Salida en PORTB.4

    pause 1000
    low PORTA.0
    serout 4,N2400,["De Sistemas Digitales",13,10]

Goto loop                    ' infinito
  
```

6. Guarde su programa y compílelo.
7. Retire por un momento el PIC16F84A del circuito que armo de acuerdo al paso 1 y proceda a cargar el programa en el microcontrolador.
8. Coloque de nuevo el PIC a su circuito y alimente con una fuente de 5V de CD.
9. Ahora abra de nuevo el MCS y haga clic en el botón **“Serial Communications Windows – F4”** de la barra de herramientas, que se muestra encerrado en el círculo de color rojo de la *figura 42*.

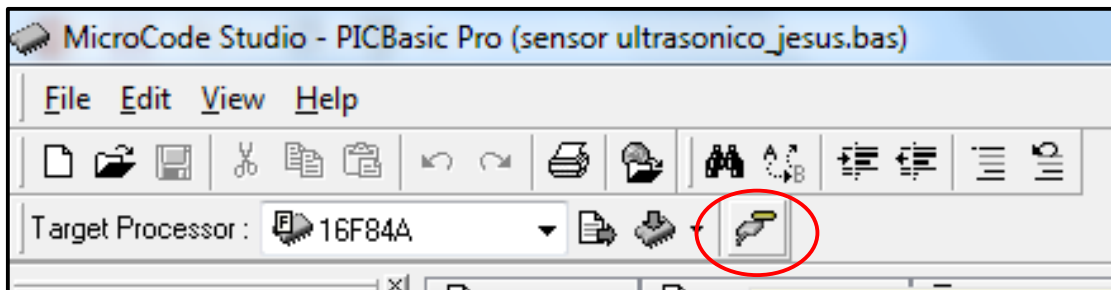


Figura 40. Botón para activar la comunicación serial.

10. Se abrirá una ventana idéntica a la de la *figura 43*. Observe en la parte izquierda de la figura se encuentran los mismos datos que configuramos en el **“Administrador de dispositivos de Windows”**. Es importante que configure con los mismos parámetros esta ventana, observe la *figura 44*.

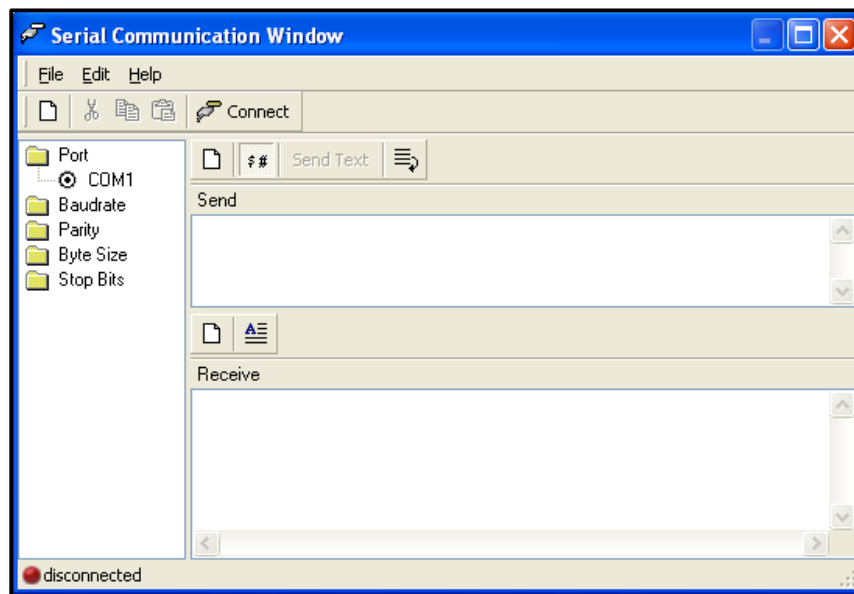


Figura 41. Serial Communication Window.

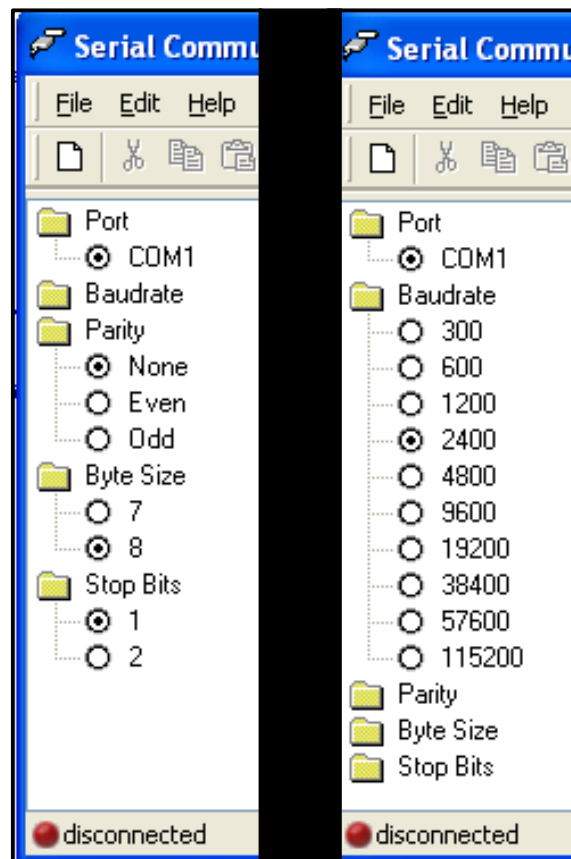


Figura 42. Configuración de la herramienta "Serial Communication Window"

12. Una vez configurado los parámetros, hacemos clic en el botón con la leyenda "connect", figura 45.

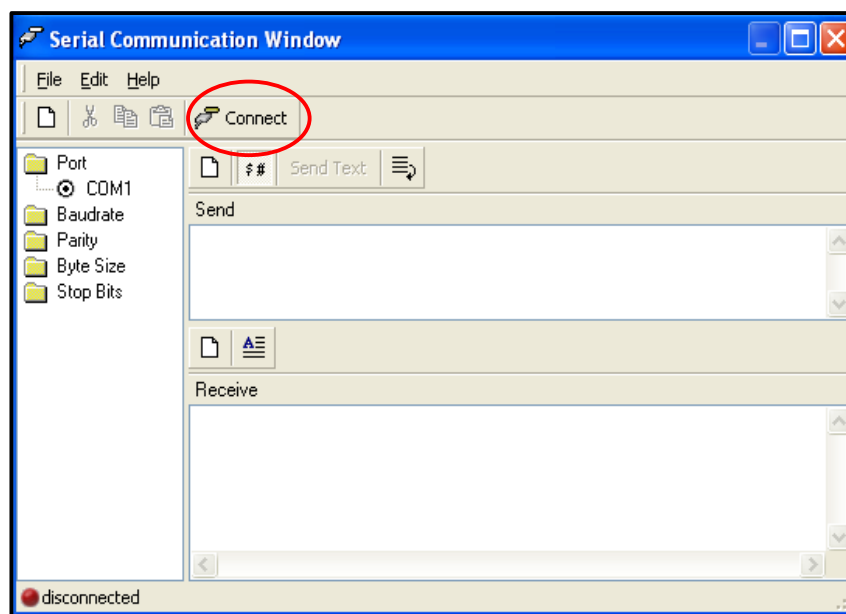


Figura 43. Conexión entre el PIC y la PC.

Al oprimir el botón “connect” ya podrá observar los datos que el PIC16F84A envía por comunicación serial a la PC, observe la figura 46.

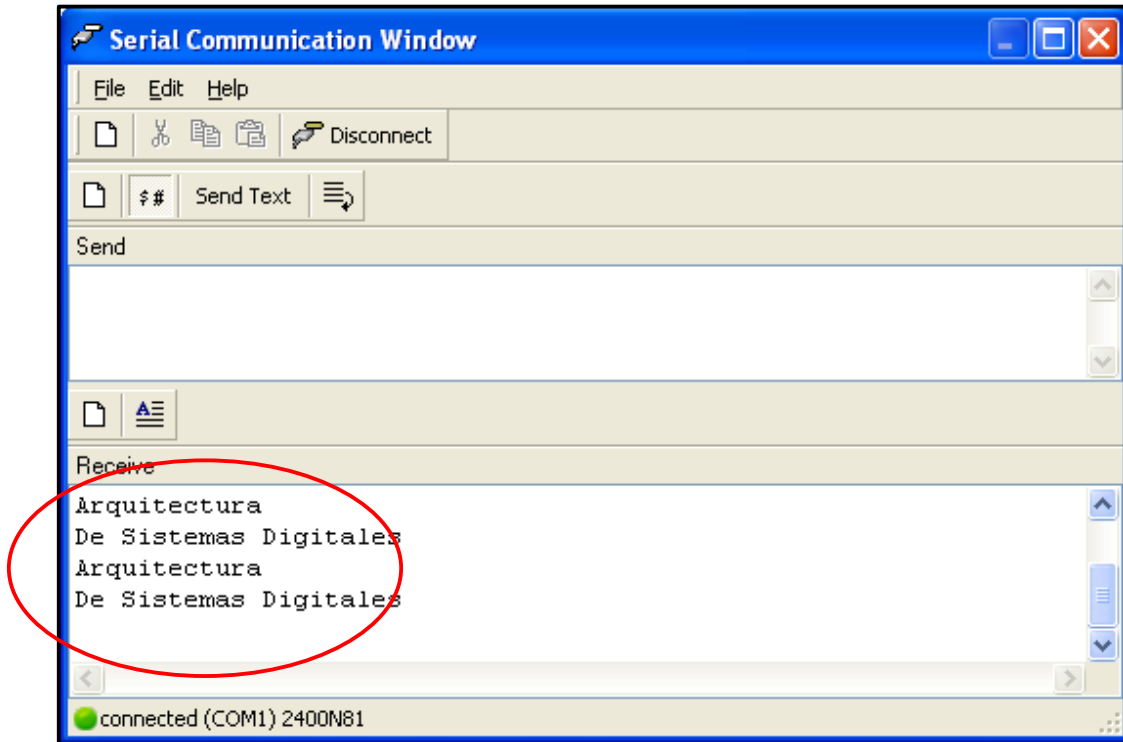


Figura 44. Conexión establecida entre el PIC y la PC.

Anote sus conclusiones individualmente:

1. _____

2. _____

3. _____

4. _____

5.3 Problemas reto.

1. Comunicación de la PC al PIC.

Ya vimos cómo realizar la comunicación del PIC a la PC, ahora tendrás que realizar lo contrario.

1. Arma el circuito de la *figura 47*.

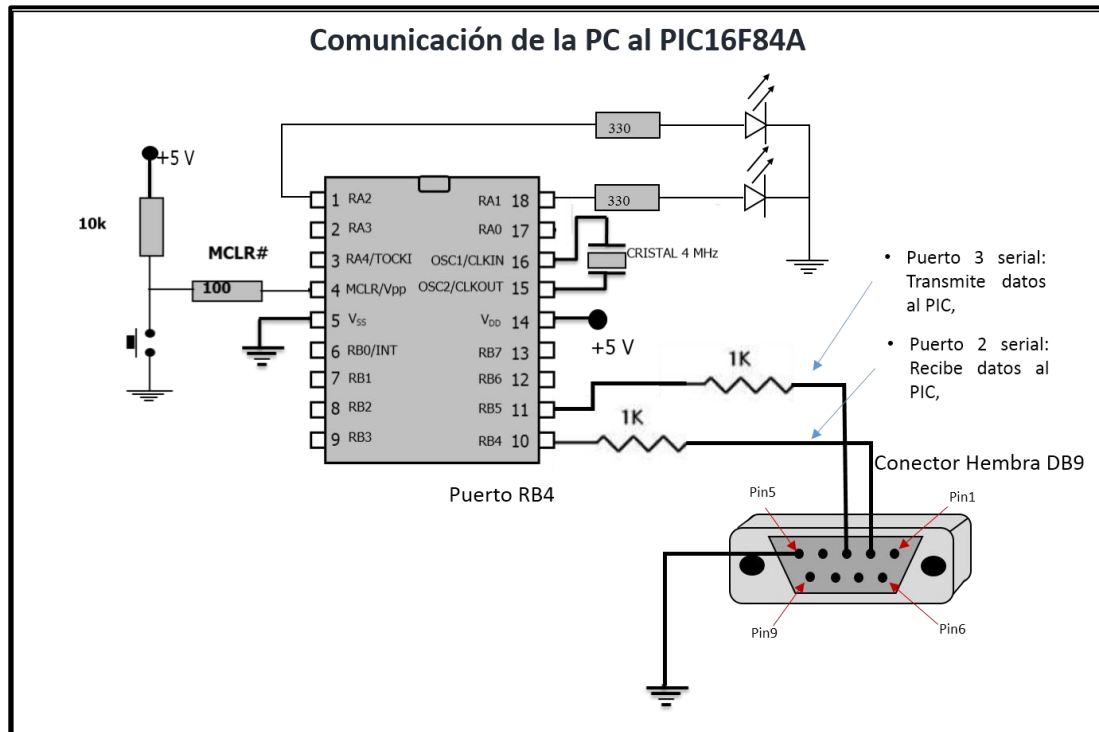


Figura 45. Diagrama de conexión para el envío de datos de la PC al PIC.

2. El programa tendrá que realizar un conteo de 1-5, por cada valor que aumente el contador, el PIC por medio del puerto B.4 tendrá que mandar el siguiente dato a la PC “*contador= valor del contador*”, usando el pin2 del puerto serial, como se muestra en la *figura 48*, al mismo tiempo por cada conteo el LED conectado al **puerto A.1** hará un parpadeo.

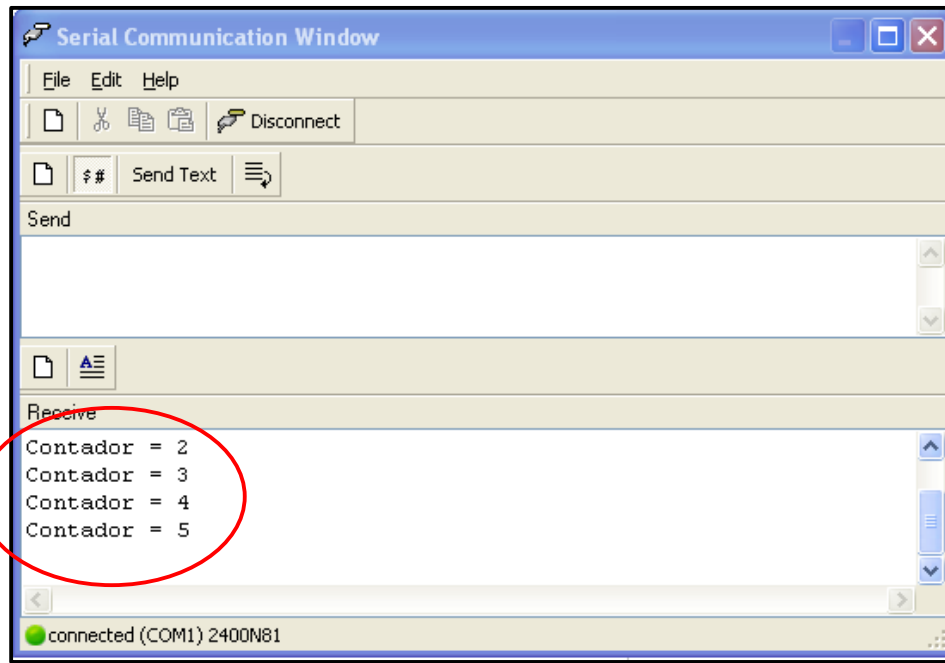


Figura 46. Envío de una variable por comunicación serial.

3. Una vez que el contador llegue a 5 se detendrá el conteo, y encenderá el LED conectado al **Puerto A.2**, indicando que el conteo ha terminado.

4. Para reiniciar el contador la PC deberá mandar una orden usando el **pin3** del puerto serial al **puerto B.5** del PIC. En este caso el conteo se deberá reiniciar cuando la PC envíe el **carácter "C" mayúscula**. El PIC deberá mandar la leyenda "**---->Contador inicializado--->**", como se observara en la *figura 49*. El LED que indica que el contador ha finalizado deberá apagarse y esperar que el conteo termine nuevamente.

Recuerda usar el comando **SERIN** para enviar datos al PIC y **SEROUT** para que el PIC mande a la PC. El carácter que tienes que enviar lo deberás de capturar en el área de texto con la leyenda "**send**" y después enviarlo al PIC dando clic en el botón "**Send Text**", *figura 49*.

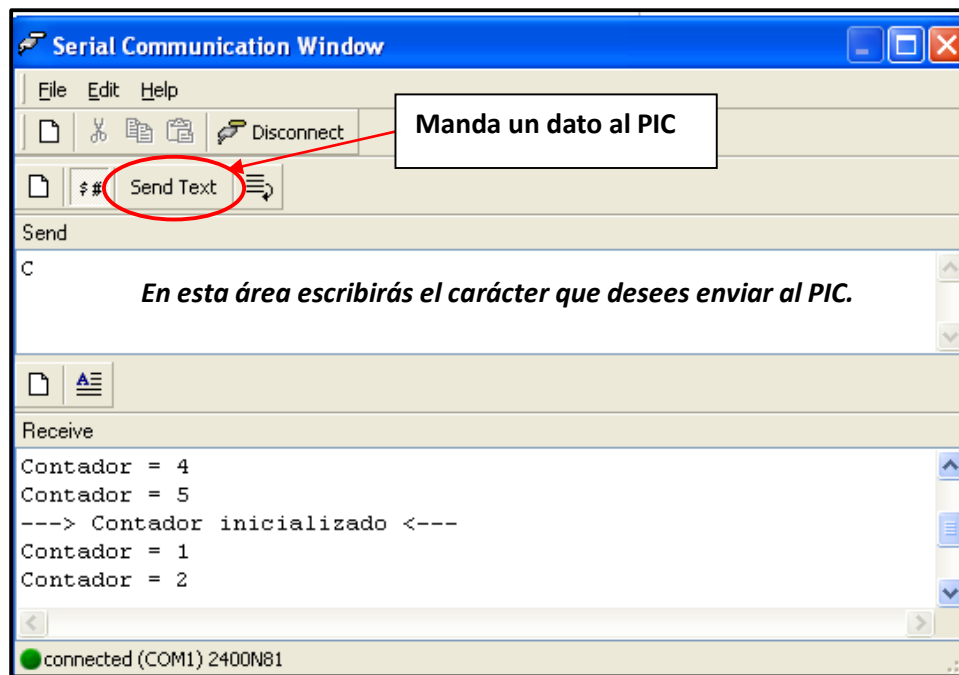
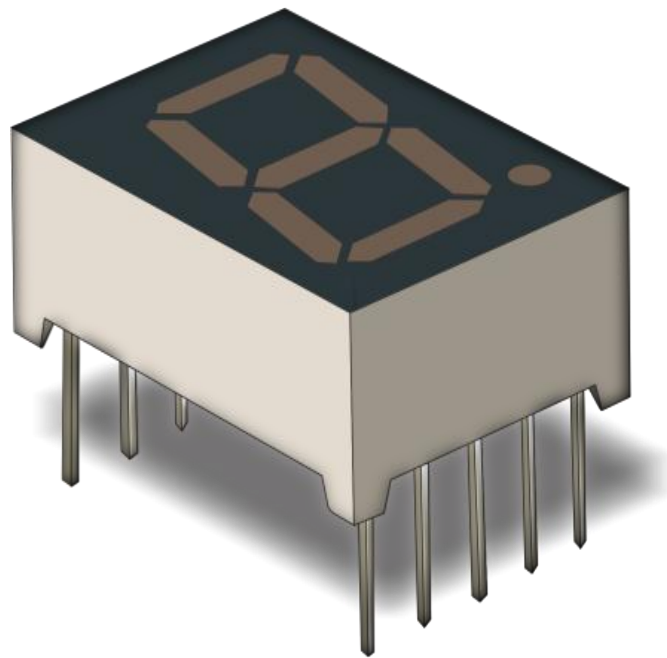


Figura 47. Envío de datos de la PC al PIC.

6. Programación aplicada usando displays de 7 segmentos.



6.1 Marco teórico.

6.1.1 El display de 7 segmentos

El visualizador de 7 segmentos o display de siete segmentos es un componente electrónico que se utiliza para la representación de números y es muy popular debido a su simplicidad al usarlo en aplicaciones electrónicas. Se le denomina display de 7 segmentos por que internamente contiene un arreglo de 7 LED's (figura 50) colocados estratégicamente para formar un número ocho, donde a cada LED se le puede encender o apagar individualmente mediante las terminales o pins externas al componente.

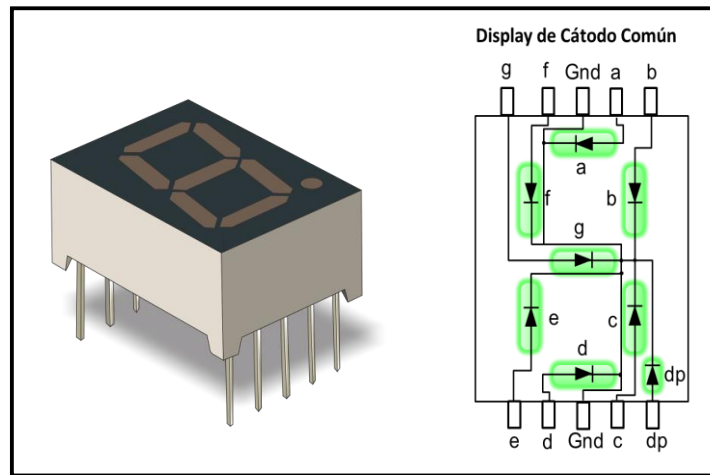


Figura 48. Estructura interna de un display de 7 segmentos.

Existen dos tipos de displays de acuerdo a su conexión interna o topología. El primero es el display de cátodo común (figura 51), donde todos los cátodos de los LED's internos están interconectados entre sí, teniendo en común una sola conexión la cual es accesible por medio de una de las terminales del componente, el fabricante indicará que esa terminal debe de estar conectada a tierra (ground). Para este display el microcontrolador deberá mandar a cada uno de sus segmentos (a,b,c....g) un 1 digital para polarizar directamente los LED's y así poder encenderlos.

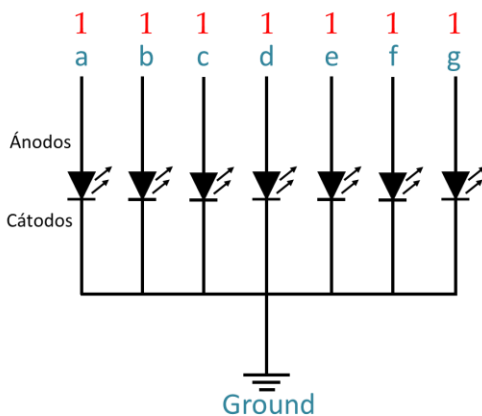


Figura 51. . Estructura de un display de cátodo común.

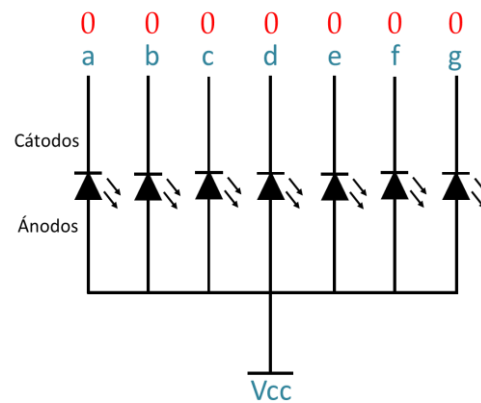


Figura 52. Estructura interna de un display de ánodo común.

El segundo es el display de ánodo común (*figura 52*) en este caso todos los ánodos estarán interconectados. En esta topología, la terminal en común deberá estar conectada a una fuente de CD indicada como Vcc por el fabricante. Para esta configuración el microcontrolador deberá mandar un 0 digital para polarizar directamente los LED's. Recordar que la configuración de las terminales dependerá del fabricante, por lo tanto antes de realizar una conexión hay que consultar la hoja de datos o *datasheet* para que el display opere correctamente.

Para formar los números se debe de decidir que segmento se encenderá, puedes guiarte de la figura . Si se usa un display de cátodo común, los segmentos del display encenderán con “unos” digitales como se mencionó anteriormente. En la *tabla 12* se muestra la secuencia de 0 y 1 digitales que el microcontrolador deberá enviar al display para formar cada uno de los números del 0 al 9.

Tabla 12. Secuencia de valores para formar números en el display.

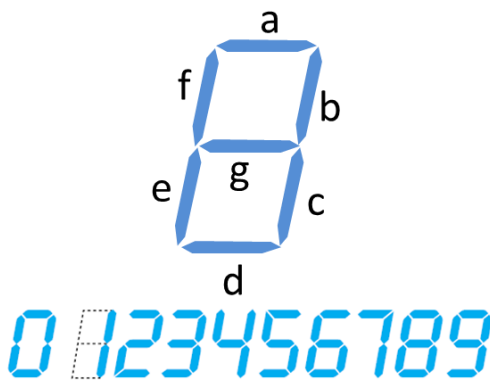


Figura 49. Los 7 segmentos del Display.

No. Decimal	a	b	c	d	e	F	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1

6.1.2 Multiplexación con Displays

La multiplexación en Displays consiste en transmitir los datos de los segmentos (a,b,c...g) por medio de los puertos del microcontrolador a múltiples displays simultáneamente, pero controlando al mismo tiempo el encendido de cada uno de ellos y así poder elegir en cual display se mostrarán los números.

Suponga que requiere mostrar el **número 285** en tres displays de cátodo común como se observa en la *figura 54*, usando la técnica de la multiplexación el microcontrolador inicialmente deberá mandar las

instrucciones que generan el primer dígito a través de sus puertos **B**, en este caso el número **5**. El problema será que el número **5** se visualizará en los tres displays al mismo tiempo. Para que esto no suceda el PIC deberá controlar por medio de sus puertos **A**, el encendido y apagado de cada display, permitiendo elegir en cuál de ellos se mostrará el primer dígito. Este control se realiza mediante los transistores (**T0, T1 y T2**): en este caso transistores BJT del tipo NPN. Si el transistor no es excitado en su base, el display estará inactivo, por lo tanto el único transistor que recibirá una señal de activación por parte del microcontrolador será el **T0**, para así poder visualizar el dígito **5** en el display **D0**. Para mostrar el segundo dígito el microcontrolador deberá apagar el display **D0**, posteriormente activar solamente **D1** y mandar las instrucciones del dígito **8**. Por último se realizará el mismo procedimiento para el tercer dígito, solamente estará activado el display **D2** el cuál recibirá las instrucciones para formar el número **2**. Este control de encendido y apagado de displays se le conoce como multiplexación.

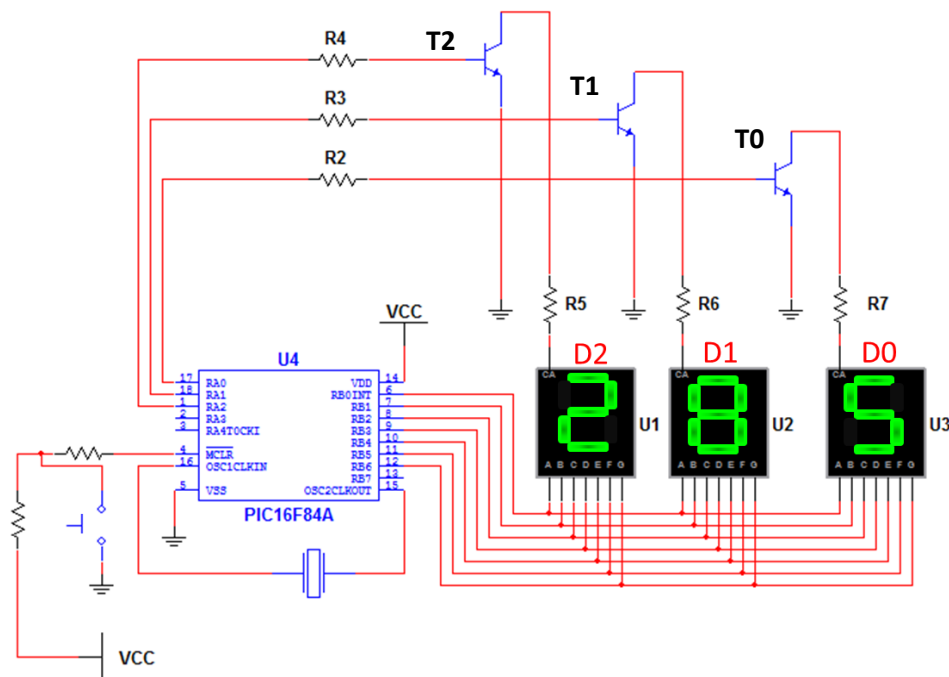


Figura 50. Multiplexación con Displays de 7 segmentos.

Ahora solamente falta visualizar el número **285** completamente estático sin ver la transición debida al encendido y apagado de los displays. Para esto se aprovecha el defecto que tenemos los humanos en los ojos la “persistencia de la retina”, donde nos da la sensación de ver una imagen cuando esta ya ha desaparecido. Este defecto de los ojos es usado positivamente ya que nos permite ver películas, siendo estas secuencias de imágenes, pero pasando tan rápido que no logramos ver los cambios. Incluso nos permite ver la luz de un foco sin poder darnos cuenta que este tiene una intermitencia de 60 Hz. Por lo tanto la transición entre displays se tiene que realizar a frecuencias tan rápidas para que la retina no alcance a percibir el cambio y así poder ver el número 285 estático. Prácticamente no somos capaces de percibir la intermitencia a frecuencias superiores a los 30 Hz, por lo tanto al momento de realizar la programación se debe de tomar en cuenta los tiempos de transición.

Las conexiones con los transistores dependerán si se está usando un display de ánodo o de cátodo común. En las figura 55 se muestra el diagrama de conexión para cada uno de ellos.

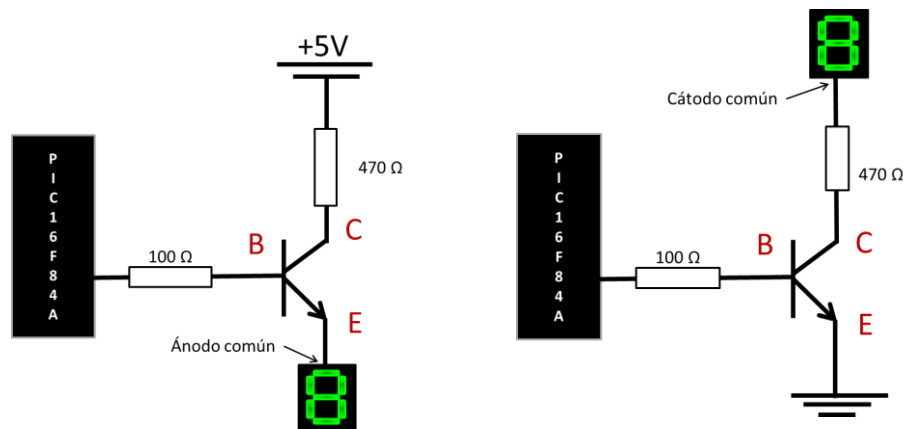


Figura 51. Diagramas de conexión para un display de ánodo y cátodo común.

6.1.3 Comandos sugeridos

Para los ejercicios de esta sección se recomienda aplicar los siguientes comandos:

Comando “LOOKUP”

Puede ser usado para obtener valores de una tabla de constantes de 8 bits. Si “Index” es 0 “Var” toma el valor de “constante0”. Si “Index” es 1, “Var” toma el valor de “constante1” y así sucesivamente.

Estructura: LOOKUP +puerto, [Constant0, Constant1,...constanten}], + Variable

La lista de constantes puede ser una mezcla de constantes numéricas y cadenas.

Ejemplo:

```
FOR B0 = 0 to 3           ' Cuenta del 0 al 3
LOOKUP B0,[ "Hola" ],B1 ' Obtiene el carácter B0 de la cadena y lo guarda en B1
Serout 0,N2400, [ B1 ]   ' envía el carácter en B1 al Pin0 en formaSerial
Next B0                  ' Siguiente conteo.
```

Comando “DIG”

Asigna el valor de un dígito decimal. Simplemente se le indica el número de dígito a conocer (0-4, siendo 0 el primero de la derecha). Este comando puede ser útil al momento de realizar un conteo y elegir el dígito a mostrar en el display.

Ejemplo:

```
B0 = 123                ' Asigna a la variable B0 el valor de 123
B1= B0 DIG 1            ' Asigna a B1 el valor de 2 (dígito 1 de 123)
```

Comando “DCD”

Devuelve un valor decodificado de un número de bits. Cambia un número de bit (0 a 15) de una variable por un “1”. Todos los demás bits son 0.

Ejemplo:

```
B0 = DCD 2              ' Asigna a B0 el valor %00000100
```

El comando **DCD** puede ser útil al momento de mandar la señal de activación a los transistores para elegir el display que encenderá y mostrará el dígito deseado. El siguiente ejemplo es parte de un código para un PIC16F84A donde se requiere mostrar números de 4 cifras por lo que se necesitará controlar 4 displays con el multiplexado. El control se hará por medio de los puertos A del PIC (A.0, A.1, A.2 y A.3), ver figura 56.

Ejemplo para un PIC16F84A:

```

Digits VAR PORTA      ' Asigna a los 4 puertos de PIC16F84A el nombre Digits. Control de
                        ' displays de 0-3.

i VAR Byte             ' Variable que realizará el conteo de los displays.

For i = 0 to 3         ' En el primer conteo la variable i toma el valor de 0.
    Digits = DCD i     ' Enciende el primer dígito y el resto permanecen apagados.
Next i                 ' Siguiente conteo.
  
```

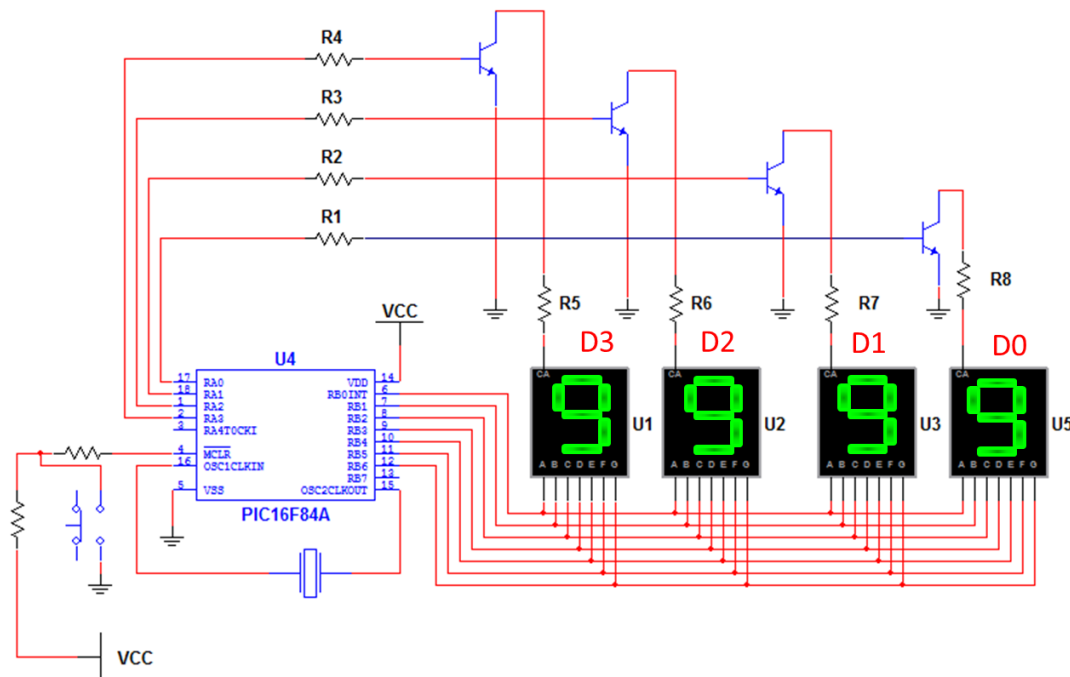


Figura 52. Multiplexeo con 4 displays.

En este ejemplo, al renombrar “Digits” al conjunto de los puertos A (RA.0-RA.4) se logra un control más sencillo de los 4 displays. Al momento de realizar el conteo (0-3) con el **FOR**, el comando **DCD** irá encendiendo los displays. Tome en cuenta que el valor

de “1” manda la señal de encendido del display, mientras en “0” estará inactivos. Lo anterior puede observarse en la *tabla 13*.

Tabla 13. Decodificación del comando DCD

Valor de DCD (Contador FOR)	Valor de la variable Digits decodificado	Puerto que se activará.	Display encendido.
0	%0001	PORTA.0	D0
1	%0010	PORTA.1	D1
2	%0100	PORTA.2	D2
3	%1000	PORTA.3	D3

6.2 Prácticas de Laboratorio No.4.- Convertidor BCD a 7 segmentos.

Integrantes:

1. _____
2. _____
3. _____
4. _____

Objetivo general:

Implementar un convertidor de BCD a 7 segmentos el cual despliegue el resultado del incremento de un contador de décadas (conteo del 0-9). Los incrementos serán cada segundo.

Objetivos particulares:

- Conocer el funcionamiento, formas de conexión, tipos y aplicaciones del display de 7 segmentos.
- Programar aplicaciones usando el display de 7 segmentos.
- Usar nuevos comandos para programar aplicaciones: LOOKUP, DCD y DIG.
- Repasar algunos conceptos como código BDC, números hexadecimales y binarios.

Equipo:

- PC de escritorio o laptop.
- Programador PIC-600.
- Cable USB.
- Protoboard.

Material a utilizar:

- PIC16F84A
- Resistencia de 470 Ω .

- Resistencia de $100\ \Omega$.
- Resistencia de $10\ \text{K}\Omega$.
- Display de siete segmentos de cátodo o ánodo común, según su preferencia.
- Cristal oscilador de 4 MHz
- 1 push-button.
- Cables para conexión.

Desarrollo de la práctica.

1. Arme el circuito que se muestra en la *figura 57* en su protoboard.

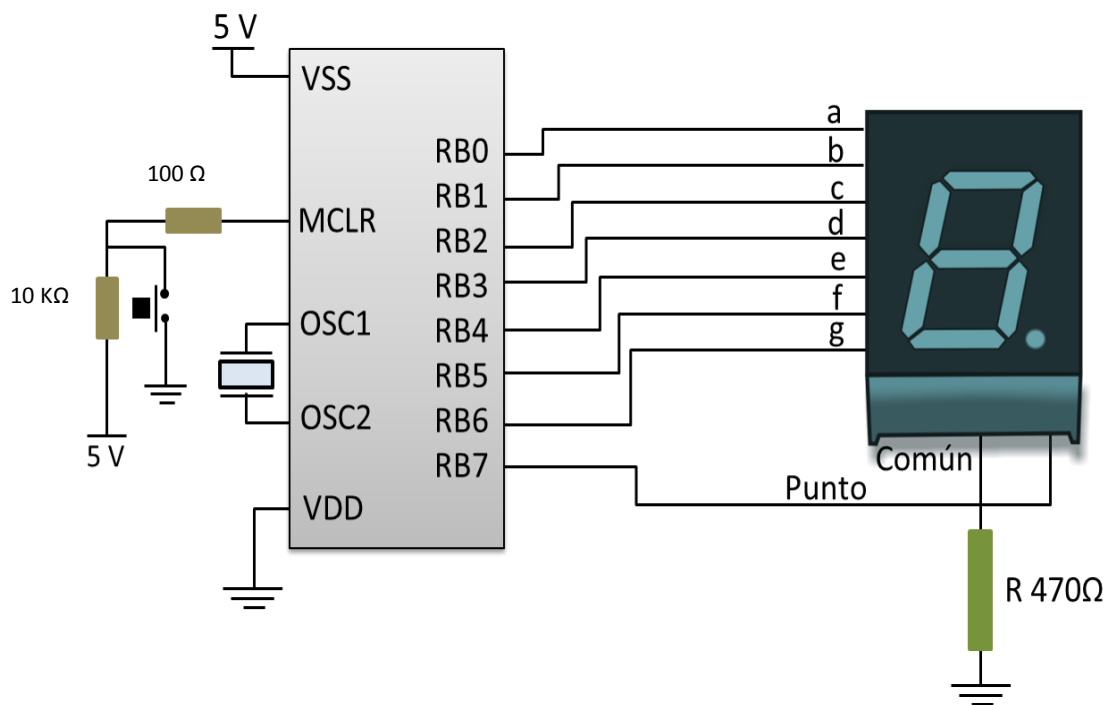


Figura 53. Diagrama del circuito para un convertidos BCD a 7 segmentos.

2. Después de armar el circuito proceda a llenar la *tabla 14* con los datos necesarios para formar cada número del display. Esta información será necesaria en la programación para formar el convertidor BCD a 7 segmentos. Considere que el PORTB.7 es el más significativo y el PORTB.0 el menos significativo. En la tabla se muestra un ejemplo del código necesario para mostrar el número “0” con un display de cátodo común. Recuerde si se usa un display de ánodo común los LED’s internos del display necesitan encenderse con “0” lógicos y los valores serán diferentes al mostrado en el ejemplo.

Tabla 14. Tabla de valores para generar el código de BCD a 7 segmentos.

Valor Decimal	Entrada BCD				Decodificador de 7 segmentos								Valor hexadecimal
	A	B	C	D	Punto	g	F	e	d	c	b	a	
	A	B	C	D	B.7	B.6	B.5	B.4	B.3	B.2	B.1	B.0	
0	0	0	0	0	1	0	1	1	1	1	1	1	\$ BF
1	0	0	0	1									\$
2	0	0	1	0									\$
3	0	0	1	1									\$
4	0	1	0	0									\$
5	0	1	0	1									\$
6	0	1	1	0									\$
7	0	1	1	1									\$
8	1	0	0	0									\$
9	1	0	0	1									\$

NOTA: Picbasic permite definir constantes numéricas en tres fases: decimal, binario y hexadecimal. Valores binarios son definidos por el prefijo “%” y los valores hexadecimales usando el prefijo “\$”. Los valores decimales se toman por defecto y no requieren prefijo. Observe la tabla 15 se muestran algunos ejemplos.

Tabla 15. Valores decimales, binarios y hexadecimales.

Valores	Descripción
100	Valor decimal 100
%100	Valor binario para el decimal 4.
\$100	Valor hexadecimal para el decimal 256.

3. Una vez obtenidos los valores del convertidor BCD-7 segmentos proceda a realizar la programación. Use los ejemplos propuestos.

Opción 1. Forma extensa sin usar el comando LOOKUP.

```
n var byte           'variable temporal
TRISB=%00000000     'puerto b configurado como salida

loop:

    n=$BF             'código del cero
    PORTB=n
    pause 500
    n=$86             'código del uno
    PORTB=n
    pause 500
    ...
    n=$EF             'código del 9
    PORTB=n
    pause 500
goto loop
```

Opción 2. Forma simplificada usando el comando LOOKUP.

```
n var byte           'contador
TRISB=%00000000     'puerto b configurado como salida

loop:
    for n=0 to 9
        lookup n, [$BF,$86,$DB,$CF,$E6,$ED,$FD,$87,$FF,$EF],PORTB
    pause 500
    next n
goto loop
```

Observe la estructura del comando LOOKUP y describa cuál es la tarea que realiza en ejemplo mostrado.

4. Una vez escrito el algoritmo proceda a cargar el programa al PIC16F84A con ayuda del PIC-600.

5. Una vez cargado el programa al microcontrolador pruebe su programa en el circuito que armó en el protoboard.

Anote sus conclusiones individualmente:

1. _____

2. _____

3. _____

4. _____

6.3 Problemas reto.

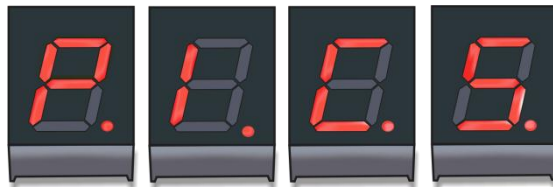
Instrucciones: Resuelve cada uno de los problemas propuestos, recuerda usar la información dada en el marco teórico y los comando propuestos para esta sección.

Use las siguientes sugerencias:

- No olvide definir cuáles serán los puertos de salida y los de entrada. Por ejemplo en el ejercicio 4, el push-button que tendrá la función de controlar el conteo ascendente y descendente tiene que ser declarado como puerto de entrada. Use el comando **TRIS** y recuerde que un valor de **1** es considerado como entrada y el valor **0** como salida.
- Use el comando **LOOKUP, DCD Y DIG** para simplificar su programación.
- Recuerde que los tiempos de multiplexado deben de ser demasiado rápidos para que nuestra retina no alcance a percibir los cambios. Se recomienda usar el comando **PAUSE** con un tiempo de 10 ms.
- Recuerde la forma de conectar físicamente los transistores, esto será muy importante para realizar el multiplexado. La conexión dependerá del tipo de transistor a usar: NPN o PNP.

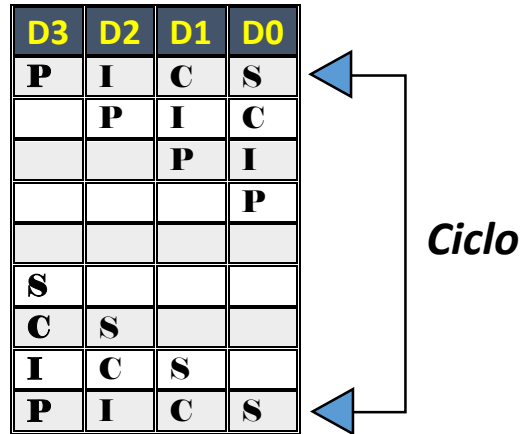
1. Anuncio estático.

Crear un programa que muestre la palabra *PICS* en 4 displays de tal manera que la palabra se pueda observar de manera estática.



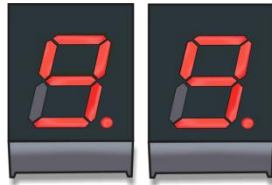
2. Anuncio en Movimiento.

Basándose en el ejercicio anterior realiza un programa que muestre la palabra PICS pero desplazándose de izquierda a derecha hasta desaparecer, luego tendrá que reaparecer en el otro extremo y volver a realizar la acción infinitas veces. Similar a un anuncio de LED'S en un estación de autobuses.



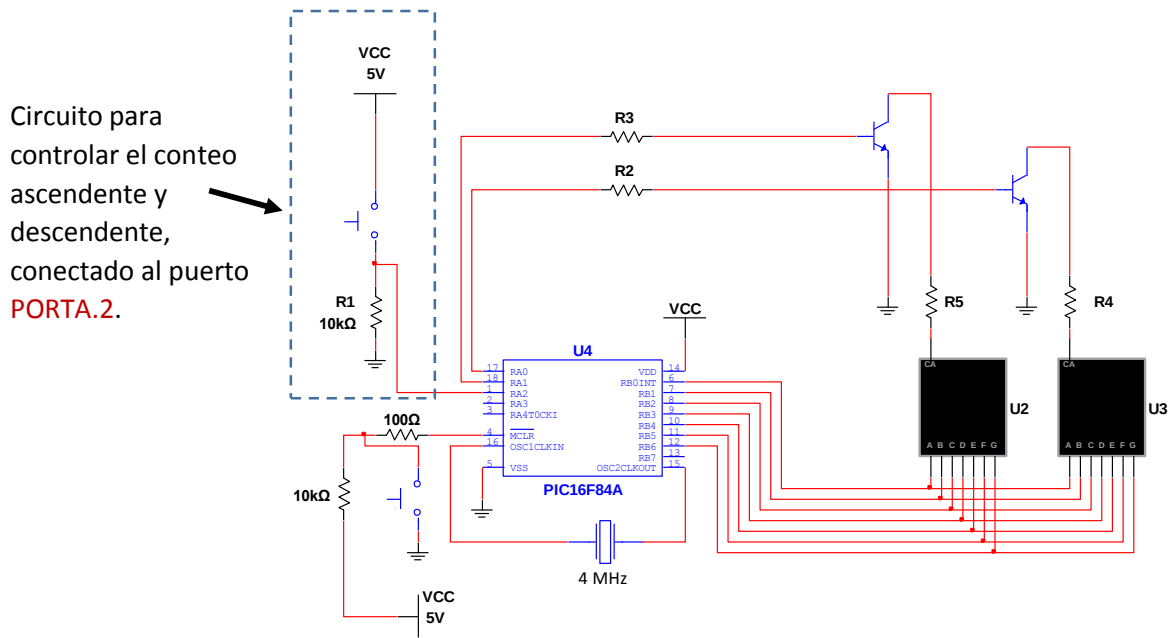
3. Contador digital de 0 a 99.

Realizar un programa el cual haga un conteo de 0-99. Puede guiarse del circuito de la figura 54 y 56 ubicadas en el marco teórico de esta sección. .



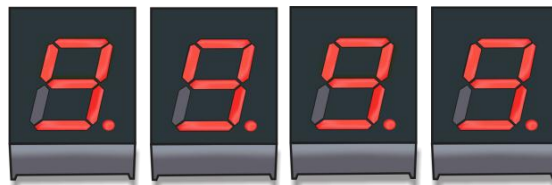
4. Contador ascendente y descendente de 0-99.

Usando un push-button como control, realice un programa el cual haga un conteo de 0-99 de forma ascendente cuando el push-button no se encuentre oprimido, y cuando este se oprima el conteo debe de realizarse de forma descendente pero a partir del valor en el que se encontraba. Si el push-button se suelta de nuevo debe continuar con el conteo ascendente y así sucesivamente. Guíese del diagrama propuesto.



5. Contador de 0 a 9999.

Realice un contador ascendente de 0-9999 usando los cuatro displays. Puede guiarse del circuito de la figura 54 y 56 ubicada en el marco teórico de esta sección.



7. Programación aplicada usando la pantalla LCD.



7.1 Marco teórico

7.1.1 La pantalla LCD 16X2

La pantalla de cristal líquido o LCD (Liquid Crystal Display) es un dispositivo micro-controlado de visualización gráfico para la representación de caracteres, símbolos e incluso dibujos (en algunos modelos). La pantalla que se usará en las prácticas de este manual es la pantalla de 16X2, es decir esta dispone

de 2 filas de a 16 caracteres cada una. Además cada carácter dispone de una matriz de 5x7 puntos (píxeles), aunque los hay de otros números y filas, es una de las más comunes. Este dispositivo está gobernado internamente por un microcontrolador que regula todos los parámetros de presentación.



Figura 54. Pantalla LCD de 16x2.

Las siguientes son sus características principales:

- Pantalla de caracteres ASCII, además de los caracteres Kanji y Griegos.
- Desplazamiento de los caracteres hacia la izquierda o la derecha.
- Proporciona la dirección de la posición absoluta o relativa del carácter.
- Memoria de 40 caracteres por línea de pantalla.
- Movimiento del cursor y cambio de su aspecto.
- Permite que el usuario pueda programar 8 caracteres.
- Conexión a un procesador usando un interfaz de 4 u 8 bits.

7.1.2 Terminales de la pantalla LCD de 16x2.

En la *figura 59* se puede observar la configuración de las terminales.

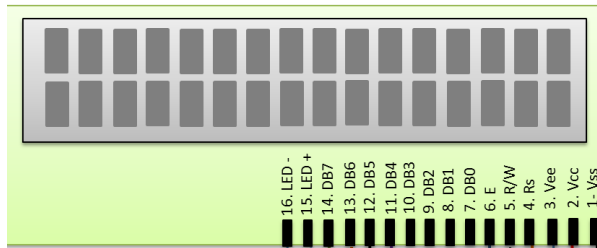


Figura 5955. Configuración de terminales de la Pantalla LCD de 16x2.

Ter	Sim	Descripción
1	Vss	Tierra del circuito
2	Vdd	Alimentación a 5v
3	Vo	Ajuste del contraste de la pantalla vara de 0 - 5v
4	RS	Selección del registro control - datos. RS = 0 - Control RS = 1 - Datos
5	R/W	Lectura o escritura en el LCD R/W = 0 - Escritura R/W = 1 - Lectura
6	E	Habilitación del LCD E = 0 - Desconectado E = 1 - Conectado
7	D0	Bit de dato 0
8	D1	Bit de dato 1
9	D2	Bit de dato 2
10	D3	Bit de dato 3
11	D4	Bit de dato 4
12	D5	Bit de dato 5
13	D6	Bit de dato 6
14	D7	Bit de dato 7
15	A	Alimentación de luz de fondo backlight puede ser 3.5v o 5v
16	K	Tierra del backlight

7.1.3 Comandos sugeridos

Locaciones de Memoria en la pantalla LCD de 16x2

Cuando se requiere mostrar un carácter en la pantalla LCD, tenemos que indicar en nuestro algoritmo de programación en que parte de la pantalla se visualizará éste. Cada una de las 16 posiciones de la pantalla tienen una dirección única de memoria (DD RAM), a la cual se puede hacer referencia y así decidir la posición del carácter. En la *tabla 16*, se pueden observar las 32 direcciones DD RAM de una pantalla de 16x2, *figura 59*. Observe que las direcciones están representados en numeración hexadecimal, sin embargo en la programación puedes usar numeración binaria y decimal.

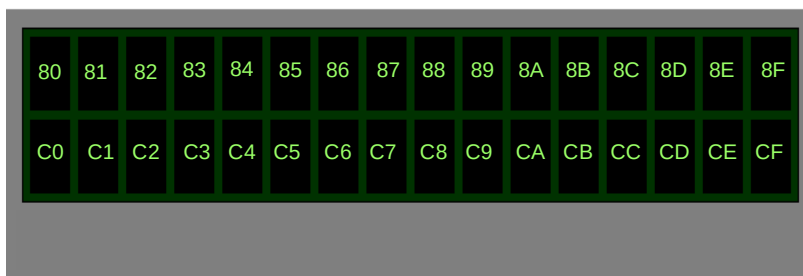


Figura 60. Localidades de memoria de una pantalla LCD de 16x2.

Tabla 15. Localidades de memoria de una pantalla LCD de 16x2.

Código de direcciones de caracteres. (Valores hexadecimales.)																
Posición en la pantalla	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Dirección DD RAM	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
Dirección DD RAM	C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF

Comando "LCDOUT"

Para mostrar caracteres en la pantalla LCD es muy sencillo basta con escribir el comando LCDOUT.

Estructura: *LCDOUT + \$fe, +dirección DD RAM+"Caracteres"*

Los valores están representados en valores hexadecimales, por lo tanto en la programación se debe anteponer de un signo de "\$".

Ejemplo:

Lcdout \$fe, \$85, "Hello" *' Muestra la palabra "Helo" a partir de la posición 6 de la pantalla.*

De igual manera los valores se pueden expresar de forma decimal.

Ejemplo:

Lcdout \$fe, 133, "Hello" *'Realiza la misma operación que la sentencia anterior.*

Otra sentencia muy útil es expresar los valores por medio de una variable. Por ejemplo la variable puede depender de un contador el cual puede ir desplazando una palabra. Dependiendo del valor en que se encuentre la variable será la posición de la palabra, idéntica a una "marquesina".

Ejemplo:

Lcdout \$fe, i, "Hello" *'Mostrará la palabra "Hello" en la posición que dependerá del valor de la variable "i".*

Comandos básicos.

PICBASIC cuenta con comandos ya predefinidos que pueden ser de ayuda al momento de programar. Algunos de los comandos más comunes se muestran en la *tabla 16*.

Tabla 16. Comandos básicos para la pantalla LCD.

INSTRUCCIÓN	FUNCIÓN
\$FE,1	Limpia el visor del LCD y coloca el cursor en la primera línea.
\$FE,2	Vuelve al inicio (comienzo primera línea)
\$FE,\$0C	Apagar el cursor
\$FE,\$0E	Subrayado del cursor activo <u> </u>
\$FE,\$0F	Parpadeo activo del cursor ☐
\$FE,\$10	Mover el cursor una posición a la izquierda
\$FE,\$14	Mover el cursor una posición a la derecha
\$FE,\$80	Mover el cursor al comienzo de la primera línea
\$FE,\$C0	Mover el cursor al comienzo de la segunda línea
\$FE, \$7	Configura para desplazamiento a la izquierda.

7.2 Prácticas de Laboratorio No.5.- La pantalla LCD.

Integrantes:

1. _____
2. _____
3. _____
4. _____

Objetivo general:

Crear aplicaciones usando la pantalla LCD de 16x2.

Objetivos particulares:

- Conocer el funcionamiento, formas de conexión, tipos y aplicaciones de una pantalla LCD de 16x2.
- Usar el comando: LCDOUT.

Equipo:

- PC de escritorio o laptop.
- Programador PIC-600.
- Cable USB.
- Protoboard.

Material a utilizar:

- 1 PIC16F84A
- 1 Resistencia de 330 Ω .
- Resistencia de 10 K Ω .
- 1 potenciómetro de 10K Ω .
- Pantalla LCD de 16x2.
- Cristal oscilador de 4 MHz
- Cables para conexión.

Desarrollo de la práctica.

1. En la figura se muestra el diagrama que deberás de armar en el protoboard para la pantalla LCD con un bus de 4 bits.

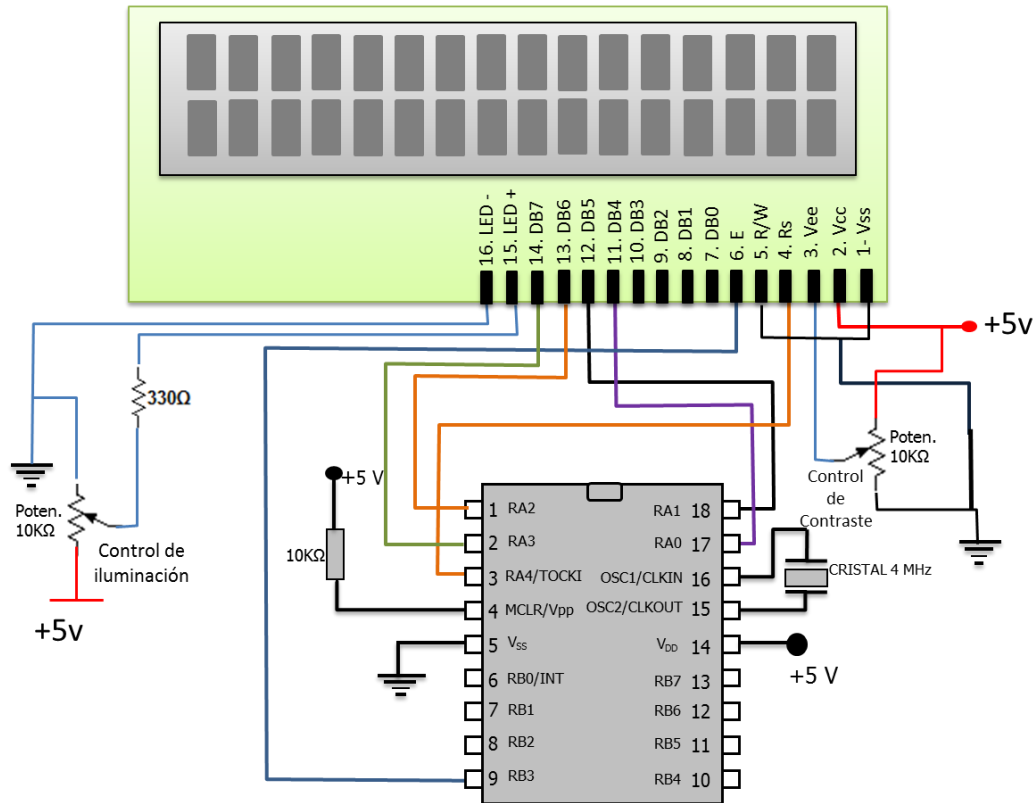


Figura 61. Conexión de 4 bits de una pantalla LCD de 16x2.

NOTA: Si cuentas con más de un protoboard, arma en uno solamente el circuito que se muestra en la figura 61 y no lo desarmes hasta finalizar el curso de Arquitectura de Sistemas Digitales, ya que lo estarás usando hasta el final de éste.

2. Captura el siguiente código en MicroCode Studio y cárgalo al PIC16F84A.

```
Pause 500      'Tiempo para que la LCD inicie
loop:
  Lcdout $fe, 1  ' Limpia la pantalla.
  Lcdout "Hello" ' Muestra la palabra «Hello»
  Pause 500
  Lcdout $fe, 1  ' Limpia la pantalla
  Lcdout "World " ' Muestra la palabra «World»
  Pause 500
  Goto loop
```

El programa anterior muestra la palabra “Hello” en la posición número uno de la pantalla LCD posteriormente se borra y muestra la palabra “World” en la misma posición, *figura 62*.

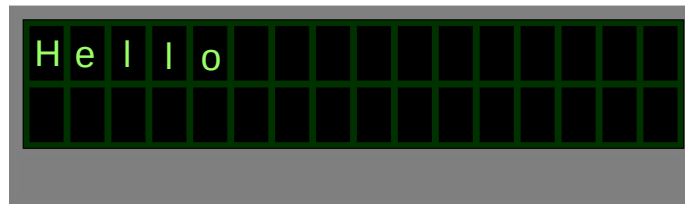


Figura 62. Primer programa en la pantalla LCD.

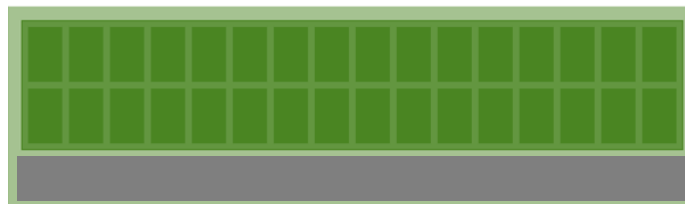
Anota tus observaciones:

3. Cambia los valores de las direcciones DD RAM de la LCD de tal manera que la palabra “Hello World” se muestre en otra posición. Luego cárgalo al PIC. Usa el siguiente código.

```
PAUSE 500      ' Tiempo de espera para que la LCD inicie
loop:  LCDOUT $fe, 1      ' Limpia la pantalla
      LCDOUT "Hello"      ' Muestra la palabra "Hello"
      PAUSE 500

      LCDOUT $fe, 1      ' Limpia la pantalla.
      LCDOUT $fe, $C0      ' C0 es la dirección DD RAM para iniciar en la
                          ' segunda línea.
      LCDOUT "World"      ' El comando mostrará la palabra "World" al
                          ' inicio de la segunda línea,
      PAUSE 500
      GOTO loop
```

4. Escribe en los cuadros de la figura la palabra “Hello Wolrd” de acuerdo a la posición que pudiste observar en pantalla.



Anota tus observaciones:

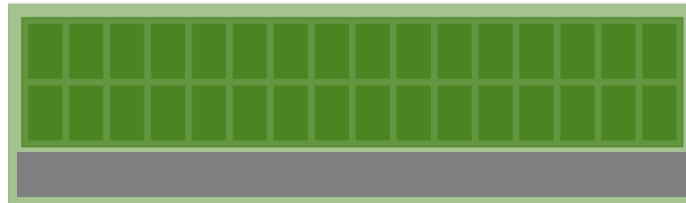
5. Nuevamente modifica el programa con los valores del siguiente código. Cárgalo nuevamente al PIC.

```
PAUSE 500      ' Tiempo de espera para que la LCD inicie
loop:  LCDOUT $fe, 1      ' Limpia la pantalla
      LCDOUT $fe, $85    ' $85 es la dirección DD RAM en hexadecimal para
                        ' iniciar en la posición 6 de la primera línea
      LCDOUT "Hello"     ' Muestra la palabra "Hello" en medio de la LCD.
      PAUSE 500

      LCDOUT $fe, 1      ' Limpia la pantalla.
      LCDOUT $fe, $CB    ' CB es la dirección DD RAM en hexadecimal para
                        ' iniciar en la posición 12 de la segunda línea.
      LCDOUT "World"     ' El comando mostrará la palabra "World" en la
                        ' parte inferior derecha.

      PAUSE 500
      GOTO loop
```

6. Escribe en los cuadros de la figura la palabra “Hello Wolrd” de acuerdo a la posición que pudiste observar en pantalla.



Anota tus observaciones:

Anote sus conclusiones individualmente:

1. _____

2. _____

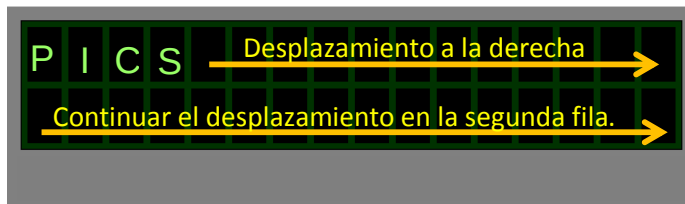
3. _____

4. _____

7.3 Problema reto.

1. Desplazamiento de una palabra en la pantalla LCD.

Realiza un programa que muestre la palabra “PICS” que se desplace desde el inicio de la primera línea hasta recorrerla por completo y luego continuar desde el inicio de la segunda hasta recorrerla de igual manera.



2. Contador de personas.

Implementa un sistema que lleve el conteo del número de personas que atraviesan entre un sensor de presencia que fabricarás con un LED infrarrojo y un fotodiodo. Similar a los dispositivos que cuentan el número de usuarios que abordan un autobús.

El sistema debe de contar con los siguientes requisitos:

1. Al iniciar el programa, en la pantalla se debe mostrar la leyenda “Bienvenidos” durante algunos segundos y luego desaparecer. Observe la posición de la palabra.



2. Seguido se debe mostrar la leyenda “personas:” y el valor del contador. Además cada vez que el sensor detecte una persona el buzzer debe de emitir un pitido corto. La distancia mínima entre el infrarrojo y el fotodiodo debe de ser de un metro.



3. Al llegar a un total de 15 personas el Buzzer debe emitir un sonido constante y en ese momento el contador se debe reiniciar a cero. Finalmente la pantalla debe de mostrar la leyenda “contador reiniciado” e iniciar de nuevo el conteo.



4. Usa el circuito de la *figura 63* para implementar el sistema.

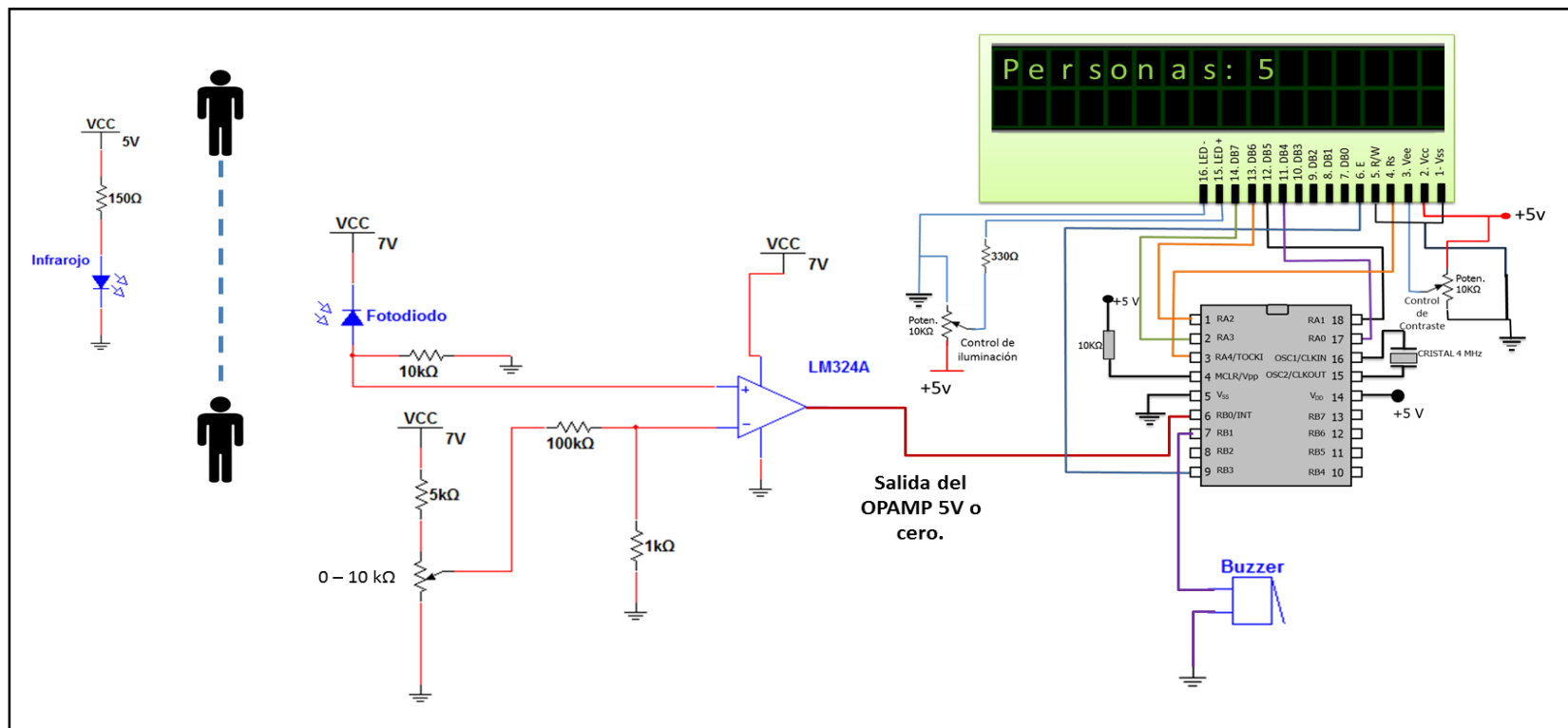


Figura 63. Diagrama de un detector de personas. .

TIPS para el armado del circuito: La fuente de 7V se usa para el OPAMP debido que este componente tiene el inconveniente de tener una caída de tensión a su salida hasta de hasta 2V. Si el OPAMP es alimentado con 5V, ocasionará que a la salida el voltaje disminuya hasta 3V o menos, presentando un inconveniente para el PIC al momento de interpretar un "0" o un "1" lógico en su puerto B.1. Observa que en el diagrama también hay una fuente de 5V para alimentar el PIC y al circuito emisor. Recordar que si se implementa un circuito con dos fuentes ambas deben de tener un mismo punto de referencia (*comúnmente llamado tierra*) para que todos los componentes operen correctamente. Sin embargo, es un poco impráctico usar dos fuentes de alimentación por lo que se recomienda emplear un regulador de voltaje lineal del tipo "7805" para bajar el nivel hasta 5V usando solamente la fuente de 7V. Puedes consultar la práctica IV "Fuente simétrica" del manual de Electrónica I para consultar sobre la conexión de reguladores de voltaje lineales.

8. Proyectos finales.



8.1 Proyectos Final No. 1.- Medición de distancia con sensor ultrasónico.

8.1.1 Marco teórico

Definición

Los ultrasónicos son antes que nada sonido exactamente igual que los que oímos normalmente, salvo que tienen una frecuencia mayor que la máxima audible por el oído humano. Ésta comienza desde unos 16 KHz y tiene un límite superior de aproximadamente 20 KHz, mientras que nosotros vamos a utilizar sonido con una frecuencia de 40 KHz. A este tipo de sonidos es a lo que llamamos Ultrasonidos.

El funcionamiento básico de los ultrasonidos como medidores de distancia se muestra de una manera muy clara en la *figura 64*, donde se tiene un receptor que emite un pulso de ultrasonido que rebota sobre un determinado objeto y la reflexión de ese pulso es detectada por un receptor de ultrasonidos:

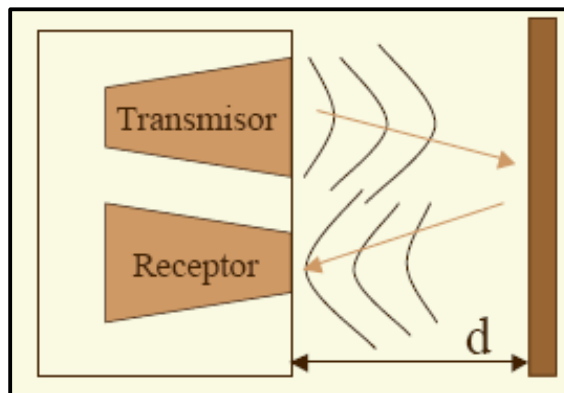


Figura 64. Funcionamiento de un sensor ultrasónico..

La mayoría de los sensores de ultrasonido de bajo coste se basan en la emisión de un pulso de ultrasonido cuyo lóbulo, o campo de acción, es de forma cónica. Midiendo el tiempo que transcurre entre la emisión del sonido y la percepción del eco se puede establecer la distancia a la que se encuentra el obstáculo que ha producido la reflexión de la onda sonora, mediante la fórmula:

$$d = \frac{1}{2} V \cdot t$$

Donde **V** es la velocidad del sonido en el aire y **t** es el tiempo transcurrido entre la emisión y recepción del pulso.

El SRF04

El sensor ultrasónico SRF04, figura, funciona emitiendo impulsos de ultrasónicos inaudibles para el oído humano. Los impulsos emitidos viajan a la velocidad del sonido hasta alcanzar un objeto, entonces el sonido es reflejado y captado de nuevo por el receptor de ultrasónicos. Lo que hace el controlador incorporado es emitir una ráfaga de impulsos y a continuación empieza a contar el tiempo que tarda en llegar el eco. Ese tiempo se traduce en un pulso de eco de anchura proporcional a la distancia a la que se encuentra el objeto, figura 65.



Figura 65. Sensor ultrasónico SRF04.

Desde un punto de vista práctico, lo que hay que hacer es mandar una señal de arranque en el pin 3 del SRF04 y después leer la anchura del impulso que nos proporciona el pin 4. El pulso de disparo tiene que tener una anchura mínima de 10 μ s. Después leemos el pulso de salida de eco y medimos su longitud que es proporcional al eco recibido. En caso de que no se produzca ningún eco, porque no se encuentra un objeto, el pulso de eco tiene una longitud aproximada de 36 ms. Hay que dejar un retardo de 10 ms desde que se hace una lectura hasta que se realiza la siguiente, con el fin de que el circuito se estabilice. El diagrama de tiempos del SRF04 se muestra en la figura 66.

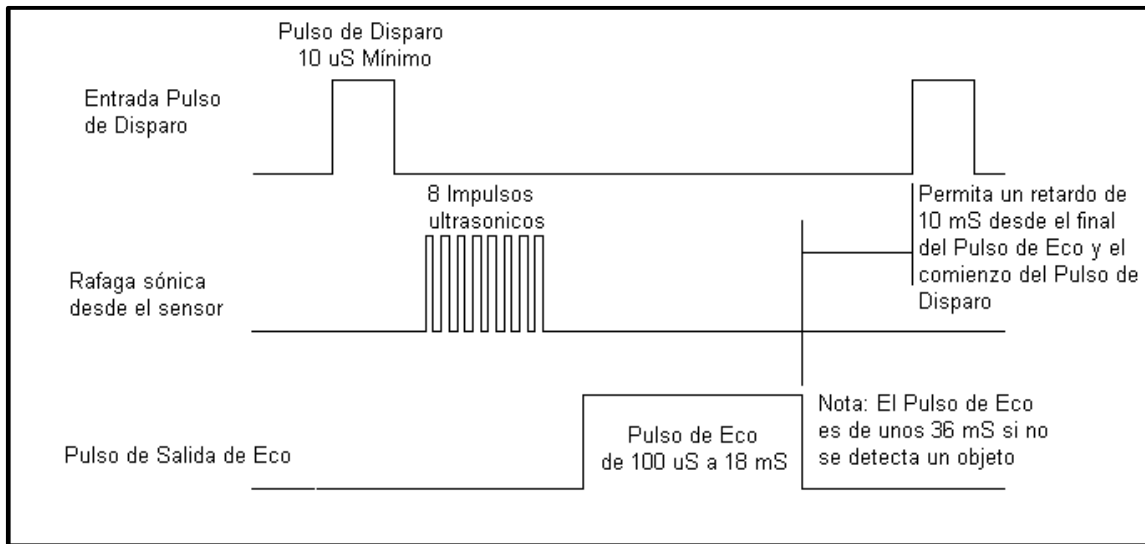


Figura 66. Diagrama de tiempos del sensor SRF04.

Características técnicas

El sensor cuenta con las características mostradas en la tabla 17.

Tabla 17. Características técnicas del sensor SRF04

Características técnicas de SRF04	
Rango	3 cm a 3 m
Sensibilidad	Detecta un palo de escoba a 3m.
Alimentación	5 V a 30 mA
Frecuencia	40 KHz
Pulso de disparo	10 μ s min. TTL
Puso del Eco	100 μ s – 18 ms
Retardo entre pulsos	10 ms mínimo.
Peso	10 gr
Tamaño	43 mm x 20 mm x 17 mm

La *figura 67* muestra la configuración de sus terminales (vista desde abajo).

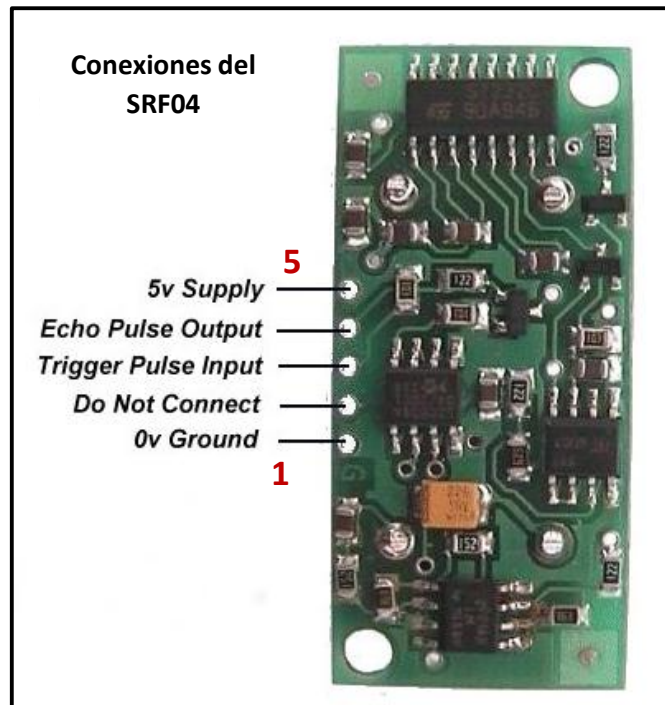


Figura 67. Terminales del sensor SRF04.

8.1.2 Comandos sugeridos

Para los ejercicios de esta sección se recomienda aplicar los siguientes comandos:

Comando “PULSOUT”

Éste comando genera un pulso hacia un pin. Servirá para mandar la señal de disparo o *trigger* al sensor SRF04 en su terminal número 3 (***trigger pulse input***), el cuál le dará la pauta para que el sensor empiece a enviar los pulsos ultrasónicos para realizar la medición de distancia. Su estructura es la siguiente:

Estructura: *PULSOUT numero de pin, periodo en microsegundos*

Ejemplo:

```
PULSOUT PORTB.5, 100
```

‘Genera un pulso de disparo de 100 μ s (1 ms) por el PORTB.5 del PIC.

Éste comando mide el ancho de pulso en un pin. Medirá el ancho del pulso de salida del SRF04 el cuál es proporcional a la distancia medida, el pulso de salida se medirá en la terminal 4 (*Echo pulse output*) del sensor. Su estructura es la siguiente:

Estructura: PULSIN numero de pin, 0/1, variable donde se guardará

Ejemplo 1:

PULSIN PORTB.4,1,dato ‘Mide el ancho de un pulso alto (**valor 1**) en el
PORTB.4 y almacena la información en la variable “dato”

Ejemplo2:

PULSIN PORTB.4,0,dato ‘Mide el ancho de un pulso bajo (**valor 0**) en el
PORTB.4 y almacena la información en la variable “dato”

Integrantes:

1. _____
2. _____
3. _____
4. _____

Objetivo general:

Implementar un circuito de medición de distancia usando un sensor ultrasónico.

Objetivos particulares:

- Aplicar los conocimientos obtenidos a lo largo del curso.
- Estudiar los comandos PULSIN y PULSOUT.
- Estudiar el sensor ultrasónico SRF04.
- Aplicación de un medidor de distancia usando un sensor ultrasónico.

Equipo:

- PC de escritorio o laptop.
- Programador PIC-600.
- Sensor ultrasónico SRF04.
- Protoboard.

Material a utilizar:

- PIC16F84A
- Buzzer de 4 KHz.
- Pantall LCD.
- Resistencia de 10 K Ω .
- Cristal oscilador de 4 MHz
- Cables para conexión.

Conclusiones y comentarios por equipo:

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

8.2 Proyectos Final No. 2.- Cisterna-Tinaco

8.2.1 Marco teórico

Comandos sugeridos.

Condicional IF...THEN...ENDIF

En el siguiente ejemplo se observa la estructura del IF..THEN.

Ejemplo:

```
IF porta.0 = 1 THEN      ' condicional IF
    high portb.4
    pause 1000
    low portb.4
ENDIF                    ' Se Cierra el condicional con la palabra ENDIF
```

Condicional IF...THEN...ELSE...ENDIF

Ejemplo:

```
IF porta.0 = 1 THEN      ' condicional IF
    HIGH portb.4
    PAUSE 1000
    LOW portb.4
ELSE                      'Sentencia que se ocupa si la primera es faltas.
    HIGH portb.5
    PAUSE 1000
    LOW portb.5
ENDIF                    ' Se Cierra el condicional con la palabra ENDIF
```

Operadores lógicos

Puedes hacer uso de operadores lógicos para implementar tu sistema.

Operador	Descripción
AND o &&	AND lógico
OR o 	OR lógico
XOR o ^	OR exclusivo lógico
NOT AND	NAND logico
NOT OR	NOR logico
NOT XOR	NXOR logico

Condicional WHILE...WEND

Ejemplo:

```

WHILE porta.0 = 1    ' condicional WHILE
    HIGH portb.4
    PAUSE 1000
    LOW portb.4
WEND                ' Se Cierra el condicional con la palabra WEND
  
```

Integrantes:

1. _____
2. _____
3. _____
4. _____

Objetivo general:

Implementar un sistema de control automático para controlar un proceso Cisterna-tinaco.

Objetivos particulares:

- Aplicar los conocimientos obtenidos a lo largo del curso.

Equipo:

- PC de escritorio o laptop.
- Programador PIC-600.
- Protoboard.

Material a utilizar:

- PIC16F84A
- Buzzer de 4 KHz.
- Pantall LCD.
- 2 Resistencias de 330Ω .
- 2 LED's del color deseado.
- 1 fotorresistencia de $1M\Omega$.
- 1 sensor de humedad (el que fabricaste en tu curso de electrónica I)
- 1 Resistencia de $10 K\Omega$.
- 1 Cristal oscilador de 4 MHz
- 8 switches o push-buttons (lo que se desee para simular entradas de sensores).
- Cables para conexión.

Desarrollo del Proyecto

1. El siguiente problema consiste en un sistema basado en una cisterna y un tinaco. Cada uno de los depósitos de agua cuenta con sensores que indican el nivel de líquido. Además cuenta con un sensor de luz (fotorresistencia) que ayudara a detectar el día y la noche, por otro lado está el sensor de humedad que detectara si el ambiente es húmedo o seco. Analiza la imagen para entender cada uno de los dispositivos que conforman el sistema y observa la función que debe de desempeñar cada dispositivo en la *tabla 18*. Puedes guiarte del esquema de la *figura 69*.

Tabla 18. Características técnicas del sensor SRF04

Dispositivo	Función	Estado	
		0	1
Sensor de nivel S1	Indica si la cisterna contiene agua.	Vacía.	Con agua.
Sensor de nivel S2	Indica si el tinaco está vacío.	Vacía.	Con agua.
Sensor de nivel S3	Indica si el tinaco está lleno.	Con agua pero no lleno.	Al máximo.
Sensor de luz L	Detectará la iluminación (si es de noche o es de día).	Noche.	Día.
Sensor de Humedad H	Detectará la humedad del ambiente (húmedo o seco)	Seco.	Húmedo.
Bomba B1	Encargada de suministrar el agua hacia el tinaco.	Apagada	Encendida
Bomba B2	Encargada del sistema de riego.	Apagada	Encendida.

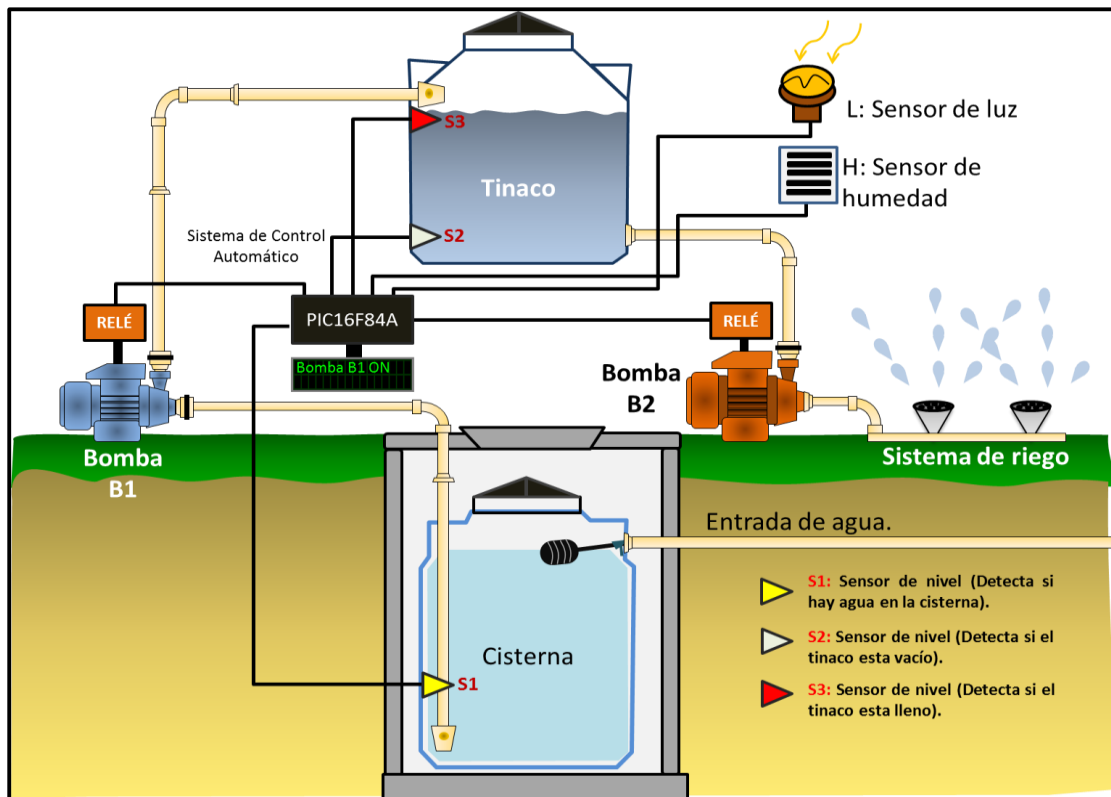


Figura 69. Esquema del sistema cisterna-tinaco.

Los sensores puedes simularlos con interruptores ya sea del tipo push-button o interruptores de un polo o dos polos. Puedes usar una fotorresistencia para el sensor de luz. Además puedes sustituir las bombas con LED's, motores de CD, o incluso con buzzers.

Requisitos del sistema.

Bomba B1.

- si $S1=1$ y $S2=0$ entonces $B1=1$. B1 encenderá si hay agua en la cisterna y si el tinaco está vacío.
- Si $S3=1$ entonces $B1=0$. La bomba B1 se apagará hasta que el tinaco este lleno.

Bomba B2.

- Si $S2=1$, es decir hay agua en el tinaco y además:
 1. $H=0$ indicando que el ambiente se encuentra seco y además $L=0$ indicando la noche, el sistema de riego debe de entrar en operación, es decir la bomba $B2$ estaría en operación.
 2. La bomba $B2$ debe de apagarse si cualquiera de los tres sensores anteriores arroja un valor diferente.

Apóyate de la *tabla 19* de la verdad para ver todos los posibles estados que debe cumplir el sistema.

Tabla 19. Condiciones que debe cumplir el sistema Cisterna-Tinaco.

S1	S2	S3	B1	Descripción
0	0	0	0	- Todos los sensores se encuentran en estado “Apagado”, en este momento el sistema debe de indicar en la pantalla LCD que los depósitos se encuentran vacíos. - Bombas apagadas.
0	0	1	X	- Condición de error. El sensor “S2” no puede indicar vacío y el sensor “S3” lleno. En este estado el sistema debe de mostrar una condición de error en la LCD. - Bombas apagadas.
0	1	0	0	- El sensor S2 indica que hay agua en el tinaco, por lo tanto, si el sensor de Humedad (H) indica seco y el de luz (L) indica que es de noche la bomba B2 debe de encender, de lo contrario ninguna bomba debe de estar encendida. - Si el sistema de riego entra en operación la LCD debe mostrar ese estado. - Bomba B1 Apagada
0	1	1	0	- Este estado indica que la cisterna está vacía y el tinaco está lleno. Sin embargo nuevamente si el sensor H indica seco y el de L indica que es de noche la bomba B2 debe de encender. - Si el sistema de riego entra en operación la LCD debe mostrar ese estado. - Bomba B1 apagada.
1	0	0	1	- Este estado indica que la cisterna tiene agua y el tinaco está vacío, por lo tanto la bomba B1 debe de entrar en funcionamiento para llenar el tinaco. - La bomba B1 se apagará hasta que el sensor S3 indique que el tinaco se ha llenado. - Si el sistema de riego entra en operación la LCD debe mostrar ese estado. - Bomba B1 encendida.
1	0	1	X	- Condición de error. El sensor “S2” no puede indicar vacío y el sensor “S3” lleno. En este estado el sistema debe de mostrar una condición de error en la LCD. - Bombas apagadas.
1	1	0	0	- Este estado indica que la cisterna y el tinaco tienen agua. Por lo tanto B1=0 y B2 dependerá nuevamente si H indica seco y L de noche. - Si el sistema de riego entra en operación la LCD debe mostrar ese estado. - B1 apagada.
1	1	1	0	- Este estado indica que todos los depósitos están llenos, por lo tanto la LCD debe de mostrar esta condición. - Nuevamente B1=0 y B2 dependerá nuevamente si H indica seco y L de noche. - Si el sistema de riego entra en operación la LCD debe mostrar ese estado. - Bomba B1 apagada.

Diagrama del circuito.

Guíate del diagrama de la *figura 70* para resolver el proyecto.

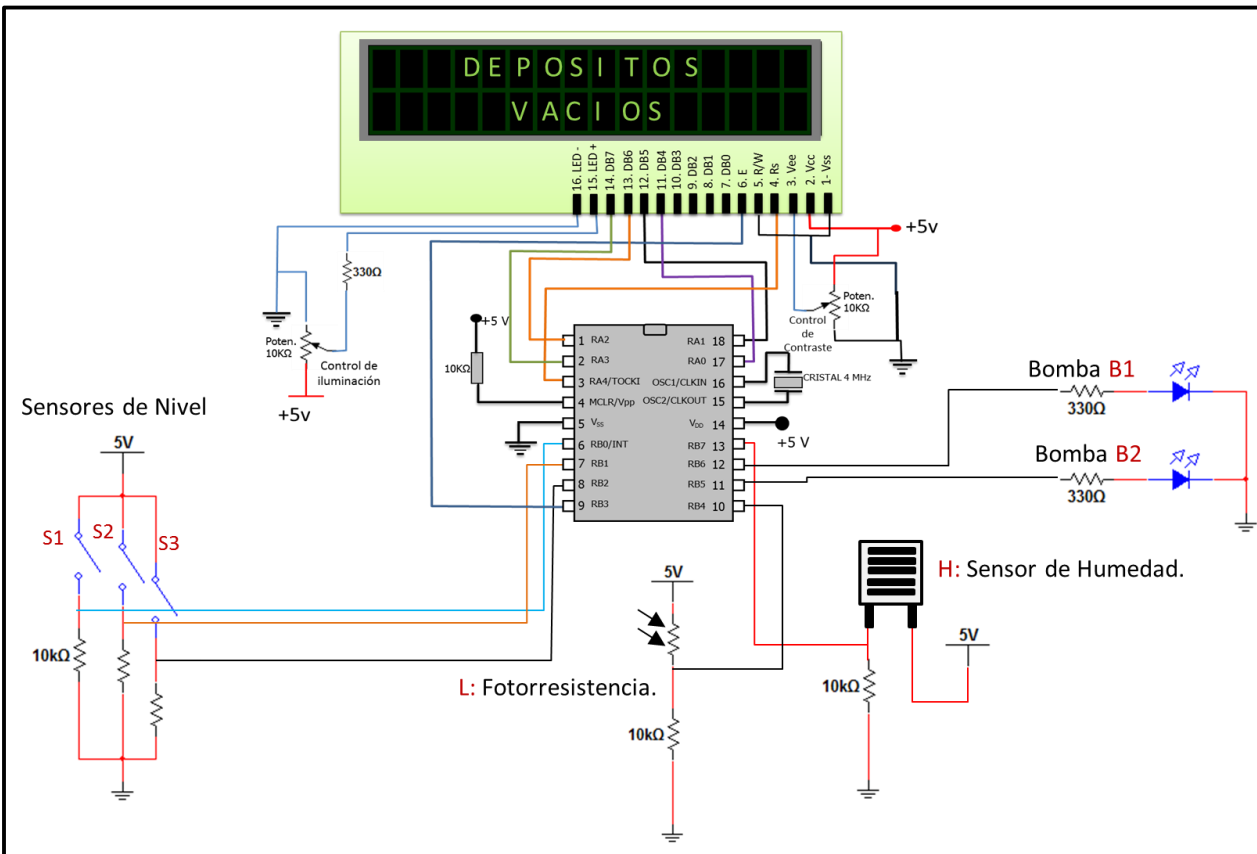


Figura 70. Diagrama del sistema Cisterna-Tinaco. .

Imprime el código de tu programa e inclúyelo en el reporte. Todo el programa debe de estar bien comentado.

Conclusiones y comentarios por equipo:

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

8.3 Proyectos Final No.3.- Cerradura Electrónica

8.3.1 Marco teórico

Comandos sugeridos

Arreglos (Arrays)

Los arreglos de variables pueden ser creados en una manera similar a las variables.

Estructura: *Etiqueta + VAR + tamaño + [Número de elementos]*

Ejemplo:

Sharks **VAR** byte [10] *'Almacena 10 elementos del tipo byte.*

Fish **VAR** bit[8] *'Almacena 10 elementos del tipo bit.*

Condicional IF...THEN...ENDIF

En el siguiente ejemplo se observa la estructura del IF..THEN.

Ejemplo:

IF porta.0 = 1 **THEN** *' condicional IF*

 high portb.4

 pause 1000

 low portb.4

ENDIF *' Se Cierra el condicional con la palabra ENDIF*

Condicional IF...THEN...ELSE...ENDIF**Ejemplo:**

```
IF porta.0 = 1 THEN      ' condicional IF
    HIGH portb.4
    PAUSE 1000
    LOW portb.4
ELSE                      'Sentencia que se ocupa si la primera es faltas.
    HIGH portb.5
    PAUSE 1000
    LOW portb.5
ENDIF                    ' Se Cierra el condicional con la palabra ENDIF
```

Integrantes:

1. _____
2. _____
3. _____
4. _____

Objetivo general:

Diseñar un sistema basado en el micro-controlador que haga la función de una cerradura electrónica, la cual funcionará a base de claves de acceso implementando el circuito propuesto.

Objetivos particulares:

- Aplicar los conocimientos obtenidos a lo largo del curso.

Equipo:

- PC de escritorio o laptop.

- Programador PIC-600.
- Cable USB
- Cable serial tipo DB9.
- Protoboard.

Material a utilizar:

- PIC16F84A
- 4 push buttons.
- Pantall LCD.
- Resistencia de 10 K Ω .
- Cristal oscilador de 4 MHz
- Cables para conexión.
-

Desarrollo del Proyecto

1. Diseñar un sistema basado en el micro-controlador para una cerradura electrónica que cumpla las siguientes especificaciones:

- 1.-Se programe al inicio (vía el puerto serial) un código de acceso de cuatro letras (solo A,B,C,D en cualquier combinación). Se empleara el LCD para visualizar el código programado (“Programar el código de acceso:”).
- 2.- Una vez programado el código de acceso, el sistema podrá ser activado por el teclado (4 push-button) siempre y cuando coincida con el código programado vía serial.
- 3.-Se deberá visualizar la solicitud de código de acceso en el LCD (“Introducir el código para accesar”)
- 4.- Cuando coincida el código introducido por los push-buttons, activar el relevador y el buzzer.

2. Diseña e implementa el circuito para cumplir con el reto, aplicando los conocimientos obtenidos a lo largo del curso. Guíate del esquema de la *figura 71*.

2. Los 4 push-buttons representan las letras (A, B, C, D), para poder introducir la clave de acceso correcta que has ingresado vía serial.

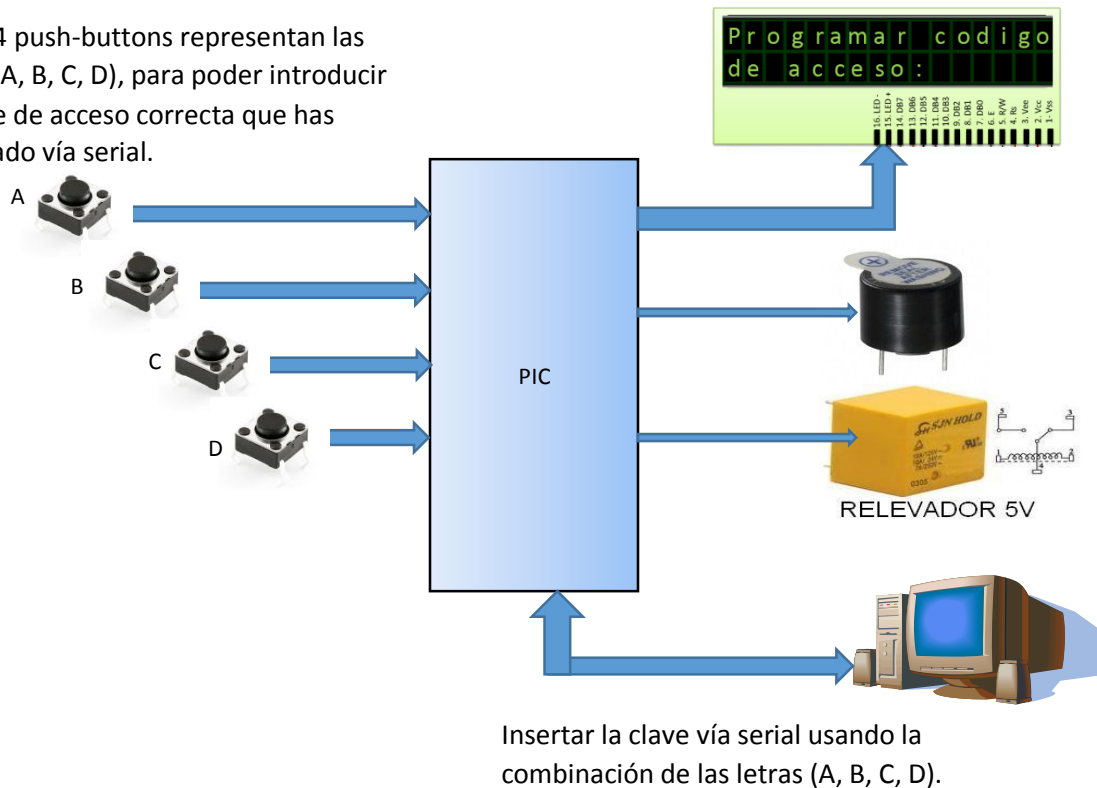


Figura 71. Esquema de la cerradura electrónica. . .

3. Imprime el código de tu programa e inclúyelo en el reporte. Todo el programa debe de estar bien comentado.

Conclusiones y comentarios por equipo:

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.